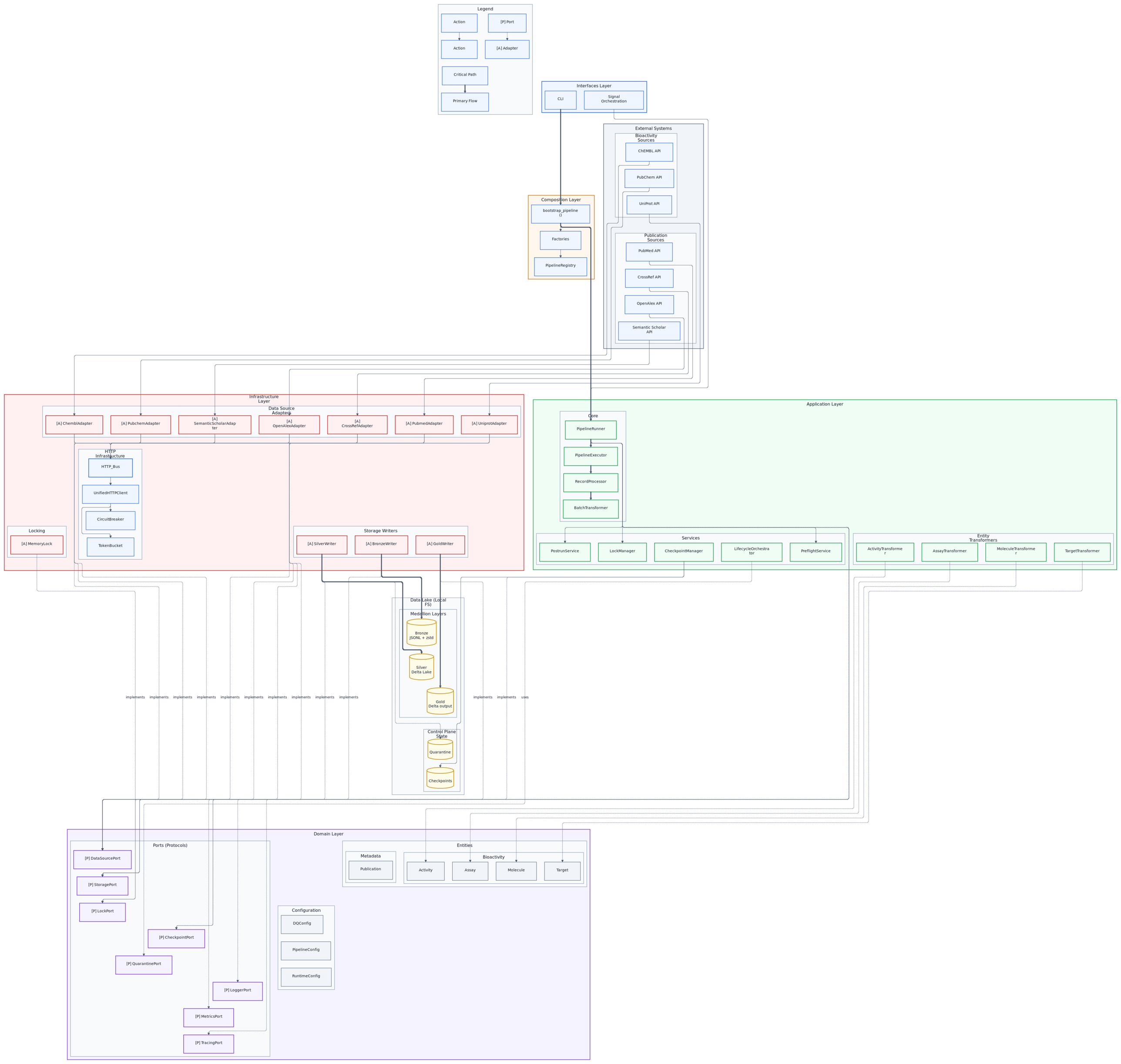
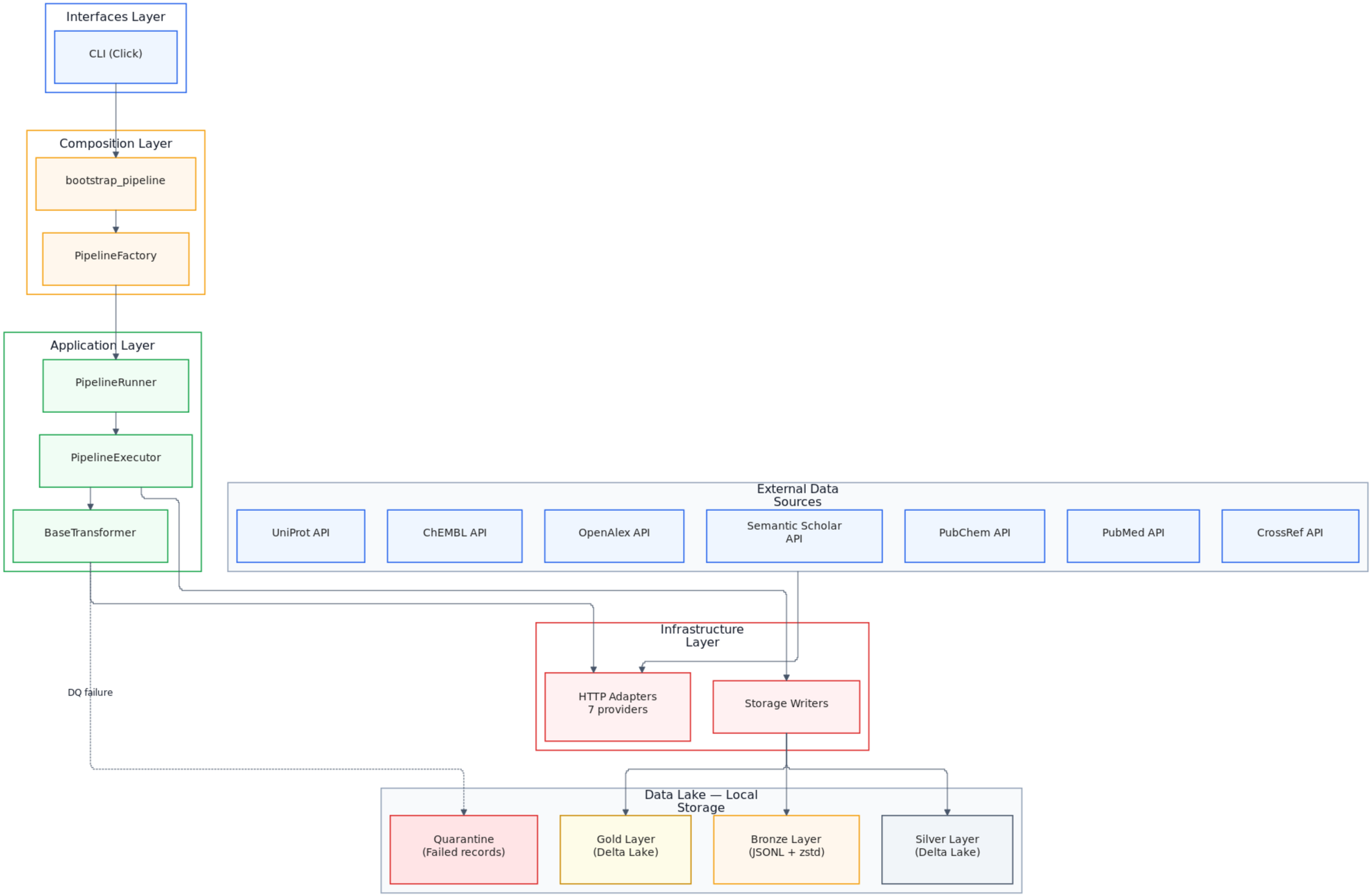

Legend
Action
[P] Port
Action
[A] Adapter
Critical Path
Primary Flow
Interfaces Layer
CLI
Signal Orchestration
External Systems
Bioactivity Sources
ChEMBL API
PubChem API
UniProt API
Publication Sources
PubMed API
CrossRef API
OpenAlex API
Semantic Scholar API
Composition Layer
bootstrap_pipeline ()
Factories
PipelineRegistry
Infrastructure Layer
Data Source Adapters
[A] ChemblAdapter
[A] PubchemAdapter
[A] SemanticScholarAdapter
[A] OpenAlexAdapter
[A] CrossRefAdapter
[A] PubmedAdapter
[A] UniprotAdapter
HTTP Infrastructure
HTTP_Bus
UnifiedHTTPClient
CircuitBreaker
TokenBucket
Locking
[A] MemoryLock
Storage Writers
[A] SilverWriter
[A] BronzeWriter
[A] GoldWriter
Application Layer
Core
PipelineRunner
PipelineExecutor
RecordProcessor
BatchTransformer
Services
PostrunService
LockManager
CheckpointManager
LifecycleOrchestrator
PreflightService
Entity Transformers
ActivityTransformer
AssayTransformer
MoleculeTransformer
TargetTransformer
Data Lake (Local FS)
Medallion Layers
Bronze JSONL + zstd
Silver Delta Lake
Gold Delta output
Control Plane State
Quarantine
Checkpoints
implements
implements
implements
implements
implements
implements
implements
implements
implements
implements
implements
implements
uses
Domain Layer
Ports (Protocols)
[P] DataSourcePort
[P] StoragePort
[P] LockPort
[P] CheckpointPort
[P] QuarantinePort
[P] LoggerPort
[P] MetricsPort
[P] TracingPort
Entities
Metadata
Publication
Bioactivity
Activity
Assay
Molecule
Target
Configuration
DQConfig
PipelineConfig
RuntimeConfig


Interfaces Layer
CLI (Click)
Composition Layer
bootstrap_pipeline
PipelineFactory
Application Layer
PipelineRunner
PipelineExecutor
BaseTransformer
External Data Sources
UniProt API
ChEMBL API
OpenAlex API
Semantic Scholar API
PubChem API
PubMed API
CrossRef API
Infrastructure Layer
HTTP Adapters 7 providers
Storage Writers
DQ failure
Data Lake — Local Storage
Quarantine (Failed records)
Gold Layer (Delta Lake)
Bronze Layer (JSONL + zstd)
Silver Layer (Delta Lake)

Lineage Tracking

External Sources

PubChem API

OpenAlex API

UniProt API

ChEMBL API

Semantic Scholar API

CrossRef API

PubMed API

Bronze Layer

BronzeWriter

Bronze Storage

🗂️ Format: JSONL + zstd
⏱️ Retention: 90 days
✅ Validation: None
📍 Path: bronze / v1 / {provider} / {entity} / {date} /

Silver Layer

SilverWriter

Silver Storage

🗂️ Format: Delta Lake
⏱️ Retention: Permanent
✅ Validation: Pandera (soft)\n🔄 Strategy: Merge / Upsert\n🔑 Key: content_hash

Gold Layer

GoldWriter

Gold Storage

🗂️ Format: Delta / Parquet
⏱️ Retention: Permanent
✅ Validation: Pandera (strict)\n🔄 Strategy: SCD2 / Append

Data Quality Branch

DQ Validator

⚠️ Soft: >5% errors
❌ Hard: >20% errors

❌ Fail

Quarantine

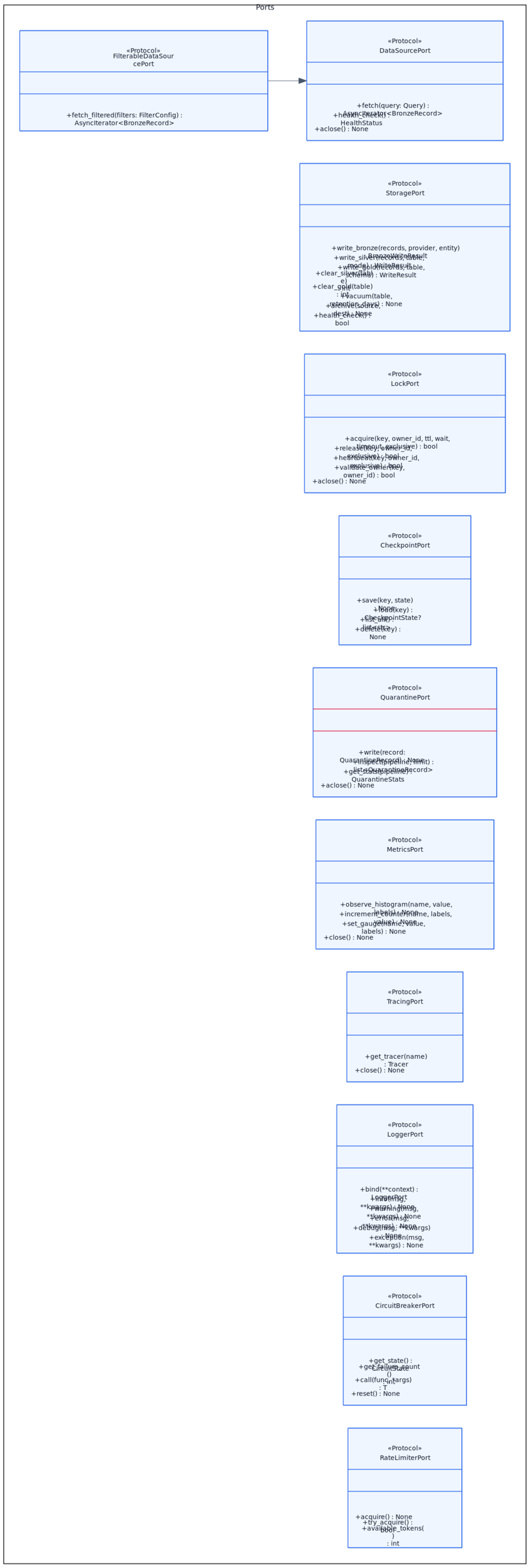
LineageLog

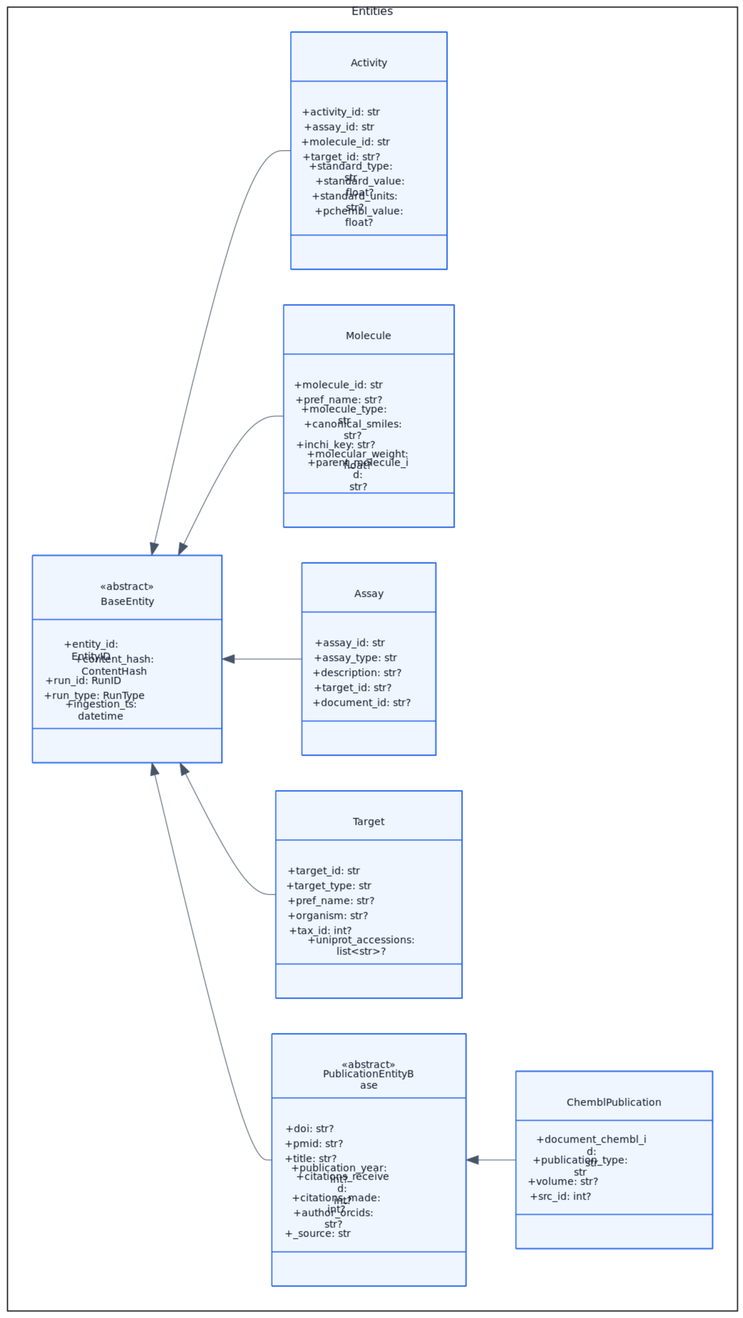
transform_version

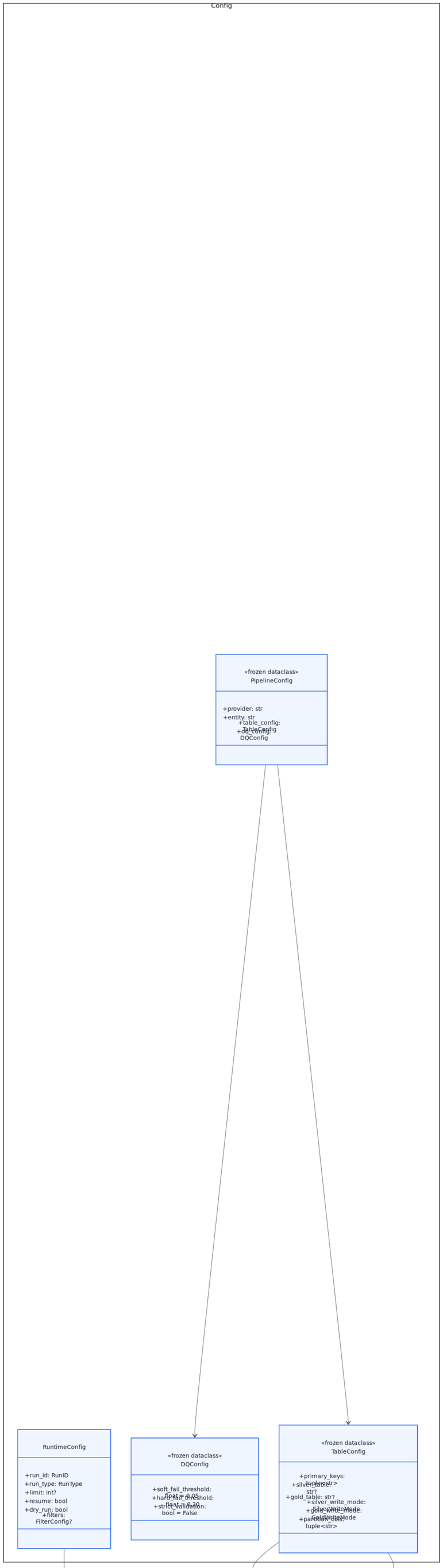
✅ Pass

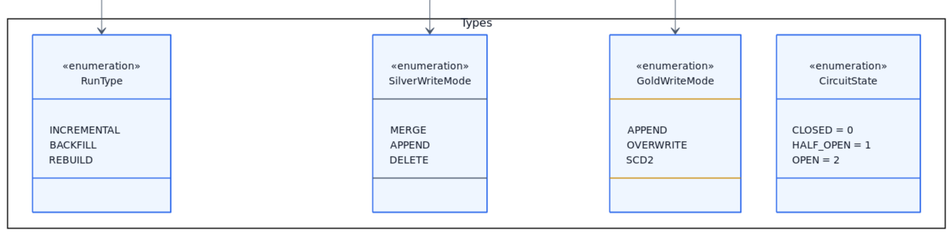
batch_id

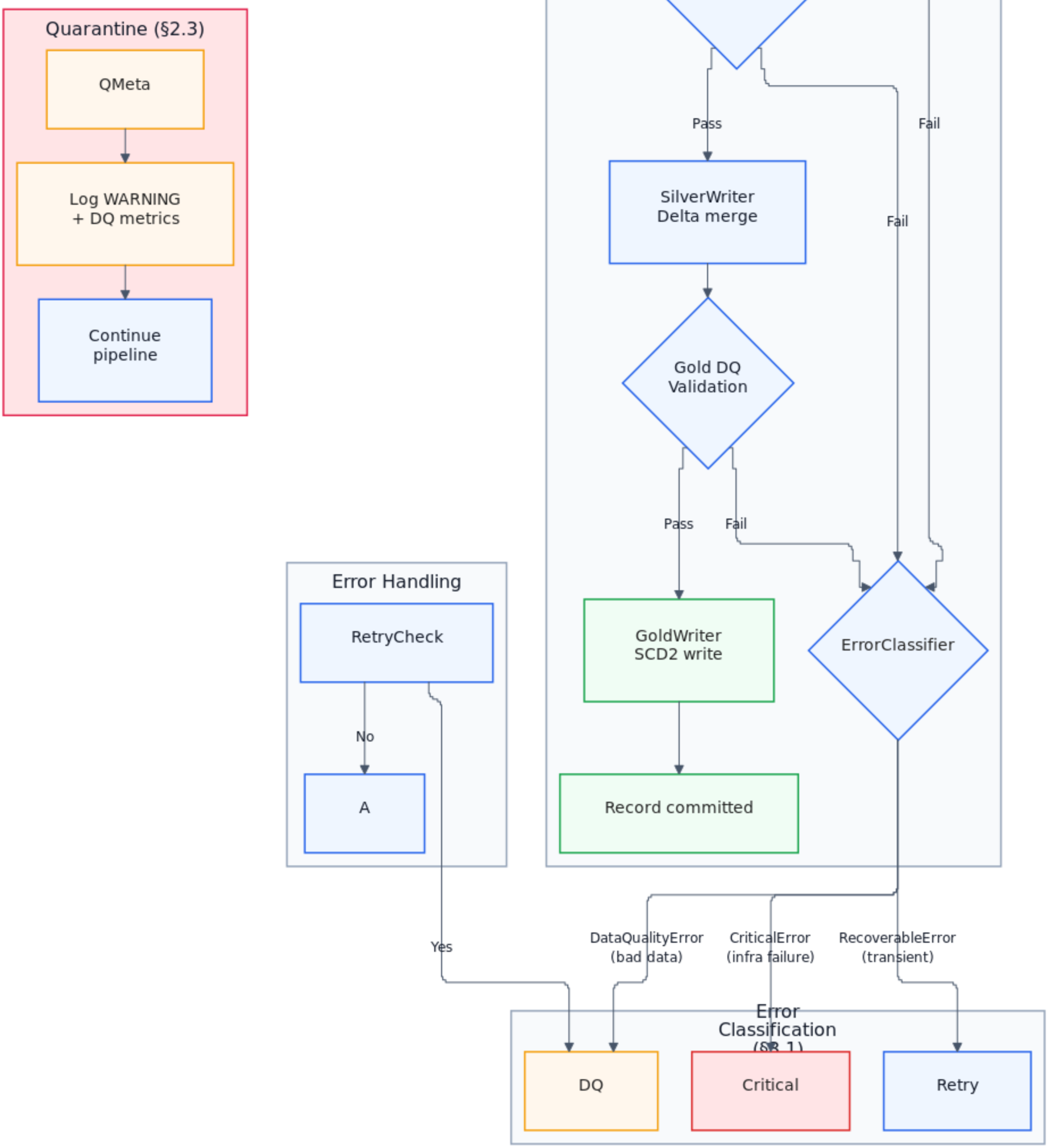
CLI
bootstrap_pipeline()
PipelineRunner
LockManager
PreflightService
PipelineExecutor
DataSourceAdapter
RecordProcessor
Bronze/Silver/Gold Writers
PostrunService
CheckpointManager
run(pipeline_name, config)
create(services, config)
Phase 1: Lock Acquisition
acquire_lock(resource, owner_id, ttl=90s)
LockResult(acquired=true)
start_heartbeat(interval=30s)
Phase 2: Preflight Checks
run_checks()
health_check()
HealthStatus.HEALTHY
PreflightResult(passed=true)
Phase 3: ETL Execution
execute(pipeline)
loop
[For each batch]
fetch(query)
AsyncIterator[BronzeRecord]
process_batch(records)
write_bronze(records)
BronzeWriteResult
transform(records)
normalize, compute hash, add metadata
merge_silver(records, primary_key)
validate_owner()
valid
WriteResult
write_gold(records, schema)
validate_owner()
valid
WriteResult
BatchResult
Every 30s
heartbeat()
ExecutionResult
Phase 4: Postrun & Cleanup
run_checks(metrics)
DQ threshold validation
PostrunResult
delete(pipeline_name)
deleted
release_lock()
released
RunResult(success=true, metrics)

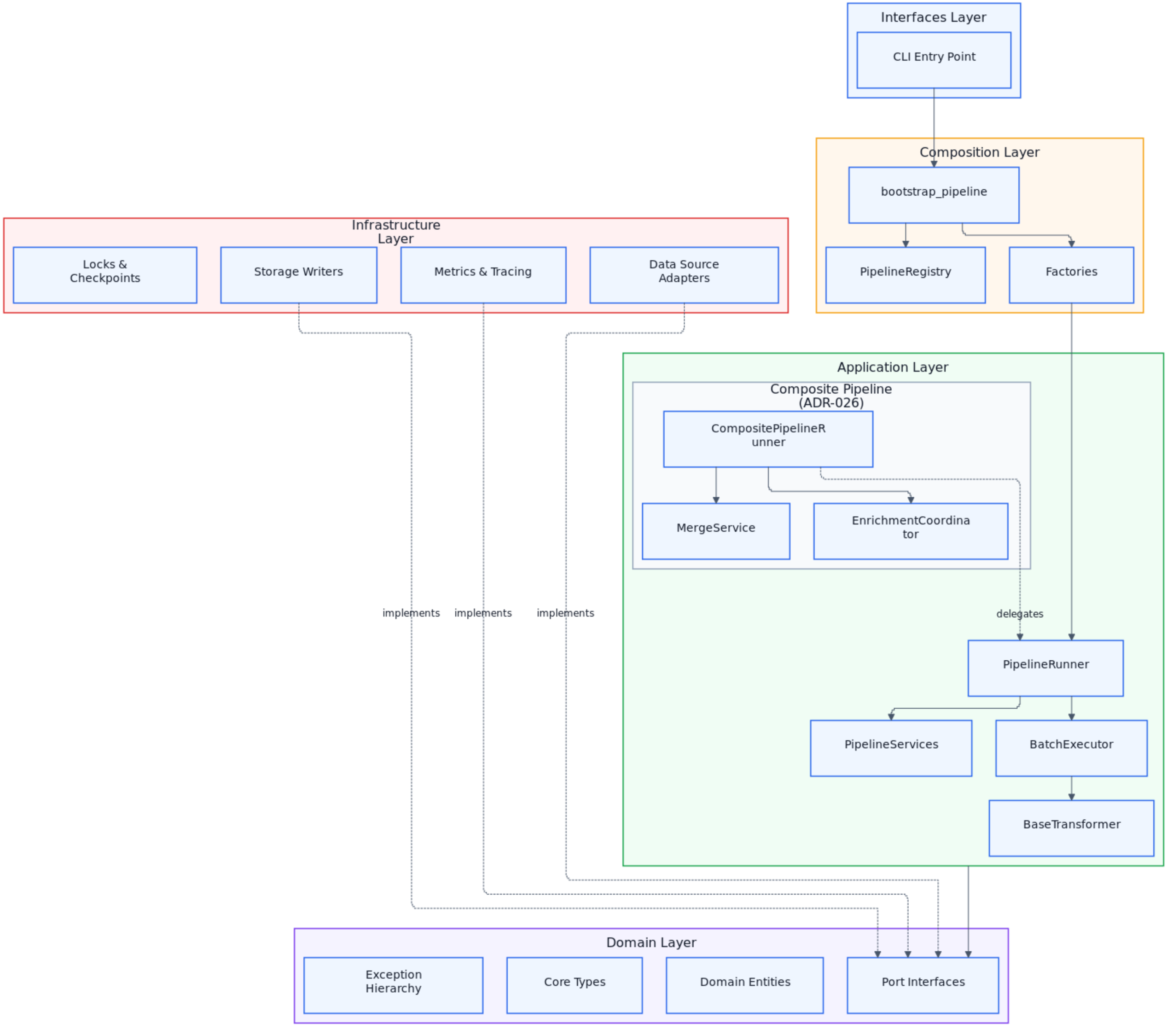

Ports
«Protocol»
FilterableDataSourcePort
+fetch_filtered(filters: FilterConfig) : AsyncIterator<BronzeRecord>
«Protocol»
DataSourcePort
+fetch(query: Query) : AsyncIterator<BronzeRecord>
+health_check() : HealthStatus
+aclose() : None
«Protocol»
StoragePort
+write_bronze(records, provider, entity) : BronzeWriteResult
+write_silver(records, table, mode) : WriteResult
+write_gold(records, table, schema) : WriteResult
+clear_silver(table)
+clear_gold(table)
+vacuum(table, retention_days) : None
+delete_source(source, dest) : None
+health_check() : bool
«Protocol»
LockPort
+acquire(key, owner_id, ttl, wait, timeout, exclusive) : bool
+release(key, owner_id, exclusive) : bool
+heartbeat(key, owner_id, exclusive) : bool
+validate_owner(key, owner_id) : bool
+aclose() : None
«Protocol»
CheckpointPort
+save(key, state) : None
+load(key) : CheckpointState?
+list_keys() : list<str>
+delete(key) : None
«Protocol»
QuarantinePort
+write(record: QuarantineRecord) : None
+write_batch(records) : int
+get_stats(pipeline) : QuarantineStats
+aclose() : None
«Protocol»
MetricsPort
+observe_histogram(name, value, labels)
+increment(name, labels, value) : None
+set_gauge(name, value, labels) : None
+close() : None
«Protocol»
TracingPort
+get_tracer(name) : Tracer
+close() : None
«Protocol»
LoggerPort
+bind(**context) : LoggerPort
+debug(msg, **kwargs) : None
+info(msg, **kwargs) : None
+warning(msg, **kwargs) : None
+error(msg, **kwargs) : None
+exception(msg, **kwargs) : None
«Protocol»
CircuitBreakerPort
+get_state() : CircuitState
+get_failure_count() : int
+call(func, *args) : T
+reset() : None
«Protocol»
RateLimiterPort
+acquire() : None
+try_acquire() : bool
+available_tokens() : int

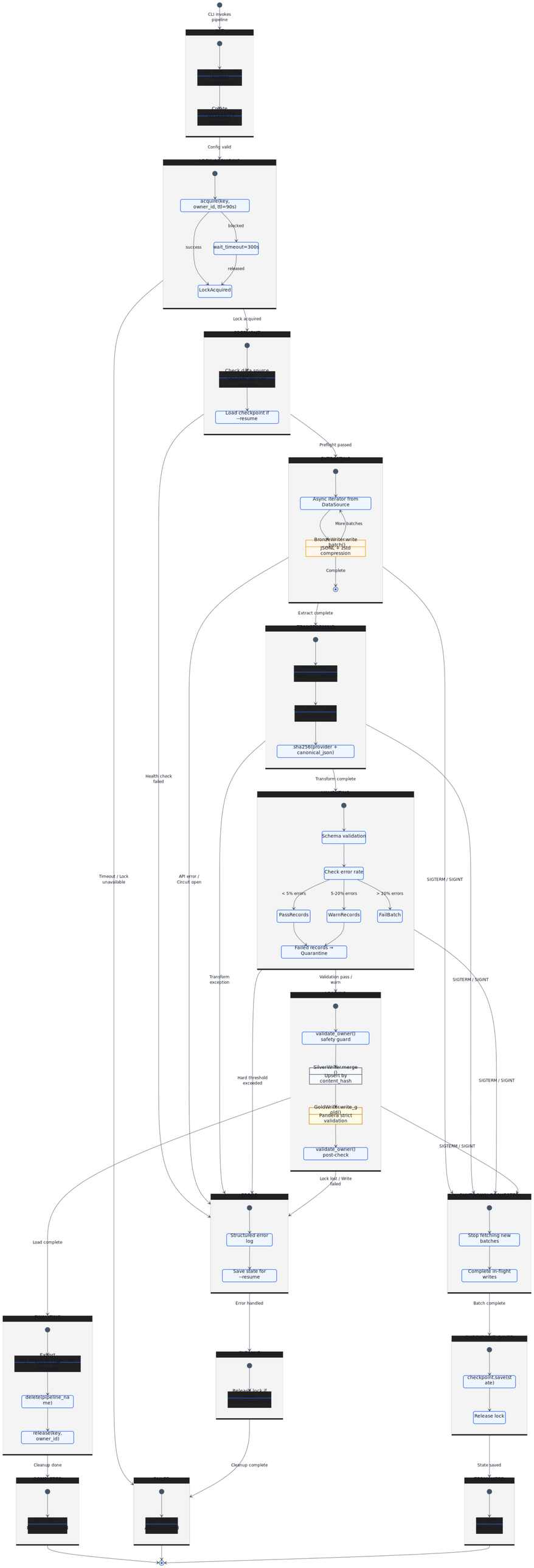

Entities
Activity
+activity_id: str
+assay_id: str
+molecule_id: str
+target_id: str?
+standard_type: str
+standard_value: float?
+standard_units: str?
+pchembl_value: float?
Molecule
+molecule_id: str
+pref_name: str?
+molecule_type: str?
+canonical_smiles: str?
+inchi_key: str?
+molecular_weight: float?
+parent_molecule_id: str?
«abstract»
BaseEntity
+entity_id: EntityID
+content_hash: ContentHash
+run_id: RunID
+run_type: RunType
+ingestion_ts: datetime
Assay
+assay_id: str
+assay_type: str
+description: str?
+target_id: str?
+document_id: str?
Target
+target_id: str
+target_type: str
+pref_name: str?
+organism: str?
+tax_id: int?
+uniprot_accessions: list<str>?
«abstract»
PublicationEntityBase
+doi: str?
+pmid: str?
+title: str?
+publication_year: int?
+citations_received: int?
+citations_made: int?
+author_orcids: str?
+_source: str
ChemblPublication
+document_chembl_id: str
+publication_type: str
+volume: str?
+src_id: int?


Config
«frozen dataclass»
PipelineConfig
+provider: str
+entity: str
+table_config: TableConfig
+dq_config: DQConfig
RuntimeConfig
+run_id: RunID
+run_type: RunType
+limit: int?
+resume: bool
+dry_run: bool
+filters: FilterConfig?
«frozen dataclass»
DQConfig
+soft_fail_threshold: float = 0.05
+hard_fail_threshold: float = 0.20
+strict_validation: bool = False
«frozen dataclass»
TableConfig
+primary_keys: tuple<str>
+silver_table: str?
+gold_table: str?
+silver_write_mode: SilverWriteMode
+gold_write_mode: GoldWriteMode
+partition_cols: tuple<str>

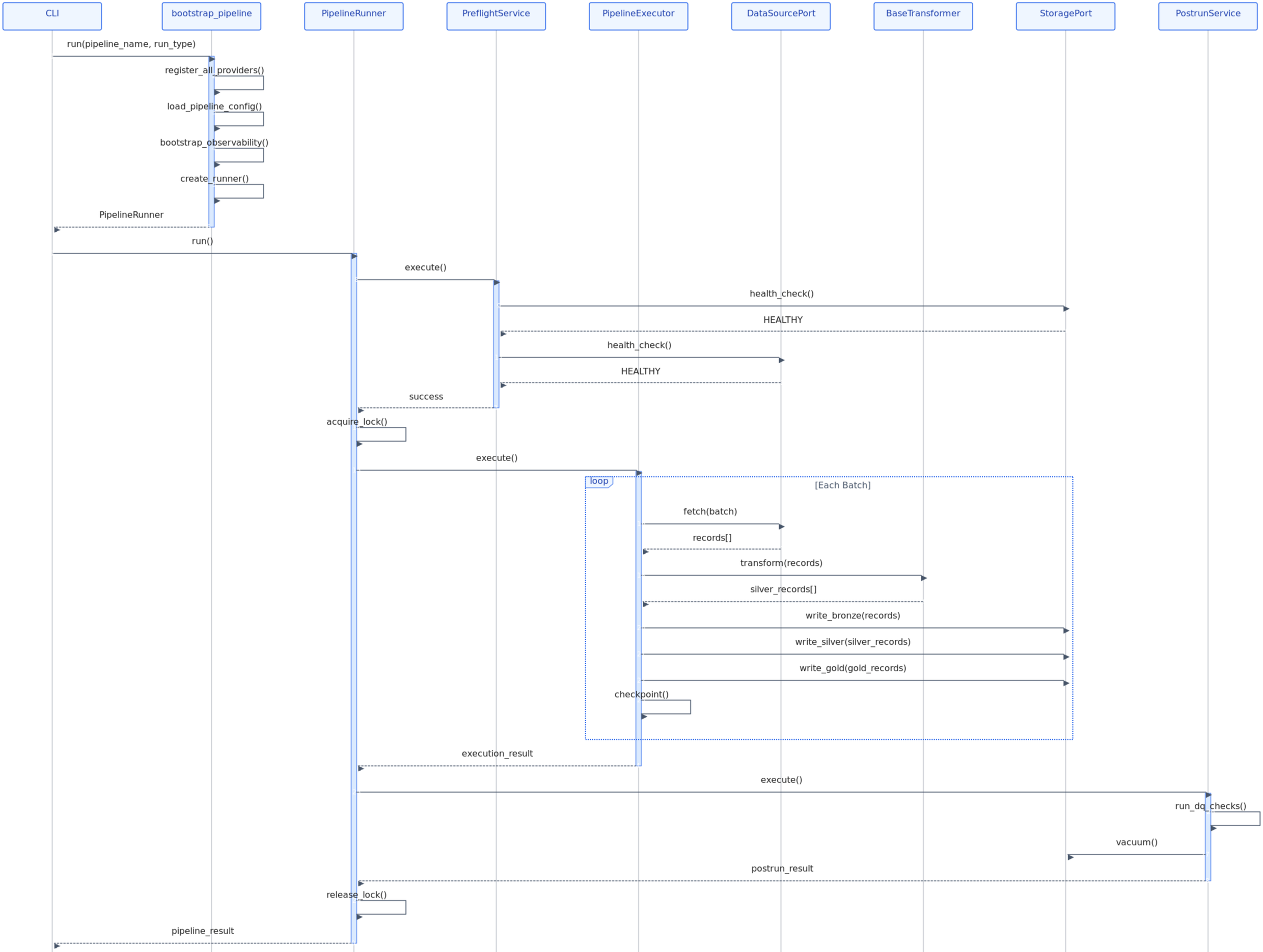

Types
«enumeration»
RunType
INCREMENTAL
BACKFILL
REBUILD
«enumeration»
SilverWriteMode
MERGE
APPEND
DELETE
«enumeration»
GoldWriteMode
APPEND
OVERWRITE
SCD2
«enumeration»
CircuitState
CLOSED = 0
HALF_OPEN = 1
OPEN = 2

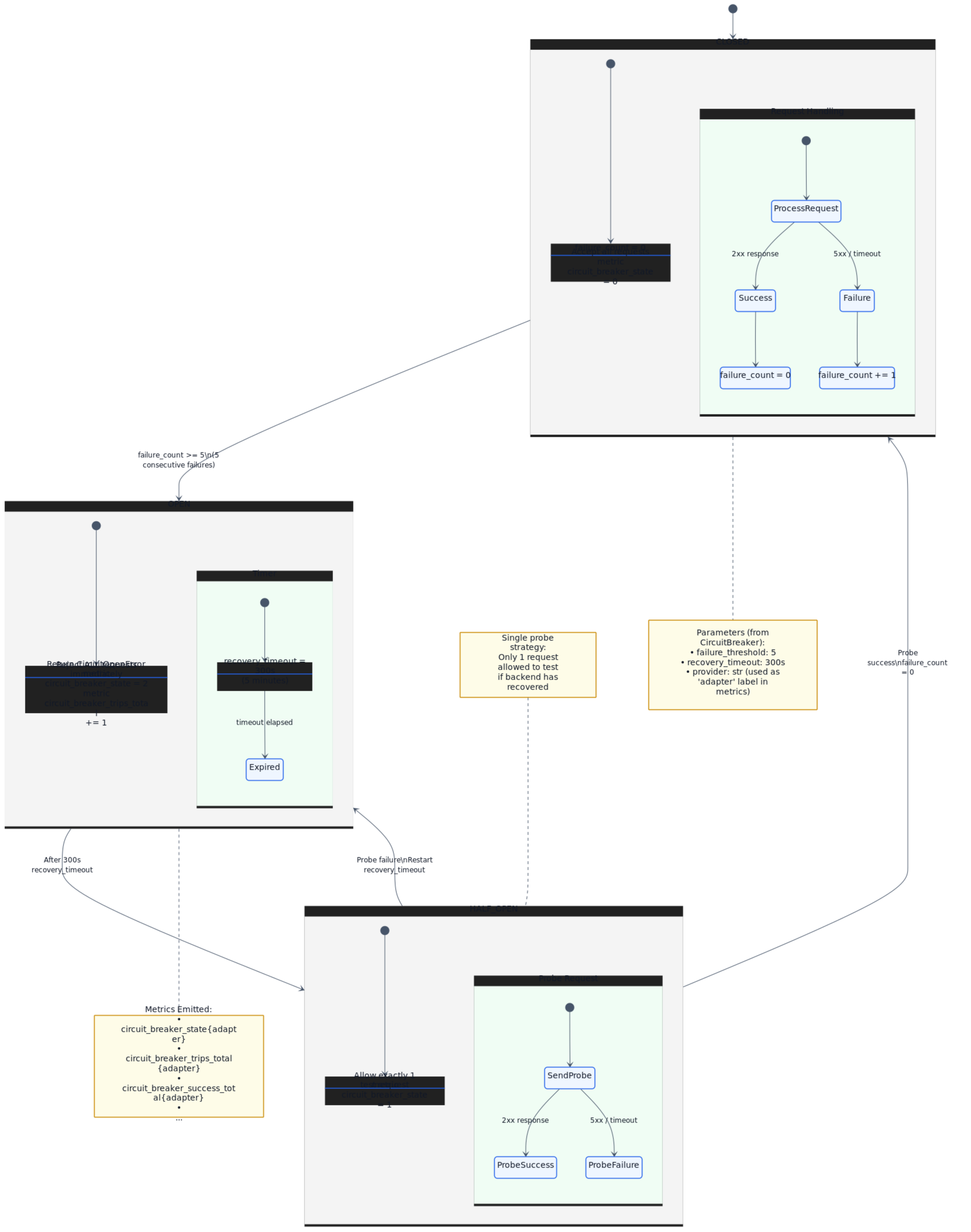

Pipeline Execution
B
Pass
BatchTransformer
normalize & enrich
Silver DQ
Validation
Pass
Fail
SilverWriter
Delta merge
Fail
Gold DQ
Validation
Pass
Fail
GoldWriter
SCD2 write
ErrorClassifier
Record committed
Error Handling
RetryCheck
No
A
Yes
Quarantine (§2.3)
QMeta
Log WARNING
+ DQ metrics
Continue
pipeline
DataQualityError
(bad data)
CriticalError
(infra failure)
RecoverableError
(transient)
Error
Classification
(§3.1)
DQ
Critical
Retry

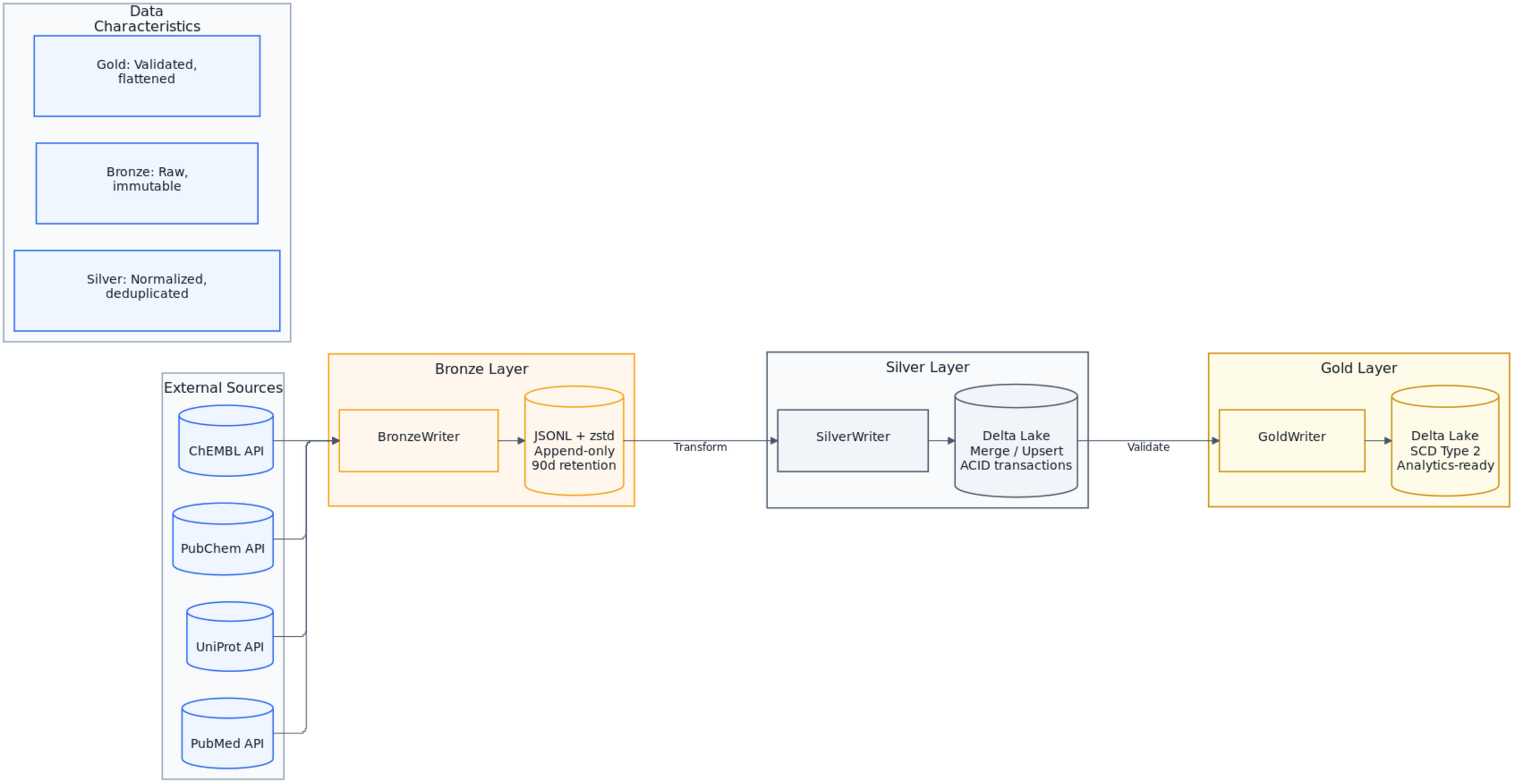

Interfaces Layer
CLI Entry Point
Composition Layer
bootstrap_pipeline
PipelineRegistry
Factories
Infrastructure Layer
Locks & Checkpoints
Storage Writers
Metrics & Tracing
Data Source Adapters
Application Layer
Composite Pipeline (ADR-026)
CompositePipelineRunner
MergeService
EnrichmentCoordinator
implements
implements
implements
delegates
PipelineRunner
PipelineServices
BatchExecutor
BaseTransformer
Domain Layer
Exception Hierarchy
Core Types
Domain Entities
Port Interfaces

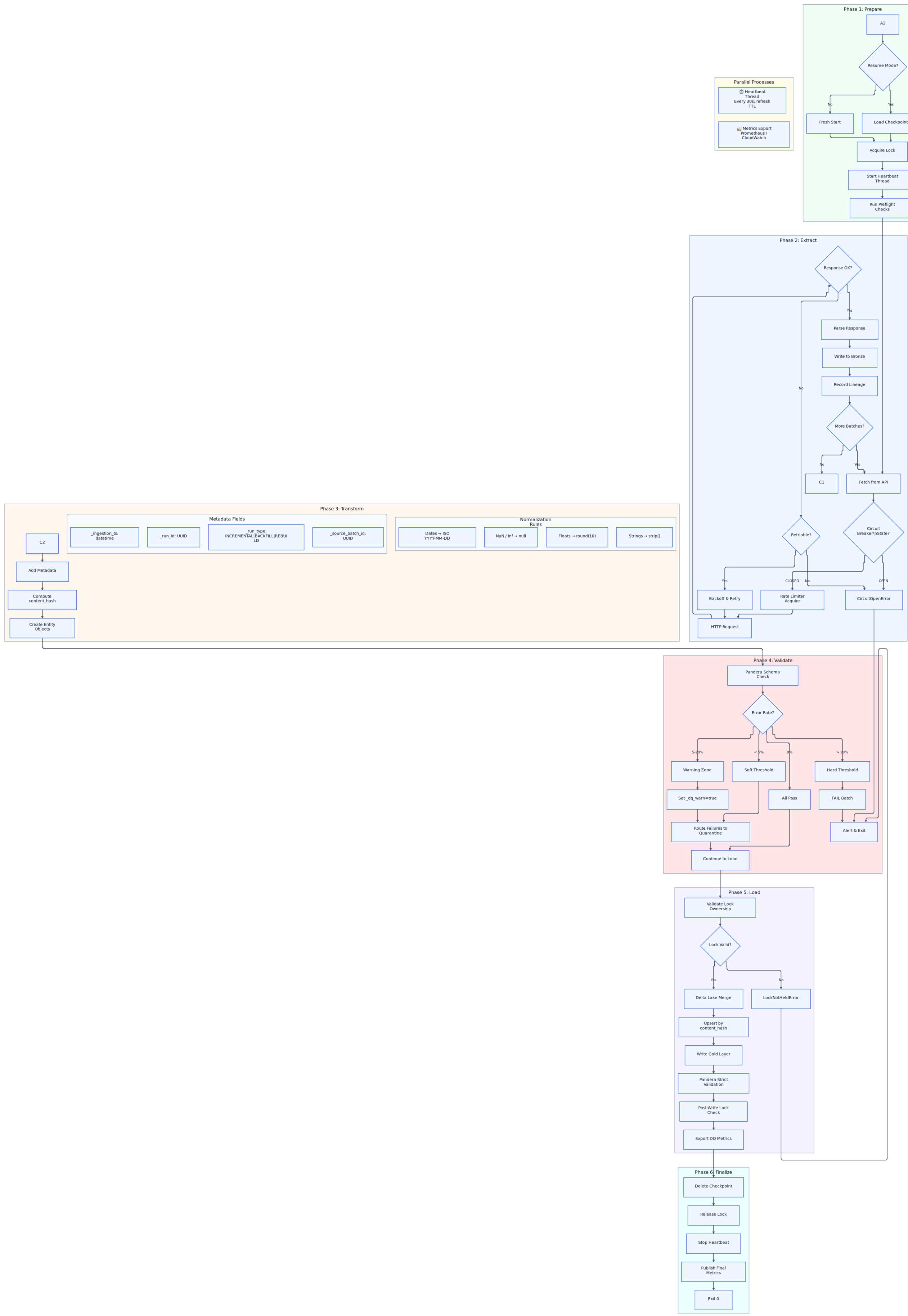

CLI invokes pipeline
Config valid
acquire(key, owner_id, ttl=90s)
blocked
success
wait_timeout=300s
released
LockAcquired
Lock acquired
Check data source
Load checkpoint if --resume
Preflight passed
Async iterator from DataSource
More batches
BronzeWriter.write batch() JSONL + zstd compression
Complete
Extract complete
sha256(provider + canonical_json)
Transform complete
Schema validation
Check error rate
< 5% errors
5-20% errors
> 20% errors
PassRecords
WarnRecords
FailBatch
Failed records → Quarantine
Validation pass / warn
validate_owner() safety guard
SilverWriter.merge() upsert by content_hash
GoldWriter.write_gold() Pandera strict validation
validate_owner() post-check
Lock lost / Write failed
Structured error log
Save state for --resume
Error handled
Stop fetching new batches
Complete in-flight writes
Batch complete
checkpoint.save(state)
Release lock
State saved
Export
delete(pipeline_name)
release(key, owner_id)
Cleanup done
Release lock if
Cleanup complete
Health check failed
Timeout / Lock unavailable
API error / Circuit open
Transform exception
Hard threshold exceeded
Load complete
SIGTERM / SIGINT

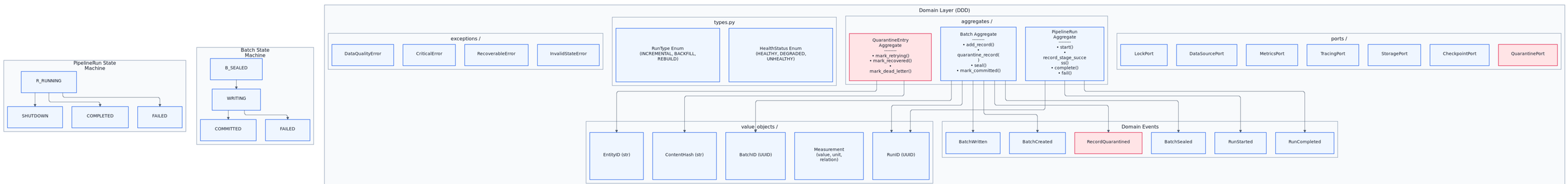
Transformers
ActivityTransforme
r
+entity: str =
"activity"
#_transform_impl(record) :
SilverRecord
MoleculeTransforme
r
+entity: str =
"molecule"
#_transform_impl(record) :
SilverRecord
AssayTransformer
+entity: str =
"assay"
#_transform_impl(record) :
SilverRecord
TargetTransformer
+entity: str =
"target"
#_transform_impl(record) :
SilverRecord
«abstract»
BaseChemblTransfor
mer
#_normalize_chembl_id
(id)
#_normalize_units(value,
units) : tuple
«abstract»
BaseTransformer
#config:
ProcessingConfig
#stages:
TransformStages
#logger:
LoggerPort
+transform(record: BronzeRecord) :
SilverRecord | None
+transform_batch(records) :
list[SilverRecord]
#_transform_impl(record) :
SilverRecord | None
#compute_hash(provider, data)
#serialize_json(obj)
#content_hash
#extract_silver_record(entity
_id, record, field_record
) : SilverRecord
#_extract_specific(record,
**kwargs) : dict
#_create_entity(**kwargs)
: BaseEntity
Services
LockManager
-_lock: LockPort
-_logger:
LoggerPort
-_ttl_default_interval
int = 30
+acquire_lock(resource, owner_id)
+release_lock(resource,
owner_id) : bool
+extend_lock(resource,
owner_id) : bool
CheckpointManager
-_checkpoint:
CheckpointPort
-_logger:
LoggerPort
+load_checkpoint(key) :
Checkpoint | None
+save_checkpoint(key,
checkpoint) : None
+delete_checkpoint(key)
: None
QuarantineManager
-_quarantine:
QuarantinePort
-_logger:
LoggerPort
+quarantine_record(record,
errors) : str
PreflightService
-_data_source:
DataSourcePort
-_storage:
StoragePort
-_logger:
LoggerPort
+run_checks() :
PreflightResult
-_check_source_health() :
CheckResult
-_check_storage_health
() : CheckResult
PostrunService
-_storage:
StoragePort
-_logger:
LoggerPort
-_config:
DQConfig
+run_checks(execution_metrics) :
PostrunResult
-_check_row_counts(
execution_metrics) :
CheckResult
LifecycleOrchestra
tor
-_lifecycle_service:
MedallionLifecycleService
-_policy:
MedallionPolicy
-_logger:
LoggerPort
+clear_for_run(run_type,
tables) : ClearResult
PipelineObserver
-_tracer:
TracerPort
-_logger:
LoggerPort
+observe_batch(batch_id,
record_count, duration,
error=None) : None
MemoryMonitor
-_threshold_pct:
float = 0.8
-_check_interval_ms:
int = 1000
+get_memory_stats() :
MemoryStats
-_get_rss_estimate() :
MemoryStats
BatchMetricsRecord
er
-_metrics:
MetricsPort
+record_fetch(count,
duration_ms) : None
+record_write(count, bytes,
duration, layer) : None
Core
PipelineRunner
-_pipeline:
BasePipeline
-_services:
RunnerServices
-_lock_key: str
+run() : RunResult
-_acquire_lock() :
bool
-_run_preflight() :
PreflightResult
-_run_postrun() :
PostrunResult
-_release_lock() :
None
PipelineExecutor
-_pipeline:
BasePipeline
-_executor:
RecordProcessor
-_checkpoint_interval:
int = 1000
+execute() :
ExecutionResult
-_load_and_process() :
AsyncIterator[BatchResult]
-_log_progress(batch_idx:
int) : None
«frozen dataclass»
RunnerServices
+lock_manager:
LockManager
+checkpoint_manager:
CheckpointManager
+preflight_service:
PreflightService
+postrun_service:
PostrunService
+lifecycle_orchestrator:
LifecycleOrchestrator
+lock_context_holder?
RecordProcessor
-_transformer:
BaseTransformer
-_batch_transformer:
BatchTransformer
-_quarantine:
QuarantineManager
-_metrics:
BatchMetricsRecorder
+process_batch(records) :
BatchResult
-_process_stream(records) :
AsyncIterator[BatchResult]
«abstract»
BasePipeline
#config:
PipelineConfig
#runner:
PipelineRunner
#services:
PipelineServices
+create(config, runtime,
**overrides) : BasePipeline
+pipeline_id() : str
+provider() : str
+run_id() : RunID
BatchTransformer
-_transformer:
BaseTransformer
-_logger:
LoggerPort
+transform_batch(records) :
TransformResult
-_transform_stream(records) :
AsyncIterator[TransformResult]
«frozen dataclass»
PipelineServices
+data_source:
DataSourcePort
+storage:
StoragePort
+lock: LockPort
+checkpoint:
CheckpointPort
+quarantine:
QuarantinePort
+logger:
LoggerPort
+tracer:
TracerPort
+dq_monitor:
DQMonitorPort?


CLI
bootstrap_pipeline
PipelineRunner
PreflightService
PipelineExecutor
DataSourcePort
BaseTransformer
StoragePort
PostrunService
run(pipeline_name, run_type)
register_all_providers()
load_pipeline_config()
bootstrap_observability()
create_runner()
PipelineRunner
run()
execute()
health_check()
HEALTHY
health_check()
HEALTHY
success
acquire_lock()
execute()
loop
[Each Batch]
fetch(batch)
records[]
transform(records)
silver_records[]
write_bronze(records)
write_silver(silver_records)
write_gold(gold_records)
checkpoint()
execution_result
execute()
run_dq_checks()
vacuum()
postrun_result
release_lock()
pipeline_result


CLOSED
Request Handling
ProcessRequest
2xx response
5xx / timeout
Success
Failure
failure_count = 0
failure_count += 1
failure_count >= 5\n(5 consecutive failures)
OPEN
Timer
recovery_timeout = 300s (5 minutes)
timeout elapsed
Expired
+= 1
Single probe strategy: Only 1 request allowed to test if backend has recovered
Parameters (from CircuitBreaker):
• failure_threshold: 5
• recovery_timeout: 300s
• provider: str (used as 'adapter' label in metrics)
Probe success\nfailure_count = 0
After 300s recovery_timeout
Probe failure\nRestart recovery_timeout
HALF_OPEN
Probe Request
SendProbe
2xx response
5xx / timeout
ProbeSuccess
ProbeFailure
Metrics Emitted:
• circuit_breaker_state{adapter}
• circuit_breaker_trips_total{adapter}
• circuit_breaker_success_total{adapter}
• ...


Data Characteristics
Gold: Validated, flattened
Bronze: Raw, immutable
Silver: Normalized, deduplicated
External Sources
ChEMBL API
PubChem API
UniProt API
PubMed API
Bronze Layer
BronzeWriter
JSONL + zstd
Append-only
90d retention
Transform
Silver Layer
SilverWriter
Delta Lake
Merge / Upsert
ACID transactions
Validate
Gold Layer
GoldWriter
Delta Lake
SCD Type 2
Analytics-ready


Phase 1: Prepare
A2
Resume Mode?
No
Yes
Fresh Start
Load Checkpoint
Acquire Lock
Start Heartbeat Thread
Run Preflight Checks
Parallel Processes
⏱️ Heartbeat Thread Every 30s: refresh TTL
📊 Metrics Export Prometheus / CloudWatch
Phase 2: Extract
Response OK?
Yes
Parse Response
Write to Bronze
Record Lineage
No
More Batches?
No
Yes
C1
Fetch from API
Retriable?
Circuit Breaker\nState?
Yes
CLOSED
No
OPEN
Backoff & Retry
Rate Limiter Acquire
CircuitOpenError
HTTP Request
Phase 3: Transform
C2
Add Metadata
Compute content_hash
Create Entity Objects
Metadata Fields
_ingestion_ts: datetime
_run_id: UUID
_run_type: INCREMENTAL|BACKFILL|REBUILD
_source_batch_id: UUID
Normalization Rules
Dates → ISO YYYY-MM-DD
NaN / Inf → null
Floats → round(10)
Strings → strip()
Phase 4: Validate
Pandera Schema Check
Error Rate?
5-20%
< 5%
0%
> 20%
Warning Zone
Soft Threshold
All Pass
Hard Threshold
Set _dq_warn=true
FAIL Batch
Route Failures to Quarantine
Alert & Exit
Continue to Load
Phase 5: Load
Validate Lock Ownership
Lock Valid?
Yes
No
Delta Lake Merge
LockNotHeldError
Upsert by content_hash
Write Gold Layer
Pandera Strict Validation
Post-Write Lock Check
Export DQ Metrics
Phase 6: Finalize
Delete Checkpoint
Release Lock
Stop Heartbeat
Publish Final Metrics
Exit 0


PipelineRun State Machine
R_RUNNING
SHUTDOWN
COMPLETED
FAILED
Batch State Machine
B_SEALED
WRITING
COMMITTED
FAILED
Domain Layer (DDD)
exceptions /
DataQualityError
CriticalError
RecoverableError
InvalidStateError
types.py
RunType Enum (INCREMENTAL, BACKFILL, REBUILD)
HealthStatus Enum (HEALTHY, DEGRADED, UNHEALTHY)
aggregates /
QuarantineEntry Aggregate
• mark_retrying()
• mark_recovered()
• mark_dead_letter()
Batch Aggregate
• add_record()
• quarantine_record(
• seal()
• mark_committed()
PipelineRun Aggregate
• start()
• record_stage_success()
• complete()
• fail()
ports /
LockPort
DataSourcePort
MetricsPort
TracingPort
StoragePort
CheckpointPort
QuarantinePort
value_objects /
EntityID (str)
ContentHash (str)
BatchID (UUID)
Measurement (value, unit, relation)
RunID (UUID)
Domain Events
BatchWritten
BatchCreated
RecordQuarantined
BatchSealed
RunStarted
RunCompleted

Quarantine			
timestamp	ingestion_ts	PK	Ingestion timestamp
string	pipeline		Source pipeline
string	error_code		Error classification
string	error_message		Human-readable message
string	payload		Truncated record (64KB max)
string	payload_hash		Hash of full payload
uuid	bronze_batch_id	FK	Reference to LineageLog
string	bronze_file_uri		Bronze file location
string	dq_status		NEW|IGNORED|REPROCESSED
timestamp	updated_at		Last status update

OpenAlexPublication			
string	openalex_id	PK	OpenAlex Work ID (W prefix)
string	doi		DOI (nullable)
string	pmid		PubMed ID (nullable)
string	title		Article title (nullable)
string	abstract		Abstract text (nullable)
json	authors		Author list (nullable)
string	journal		Journal name
int	publication_year		Publication year (1500-2100)
int	citations_received		Incoming citation count
int	citations_made		Outgoing reference count
json	author_orcids		Author ORCIDs (JSON array)
float	fwci		Field-weighted citation impact
bool	is_retracted		Retraction flag
string	_source		Source: openalex
string	_content_hash		SHA256 hash
uuid	_run_id		Pipeline run ID
timestamp	_ingestion_ts		Ingestion timestamp

SemanticScholarPublication			
string	paper_id	PK	S2 Paper ID (40-char hex)
string	doi		DOI (nullable)
string	pmid		PubMed ID (nullable)
string	title		Article title (nullable)
string	abstract		Abstract text (nullable)
json	authors		Author list (nullable)
string	journal		Journal name
int	publication_year		Publication year (1500-2100)
int	citations_received		Incoming citation count
int	citations_made		Outgoing reference count
int	influential_citation_count		Influential citations
json	author_orcids		Author ORCIDs (JSON array)
json	author_s2_ids		S2 Author IDs (JSON array)
string	tldr		AI-generated summary
string	_source		Source: semanticscholar
string	_content_hash		SHA256 hash
uuid	_run_id		Pipeline run ID
timestamp	_ingestion_ts		Ingestion timestamp

Protein			
string	protein_id	PK	UniProt accession
string	entry_name		UniProt entry name
string	protein_name		Full protein name
string	organism		Source organism
int	tax_id		NCBI taxonomy ID
string	sequence		Amino acid sequence
int	sequence_length		Sequence length
json	features		Protein features
string	_content_hash		SHA256 hash
uuid	_run_id		Pipeline run ID
timestamp	_ingestion_ts		Ingestion timestamp

Compound			
string	compound_id	PK	PubChem CID
string	iupac_name		IUPAC name
string	canonical_smiles		Canonical SMILES
string	isomeric_smiles		Isomeric SMILES
string	inchi		InChI string
string	inchikey		InChIKey
float	molecular_weight		Molecular weight
string	molecular_formula		Chemical formula
float	xlogp		Calculated LogP
int	h_bond_donor_count		H-bond donors
int	h_bond_acceptor_count		H-bond acceptors
string	_content_hash		SHA256 hash
uuid	_run_id		Pipeline run ID
timestamp	_ingestion_ts		Ingestion timestamp

bronze_batch_id FK

LineageLog			
uuid	batch_id	PK	Unique batch identifier
string	pipeline		Pipeline name
json	bronze_paths		List of Bronze file paths
string	transform_version		Transformer version
json	run_params		Runtime parameters
timestamp	created_at		Creation timestamp

batch contains

Activity			
string	activity_id	PK	ChEMBL activity ID
string	assay_id	FK	Reference to Assay
string	molecule_id	FK	Reference to Molecule
string	target_id	FK	Reference to Target (nullable)
string	standard_type		Activity type (IC50, Ki, etc.)
float	standard_value		Measured value (nullable)
string	standard_units		Units (nM, uM, etc.)
string	standard_relation		Relation (=, <, >)
float	pchembl_value		Normalized potency (nullable)
string	data_validity_comment		Validity status
string	_content_hash		SHA256 hash
uuid	_run_id		Pipeline run ID
string	_run_type		INCREMENTAL|BACKFILL|REBUILD
uuid	_source_batch_id		Bronze batch reference
timestamp	_ingestion_ts		Ingestion timestamp
bool	_dq_warn		DQ warning flag

assay_id FK

molecule_id FK

target_id FK (nullable)

target_id FK

Assay			
string	assay_id	PK	ChEMBL assay ID
string	assay_type		Binding, Functional, ADMET
string	description		Assay description (nullable)
string	assay_organism		Test organism
string	assay_strain		Organism strain
string	assay_cell_type		Cell type used
string	target_id	FK	Reference to Target (nullable)
string	document_id	FK	Reference to Document (nullable)
int	src_id		Source database ID
string	_content_hash		SHA256 hash
uuid	_run_id		Pipeline run ID
string	_run_type		Run type enum
timestamp	_ingestion_ts		Ingestion timestamp

Molecule			
string	molecule_id	PK	ChEMBL molecule ID
string	pref_name		Preferred name (nullable)
string	molecule_type		Small molecule, Antibody, etc.
int	max_phase		Max clinical phase
string	canonical_smiles		SMILES string (nullable)
string	standard_inchi		InChI string (nullable)
string	inchi_key		InChIKey (nullable)
string	molecular_formula		Chemical formula
float	molecular_weight		Molecular weight (nullable)
string	parent_molecule_id	FK	Parent molecule (nullable)
string	_content_hash		SHA256 hash
uuid	_run_id		Pipeline run ID
string	_run_type		Run type enum
timestamp	_ingestion_ts		Ingestion timestamp

document_chembl_id FK

ChemblPublication			
string	document_chembl_id	PK	ChEMBL document ID
string	publication_type		PUBLICATION, PATENT, etc.
string	doi		DOI (nullable)
string	pmid		PubMed ID (nullable, string)
string	title		Publication title (nullable)
json	authors		Author list (nullable)
string	journal		Journal name
int	publication_year		Publication year (1500-2100)
string	publication_date		Date YYYY-MM-DD
string	volume		Journal volume
string	page_first		First page
string	page_last		Last page
string	abstract		Abstract text (nullable)
int	citations_received		Incoming citation count
int	citations_made		Outgoing reference count
json	author_orcids		Author ORCIDs (JSON array)
string	_source		Source: chembl
string	_content_hash		SHA256 hash
uuid	_run_id		Pipeline run ID
timestamp	_ingestion_ts		Ingestion timestamp

Target			
string	target_id	PK	ChEMBL target ID
string	target_type		PROTEIN, CELL-LINE, etc.
string	pref_name		Preferred name (nullable)
string	organism		Source organism
int	tax_id		NCBI taxonomy ID (nullable)
json	uniprot_accessions		UniProt IDs (nullable)
json	gene_names		Gene symbols (nullable)
string	_content_hash		SHA256 hash
uuid	_run_id		Pipeline run ID
string	_run_type		Run type enum
timestamp	_ingestion_ts		Ingestion timestamp

has components

pmid match

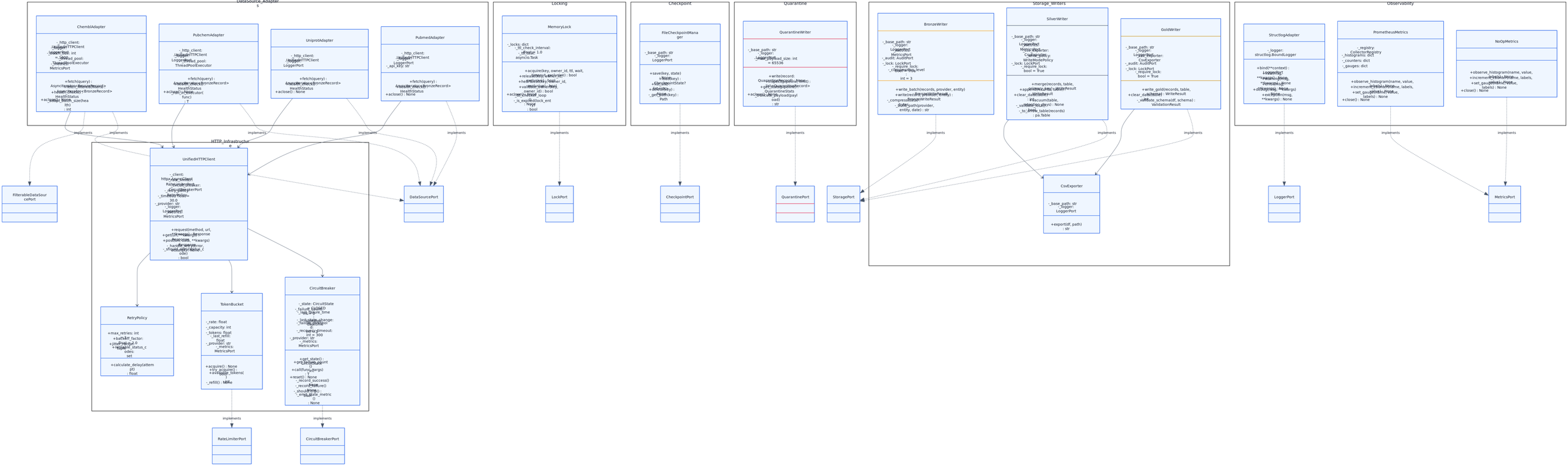
doi match

parent_molecule_id FK

TargetComponent			
string	component_id	PK	Component ID
string	target_id	FK	Parent target
string	accession		UniProt accession
string	component_type		PROTEIN, DNA, etc.
string	description		Component description

PubMedPublication			
string	pmid	PK	PubMed ID (positive integer string)
string	doi		DOI (nullable)
string	pmc_id		PMC ID (nullable)
string	title		Article title
string	abstract		Abstract text (nullable)
json	authors		Author list
string	journal		Journal name
int	publication_year		Publication year (1500-2100)
string	publication_date		Date YYYY-MM-DD
int	citations_received		Incoming citation count
int	citations_made		Outgoing reference count
json	author_orcids		Author ORCIDs (JSON array)
json	affiliation_list		Affiliations (JSON array)
json	subject_mesh		MeSH terms (JSON array)
json	subject_keywords		Keywords (JSON array)
string	_source		Source: pubmed
string	_content_hash		SHA256 hash
uuid	_run_id		Pipeline run ID
timestamp	_ingestion_ts		Ingestion timestamp

CrossRefPublication			
string	doi	PK	DOI (required)
string	pmid		PubMed ID (nullable)
string	title		Article title (nullable)
string	abstract		Abstract text (nullable)
json	authors		Author list (nullable)
string	journal		Journal name
string	publisher		Publisher name
int	publication_year		Publication year (1500-2100)
string	publication_date		Date YYYY-MM-DD
int	citations_received		Incoming citation count
int	citations_made		Outgoing reference count
json	author_orcids		Author ORCIDs (JSON array)
string	issn		ISSN (nullable)
string	license_url		License URL (nullable)
string	_source		Source: crossref
string	_content_hash		SHA256 hash
uuid	_run_id		Pipeline run ID
timestamp	_ingestion_ts		Ingestion timestamp


DataSource_Adapters
ChemblAdapter
- _http_client: UnifiedHTTPClient
- _logger: LoggerPort
- _page_size: int = 1000
- _thread_pool: ThreadPoolExecutor
- _metrics: MetricsPort
+fetch(query) : AsyncIterator[BronzeRecord]
+fetch_by_id(id) : BronzeRecord
+health_check() : HealthStatus
+aclose() : None
PubchemAdapter
- _http_client: UnifiedHTTPClient
- _logger: LoggerPort
- _thread_pool: ThreadPoolExecutor
+fetch(query) : AsyncIterator[BronzeRecord]
+health_check() : HealthStatus
+aclose() : None
UniprotAdapter
- _http_client: UnifiedHTTPClient
- _logger: LoggerPort
+fetch(query) : AsyncIterator[BronzeRecord]
+health_check() : HealthStatus
+aclose() : None
PubmedAdapter
- _http_client: UnifiedHTTPClient
- _logger: LoggerPort
- _api_key: str
+fetch(query) : AsyncIterator[BronzeRecord]
+health_check() : HealthStatus
+aclose() : None
Locking
MemoryLock
- _locks: dict
- _ttl_check_interval: float = 1.0
- _cleanup_task: asyncio.Task
+acquire(key, owner_id, ttl, wait, timeout) : bool
+release(key, owner_id) : bool
+heartbeat(key, owner_id, ttl) : bool
+validate_owner(key, owner_id) : bool
+aclose() : None
- _cleanup_loop()
- _is_expired(lock_entry) : bool
Checkpoint
FileCheckpointManager
- _base_path: str
- _logger: LoggerPort
+save(key, state) : None
+load(key) : CheckpointState
+delete(key) : None
- _get_path(key) : Path
Quarantine
QuarantineWriter
- _base_path: str
- _logger: LoggerPort
- _max_buffer_size: int = 65536
+write(record: QuarantineRecord, reason: str) : None
+get_stats() : QuarantineStats
+aclose() : None
- _flush_buffer(payload) : str
Storage_Writers
BronzeWriter
- _base_path: str
- _logger: LoggerPort
- _metrics: MetricsPort
- _audit: AuditPort
- _lock: LockPort
- _require_lock: bool = True
- _compression_level: int = 3
+write_batch(records, provider, entity) : BronzeWriteResult
+write(record, provider, entity) : BronzeWriteResult
- _build_path(provider, entity, date) : str
SilverWriter
- _base_path: str
- _logger: LoggerPort
- _metrics: MetricsPort
- _audit: AuditPort
- _lock: LockPort
- _require_lock: bool = True
+merge(records, table, primary_key) : SilverWriteResult
+append_only(records, table) : SilverWriteResult
+clear_table(table) : SilverWriteResult
- _read_existing(table, keys) : pa.Table
- _to_arrow_table(records) : pa.Table
GoldWriter
- _base_path: str
- _logger: LoggerPort
- _csv_exporter: CsvExporter
- _audit: AuditPort
- _lock: LockPort
- _require_lock: bool = True
+write_gold(records, table, schema) : WriteResult
+validate_schema(df, schema) : ValidationResult
CsvExporter
- _base_path: str
- _logger: LoggerPort
+export(df, path) : str
Observability
StructlogAdapter
- _logger: structlog.BoundLogger
+bind(**context) : LoggerPort
+unbind(*keys) : None
+debug(msg, **kwargs) : None
+info(msg, **kwargs) : None
+warning(msg, **kwargs) : None
+error(msg, **kwargs) : None
+exception(msg, **kwargs) : None
PrometheusMetrics
- _registry: CollectorRegistry
- _histograms: dict
- _counters: dict
- _gauges: dict
+observe_histogram(name, value, labels) : None
+increment_counter(name, labels, value) : None
+set_gauge(name, value, labels) : None
+close() : None
NoOpMetrics
+observe_histogram(name, value, labels) : None
+increment_counter(name, labels, value) : None
+set_gauge(name, value, labels) : None
+close() : None
HTTP_Infrastructure
UnifiedHTTPClient
- _client: httpx.AsyncClient
- _rate_limiter: RateLimiterPort
- _circuit_breaker: CircuitBreaker
- _retry_policy: RetryPolicy
- _timeout: float = 30.0
- _provider: str
- _logger: LoggerPort
- _metrics: MetricsPort
+request(method, url, **kwargs) : httpx.Response
+get(url, **kwargs) : httpx.Response
+post(url, **kwargs) : httpx.Response
- _handle_error_response(response) : None
- _should_retry(error, attempt) : bool
RetryPolicy
+max_retries: int = 3
+backoff_factor: float = 2.0
+jitter: bool = True
+retryable_status_codes: set
+calculate_delay(attempt) : float
TokenBucket
- _rate: float
- _capacity: int
- _tokens: float
- _last_refill: float
- _provider: str
- _metrics: MetricsPort
+acquire() : None
+try_acquire(tokens) : bool
- _refill() : None
CircuitBreaker
- _state: CircuitState = CLOSED
- _failure_count: int
- _last_failure_time: float
- _last_state_change: float
- _failure_threshold: int = 5
- _recovery_timeout: int = 300
- _provider: str
- _metrics: MetricsPort
+get_state() : CircuitState
+get_stats() : CircuitStats
+call(func, *args, **kwargs) : T
+reset() : None
- _record_success()
- _record_failure()
- _should_attempt() : bool
- _emit_state_metric() : None
Ports
FilterableDataSourcePort
DataSourcePort
LockPort
CheckpointPort
QuarantinePort
StoragePort
RateLimiterPort
CircuitBreakerPort
LoggerPort
MetricsPort
implements

External APIs

🌐 PubChem API
pubchem.ncbi.nlm.nih.gov

🌐 UniProt API
uniprot.org

🌐 PubMed API
eutils.ncbi.nlm.nih.gov

🌐 ChEMBL API
ebi.ac.uk / chembl

Local Machine (Single Instance)

CLI Execution

⏰ Local scheduler
(cron / systemd)

🖥️ CLI / Manual run
PipelineRunner

In-Process Locking

Local Pipeline Workers

📦 pubchem_compound
local pipeline

📦 uniprot_protein
local pipeline

📦 chembl_target
local pipeline

📦 chembl_assay
local pipeline

📦 chembl_molecule
local pipeline

📦 chembl_activity
local pipeline

Local Observability

📋 Local logs
stdout / file

📊 In-process metrics

Local filesystem (data /)

📁 gold / Delta / Parquet

📁 bronze / JSONL+zstd

📁 lineage / Parquet

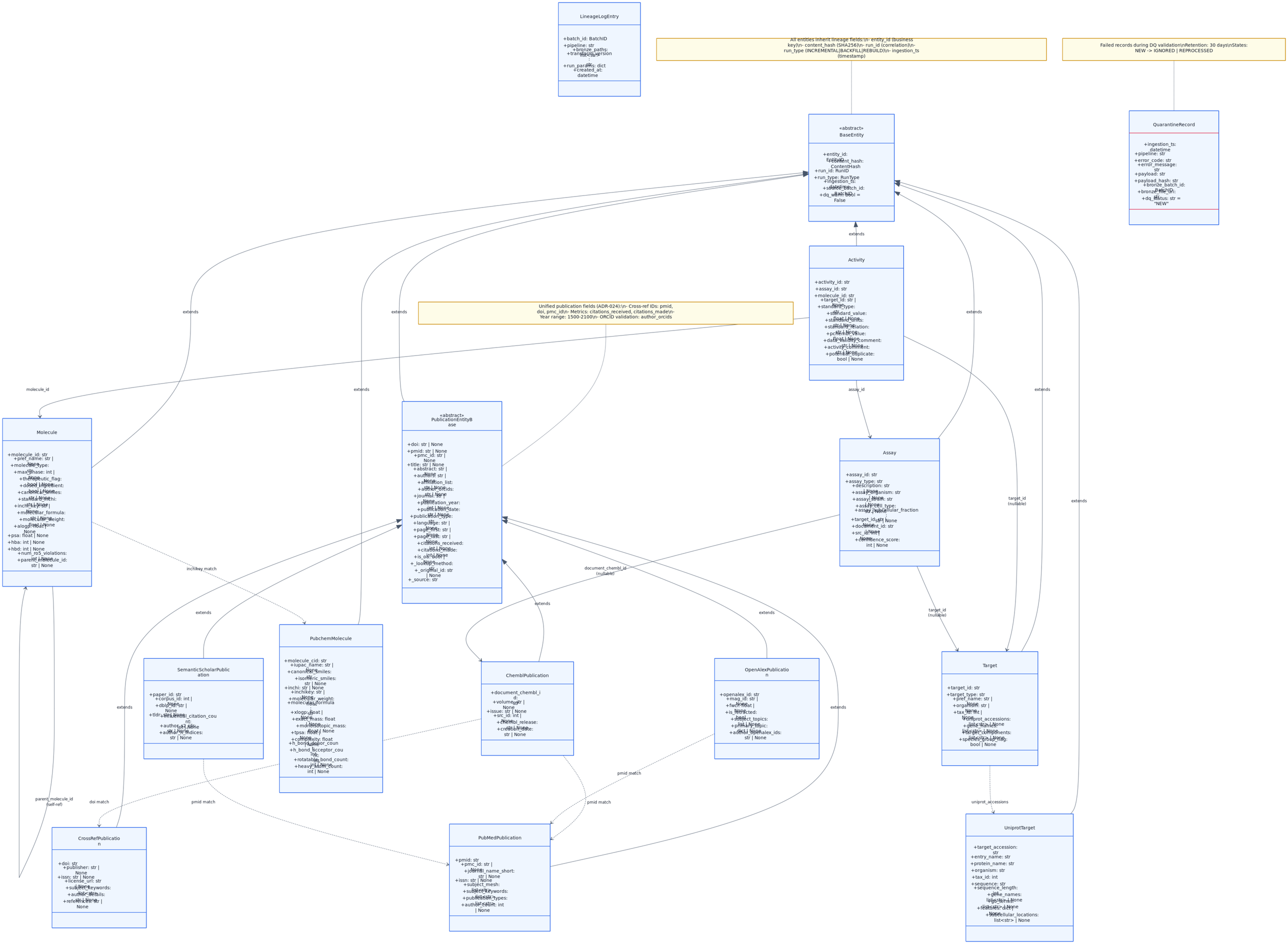
📁 silver / Delta Lake

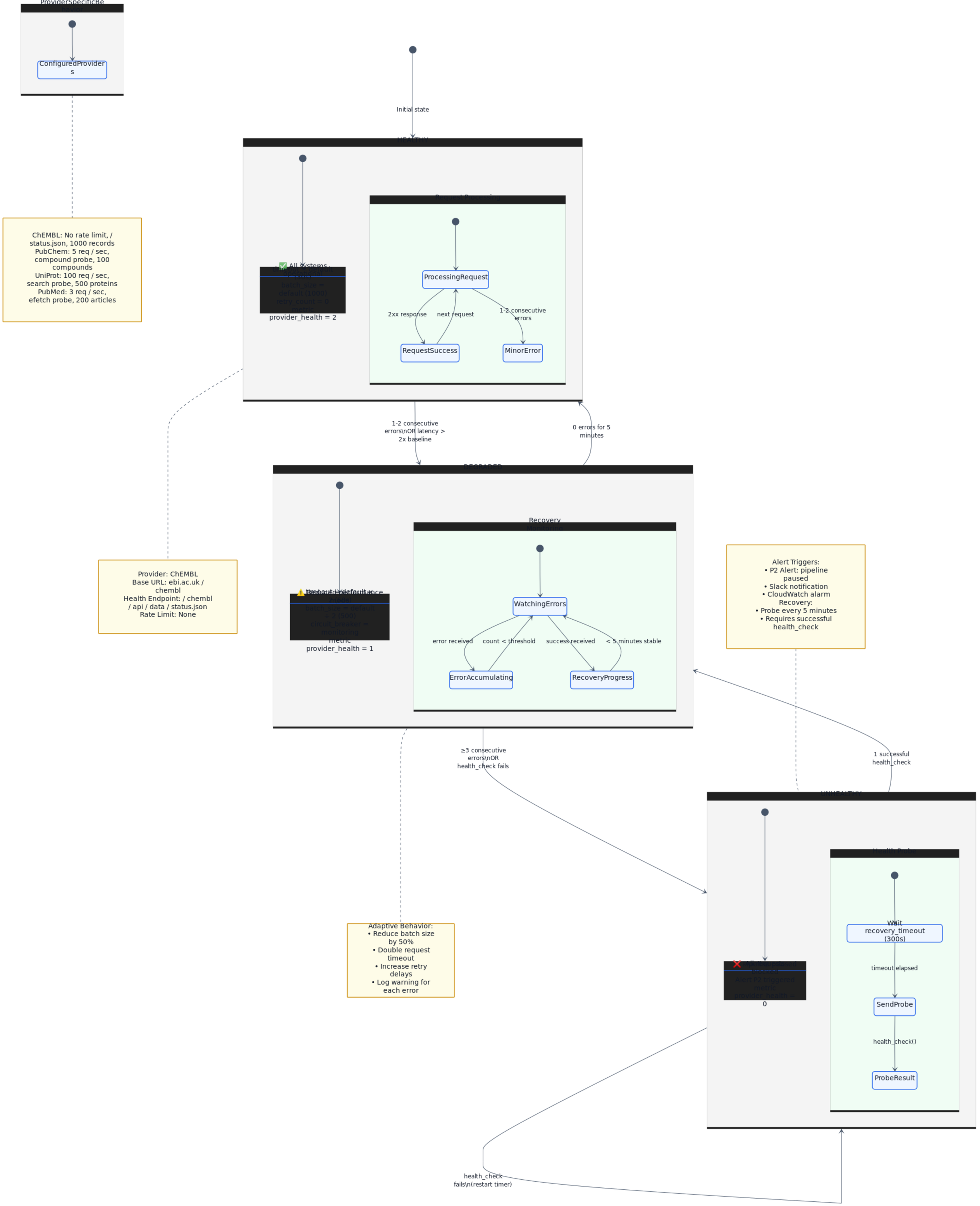
📁 quarantine / Parquet

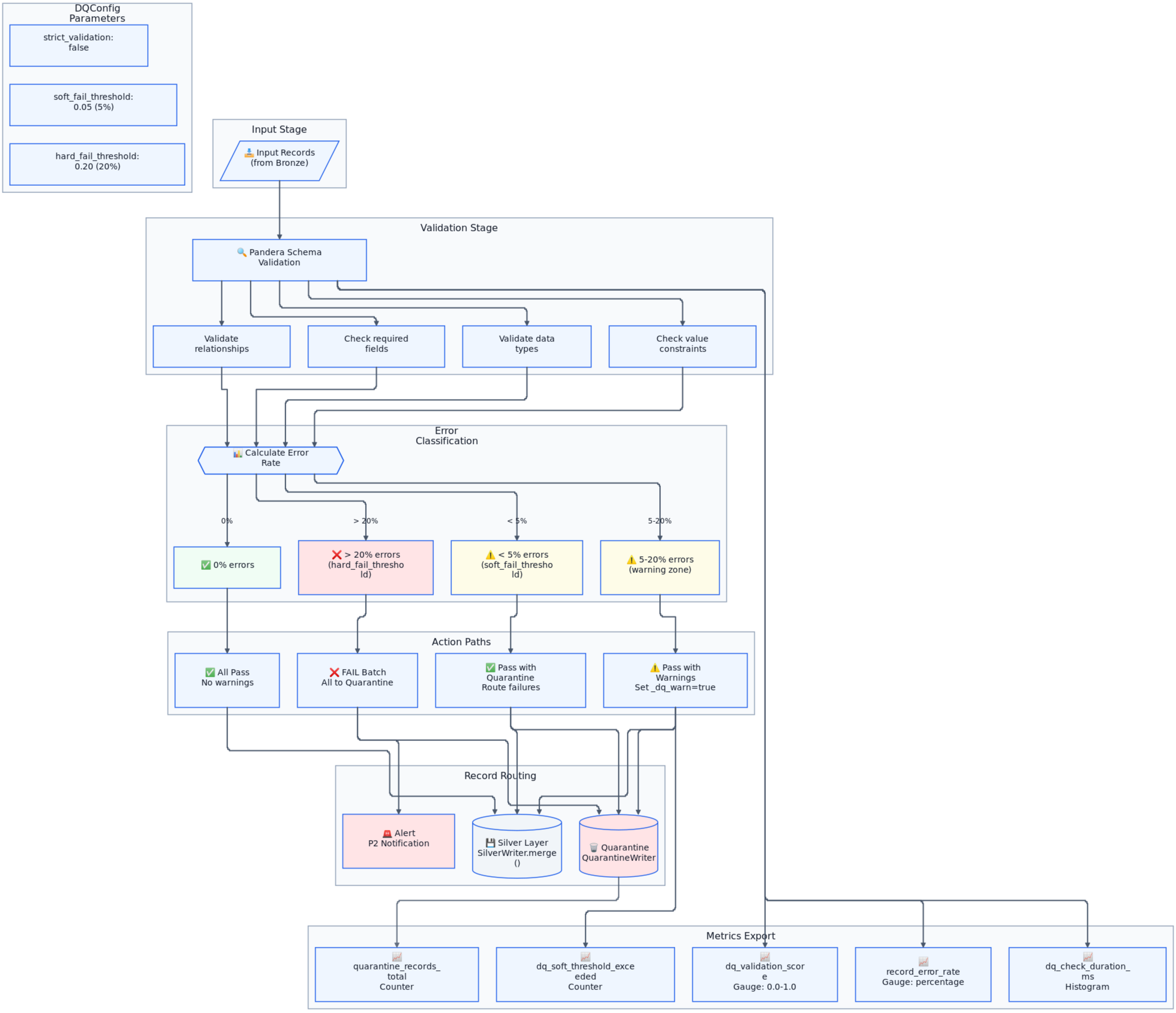
📁 checkpoints / JSON

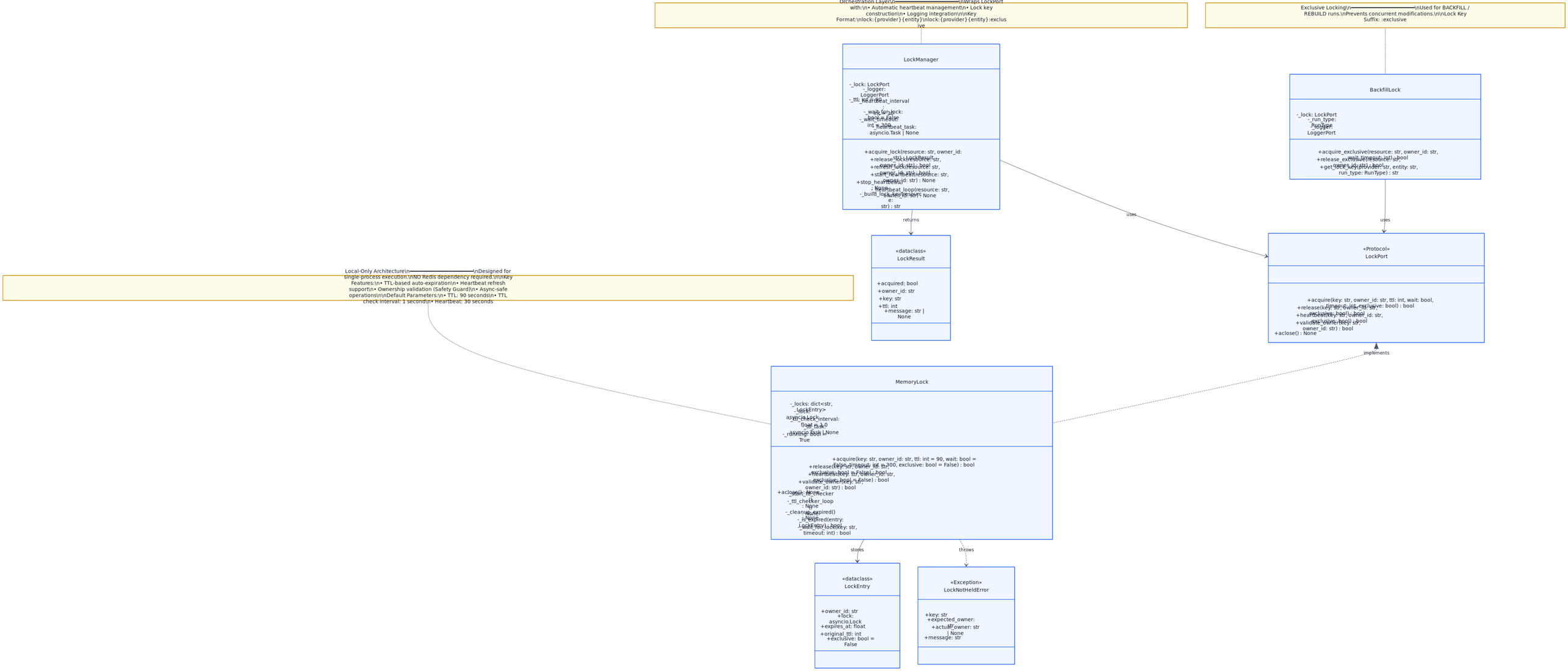
lock lock lock lock

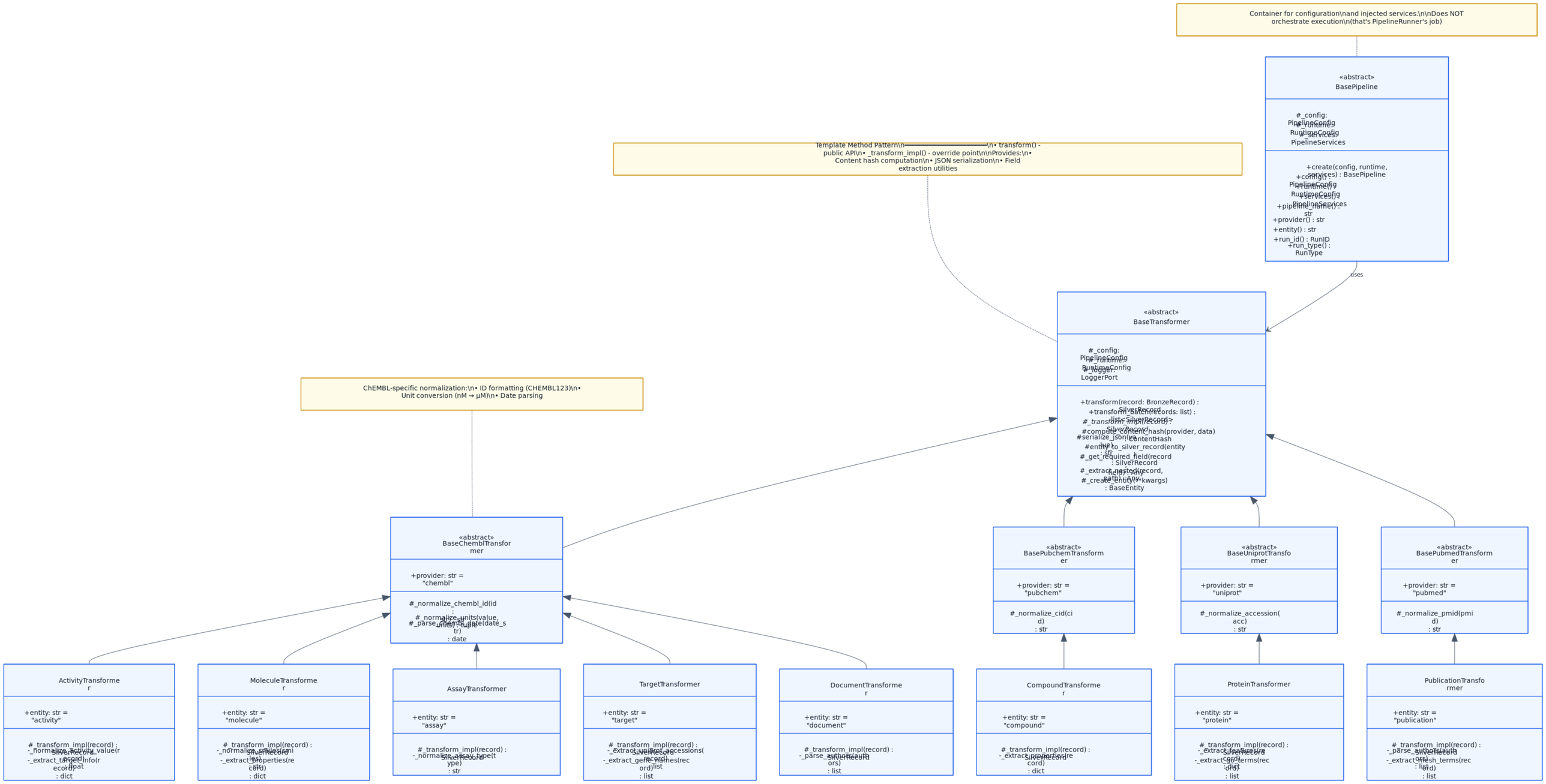
MemoryLock

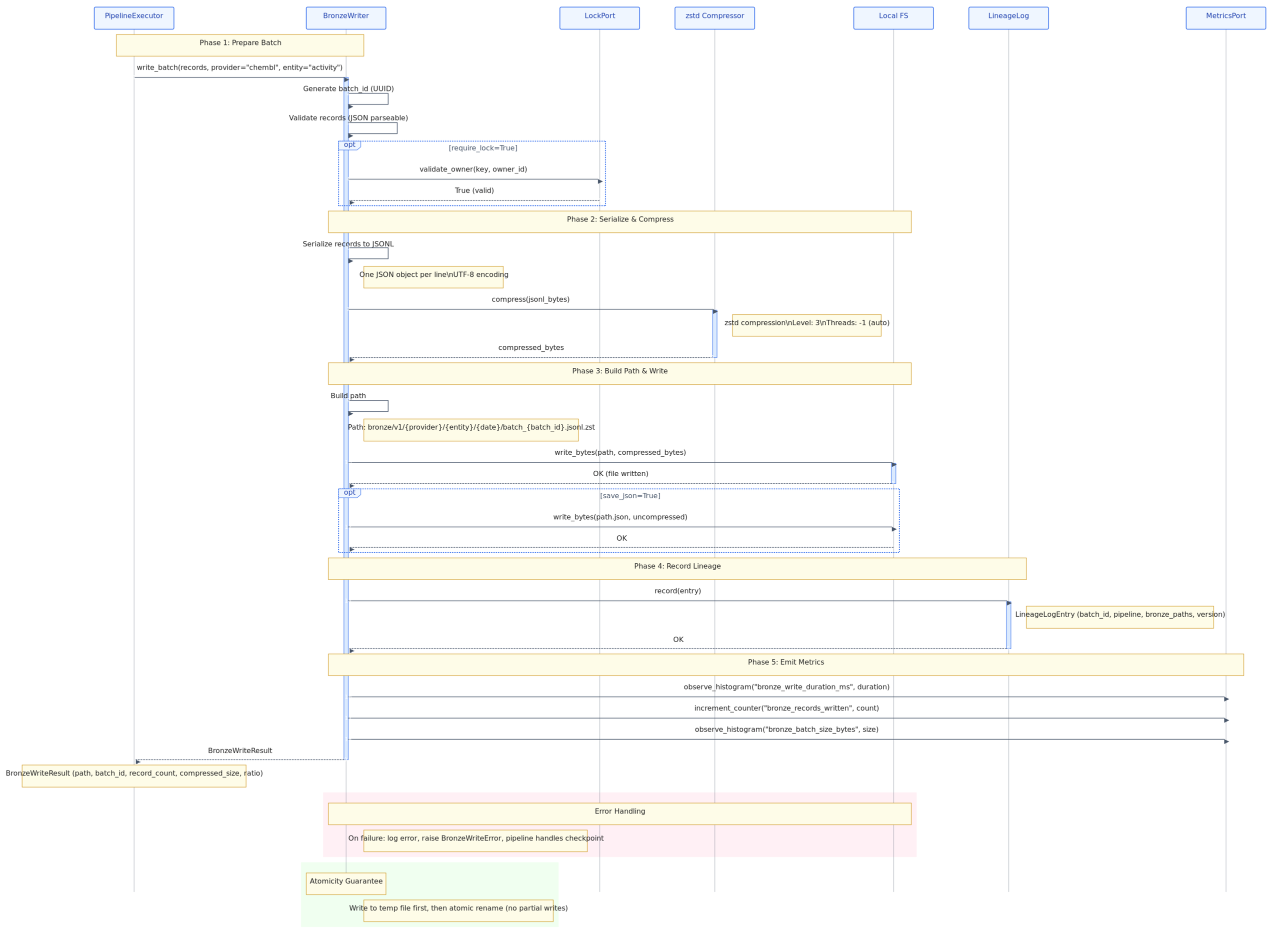

LineageLogEntry
+batch_id: BatchID
+pipeline: str
+bronze_paths: list<str>
+transform_version: str
+run_params: dict
+created_at: datetime
All entities inherit lineage fields:\n- entity_id (business key)\n- content_hash (SHA256)\n- run_id (correlation)\n- run_type (INCREMENTAL|BACKFILL|REBUILD)\n- ingestion_ts (timestamp)
Failed records during DQ validation\nRetention: 30 days\nStates: NEW -> IGNORED | REPROCESSED
«abstract»
BaseEntity
+entity_id: EntityID
+content_hash: ContentHash
+run_id: RunID
+run_type: RunType
+ingestion_ts: datetime
+source_batch_id: BatchID
+dq_warnings: bool = False
QuarantineRecord
+ingestion_ts: datetime
+pipeline: str
+error_code: str
+error_message: str
+payload: str
+payload_hash: str
+bronze_batch_id: BatchID
+bronze_file_uri: str
+dq_status: str = "NEW"
Activity
+activity_id: str
+assay_id: str
+molecule_id: str
+target_id: str | None
+standard_type: str
+standard_value: float | None
+standard_units: str | None
+standard_relation: str | None
+pchembl_value: float | None
+data_validity_comment: str | None
+activity_comment: str | None
+potential_duplicate: bool | None
Unified publication fields (ADR-024):\n- Cross-ref IDs: pmid, doi, pmc_id\n- Metrics: citations_received, citations_made\n- Year range: 1500-2100\n- ORCID validation: author_orcids
Molecule
+molecule_id: str
+pref_name: str | None
+molecule_type: str | None
+max_phase: int | None
+therapeutic_flag: bool | None
+dosed_ingredient: bool | None
+canonical_smiles: str | None
+standard_inchi: str | None
+inchi_key: str | None
+molecular_formula: str | None
+molecular_weight: float | None
+alogp: float | None
+psa: float | None
+hba: int | None
+hbd: int | None
+num_ro5_violations: int | None
+parent_molecule_id: str | None
«abstract»
PublicationEntityBase
+doi: str | None
+pmid: str | None
+pmc_id: str | None
+title: str | None
+abstract: str | None
+author_list: str | None
+author_orcids: str | None
+journal: str | None
+publication_year: int | None
+publication_date: str | None
+publication_type: str | None
+language: str | None
+page_count: str | None
+page_range: str | None
+citations_received: int | None
+citations_made: int | None
+is_oa: bool | None
+_looking_method: str
+_original_id: str | None
+_source: str
Assay
+assay_id: str
+assay_type: str
+description: str | None
+assay_organism: str | None
+assay_strain: str | None
+assay_cell_type: str | None
+assay_subcellular_fraction: str | None
+target_id: str | None
+document_id: str | None
+src_id: int | None
+confidence_score: int | None
Target
+target_id: str
+target_type: str
+pref_name: str | None
+organism: str | None
+tax_id: int | None
+uniprot_accessions: list<str> | None
+gene_symbols: list<str> | None
+ensembl_components: list<str> | None
+species_group_flag: bool | None
UniprotTarget
+target_accession: str
+entry_name: str
+protein_name: str
+organism: str
+tax_id: int
+sequence: str
+sequence_length: int
+gene_names: list<str> | None
+ec_numbers: list<str> | None
+features: dict
+subcellular_locations: list<str> | None
SemanticScholarPublication
+paper_id: str
+corpus_id: int | None
+doi: str | None
+title: str | None
+influential_citation_count: int | None
+author_list: str | None
+author_orcids: str | None
PubchemMolecule
+molecule_cid: str
+iupac_name: str | None
+canonical_smiles: str | None
+isomeric_smiles: str | None
+inchi: str | None
+inchikey: str | None
+molecular_weight: float | None
+molecular_formula: str | None
+xlogp: float | None
+exact_mass: float | None
+monoisotopic_mass: float | None
+tpsa: float | None
+complexity: float | None
+h_bond_donor_count: int | None
+h_bond_acceptor_count: int | None
+rotatable_bond_count: int | None
+heavy_atom_count: int | None
ChemblPublication
+document_chembl_id: str
+volume: str | None
+issue: str | None
+src_id: int | None
+chembl_release: str | None
+creation_date: str | None
OpenAlexPublication
+openalex_id: str
+mag_id: str | None
+fwci: float | None
+is_retracted: bool | None
+primary_topic: dict | None
+author_openalex_ids: str | None
CrossRefPublication
+doi: str
+publisher: str | None
+issn: str | None
+license_url: str | None
+subject_keywords: list<str> | None
+author_emails: str | None
+reference_dois: str | None
PubMedPublication
+pmid: str
+pmc_id: str | None
+journal_name_short: str | None
+issn: str | None
+subject_mesh: list<str> | None
+subject_keywords: list<str> | None
+publication_types: list<str> | None
+author_count: int | None
extends
molecule_id
assay_id
target_id (nullable)
inchikey match
document_chembl_id (nullable)
parent_molecule_id (self-ref)
doi match
pmid match
uniprot_accessions

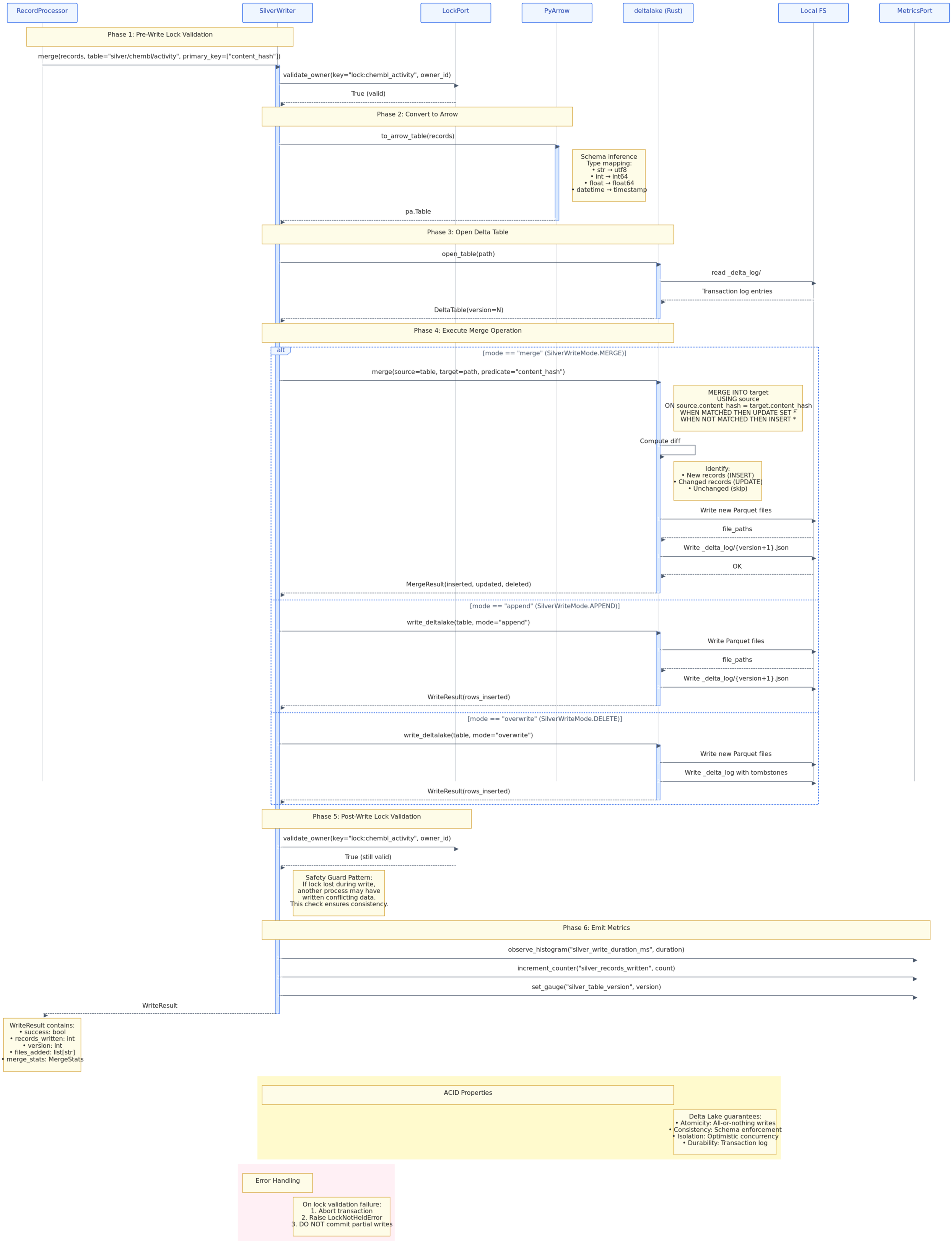

ProviderSpecificBehavior
ConfiguredProviders
ChEMBL: No rate limit, / status.json, 1000 records
PubChem: 5 req / sec, compound probe, 100 compounds
UniProt: 100 req / sec, search probe, 500 proteins
PubMed: 3 req / sec, efetch probe, 200 articles
Initial state
HEALTHY
✅ All systems operational
batch_size = default (1000)
retry_count = 0
metric
provider_health = 2
Request Processing
ProcessingRequest
2xx response
next request
1-2 consecutive errors
RequestSuccess
MinorError
1-2 consecutive errors\nOR latency > 2x baseline
0 errors for 5 minutes
DEGRADED
⚠️ Reduced performance
batch_size = default ÷ 2 (500)
circuit_breaker = monitoring
metric
provider_health = 1
Recovery Monitoring
WatchingErrors
error received
count < threshold
success received
< 5 minutes stable
ErrorAccumulating
RecoveryProgress
Provider: ChEMBL
Base URL: ebi.ac.uk / chembl
Health Endpoint: / chembl / api / data / status.json
Rate Limit: None
Alert Triggers:
• P2 Alert: pipeline paused
• Slack notification
• CloudWatch alarm
Recovery:
• Probe every 5 minutes
• Requires successful health_check
≥3 consecutive errors\nOR health_check fails
1 successful health_check
Adaptive Behavior:
• Reduce batch size by 50%
• Double request timeout
• Increase retry delays
• Log warning for each error
UNHEALTHY
❌ pipeline_paused = true
Alert P2 triggered
metric
provider_health =
0
Health Probe
Wait recovery_timeout (300s)
timeout elapsed
SendProbe
health_check()
ProbeResult
health_check fails\n(restart timer)

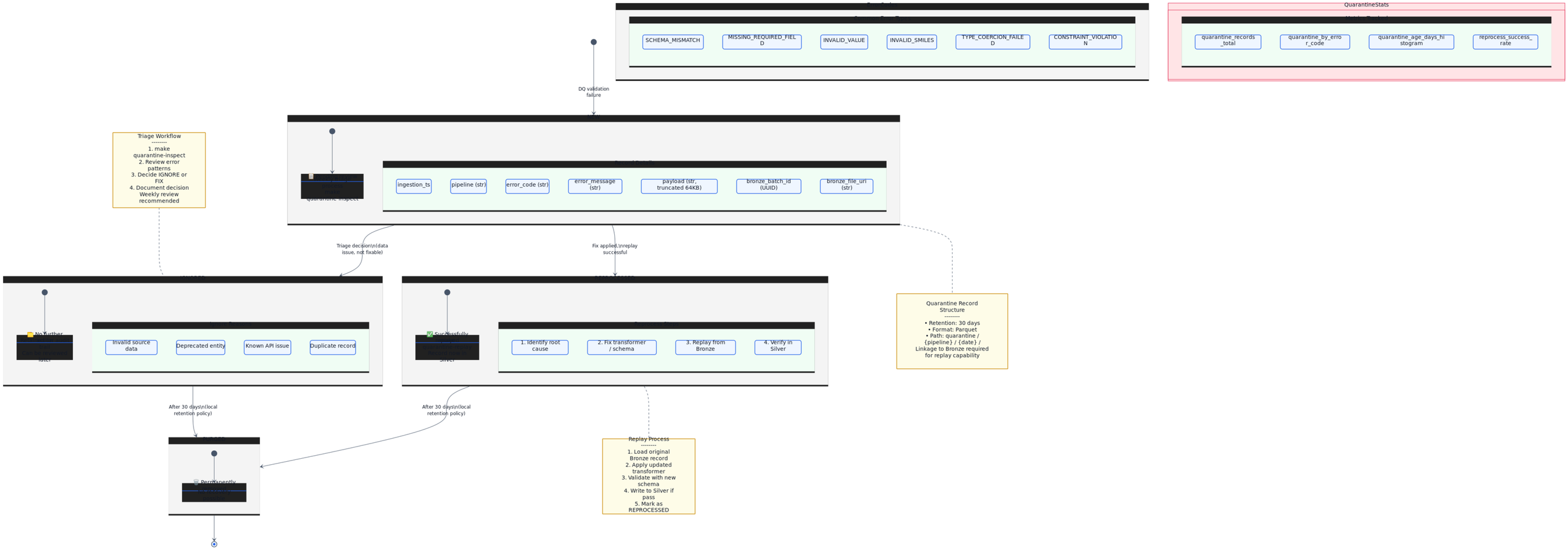

DQConfig Parameters
strict_validation: false
soft_fail_threshold: 0.05 (5%)
hard_fail_threshold: 0.20 (20%)
Input Stage
Input Records (from Bronze)
Validation Stage
Pandera Schema Validation
Validate relationships
Check required fields
Validate data types
Check value constraints
Error Classification
Calculate Error Rate
0%
> 20%
< 5%
5-20%
0% errors
> 20% errors (hard_fail_threshold)
< 5% errors (soft_fail_threshold)
5-20% errors (warning zone)
Action Paths
All Pass No warnings
FAIL Batch All to Quarantine
Pass with Quarantine Route failures
Pass with Warnings Set _dq_warn=true
Record Routing
Alert P2 Notification
Silver Layer SilverWriter.merge()
Quarantine QuarantineWriter
Metrics Export
quarantine_records_total Counter
dq_soft_threshold_exceeded Counter
dq_validation_score Gauge: 0.0-1.0
record_error_rate Gauge: percentage
dq_check_duration_ms Histogram


Orchestration Layer\n━━━━━━━━━━━━━━━━━━\nWraps LockPort with:\n• Automatic heartbeat management\n• Lock key construction\n• Logging integration\n\nKey Format:\nlock:{provider}:{entity}\nlock:{provider}:{entity}:exclusive
LockManager
-_lock: LockPort
-_logger: LoggerPort
-_ttl: int
-_heartbeat_interval
-_wait_for_lock: bool = False
-_wait_timeout: int = 0
-_heartbeat_task: asyncio.Task | None
+acquire_lock(resource: str, owner_id: str) : LockResult
+release_lock(resource: str, owner_id: str) : bool
+renew_lock(resource: str, owner_id: str) : bool
+start_heartbeat(resource: str, owner_id: str) : None
+stop_heartbeat() : None
-_heartbeat_loop(resource: str, owner_id: str) : None
-_build_lock_key(resource: str) : str
returns
«dataclass»
LockResult
+acquired: bool
+owner_id: str
+key: str
+ttl: int
+message: str | None
uses
Exclusive Locking\n━━━━━━━━━━━━━━━━━━\nUsed for BACKFILL / REBUILD runs.\nPrevents concurrent modifications.\n\nLock Key Suffix: :exclusive
BackfillLock
-_lock: LockPort
-_run_type: RunType
-_logger: LoggerPort
+acquire_exclusive(resource: str, owner_id: str, wait_timeout: int) : bool
+release_exclusive(resource: str, owner_id: str) : bool
+get_lock_key(provider: str, entity: str, run_type: RunType) : str
uses
«Protocol»
LockPort
+acquire(key: str, owner_id: str, ttl: int, wait: bool, timeout: int, exclusive: bool) : bool
+release(key: str, owner_id: str, exclusive: bool) : bool
+heartbeat(key: str, owner_id: str, ttl: int, exclusive: bool) : bool
+validate_owner(key: str, owner_id: str) : bool
+aclose() : None
implements
Local-Only Architecture\n━━━━━━━━━━━━━━━━━━\nDesigned for single-process execution.\nNO Redis dependency required.\n\nKey Features:\n• TTL-based auto-expiration\n• Heartbeat refresh support\n• Ownership validation (Safety Guard)\n• Async-safe operations\n\nDefault Parameters:\n• TTL: 90 seconds\n• TTL check interval: 1 second\n• Heartbeat: 30 seconds
MemoryLock
-_locks: dict<str, _LockEntry>
-_lock: asyncio.Lock
-_ttl_checker_interval: float = 1.0
-_ttl_task: asyncio.Task | None
-_running: bool = True
+acquire(key: str, owner_id: str, ttl: int = 90, wait: bool = False, timeout: int = 300, exclusive: bool = False) : bool
+release(key: str, owner_id: str, exclusive: bool = False) : bool
+heartbeat(key: str, owner_id: str, ttl: int = 90, exclusive: bool = False) : bool
+validate_owner(key: str, owner_id: str) : bool
+aclose() : None
-_ttl_checker_loop() : None
-_cleanup_expired() : None
-_is_expired(entry: _LockEntry) : bool
-_wait_for_lock(key: str, timeout: int) : bool
stores
throws
«dataclass»
LockEntry
+owner_id: str
+lock: asyncio.Lock
+expires_at: float
+original_ttl: int
+exclusive: bool = False
«Exception»
LockNotHeldError
+key: str
+expected_owner: str
+actual_owner: str | None
+message: str

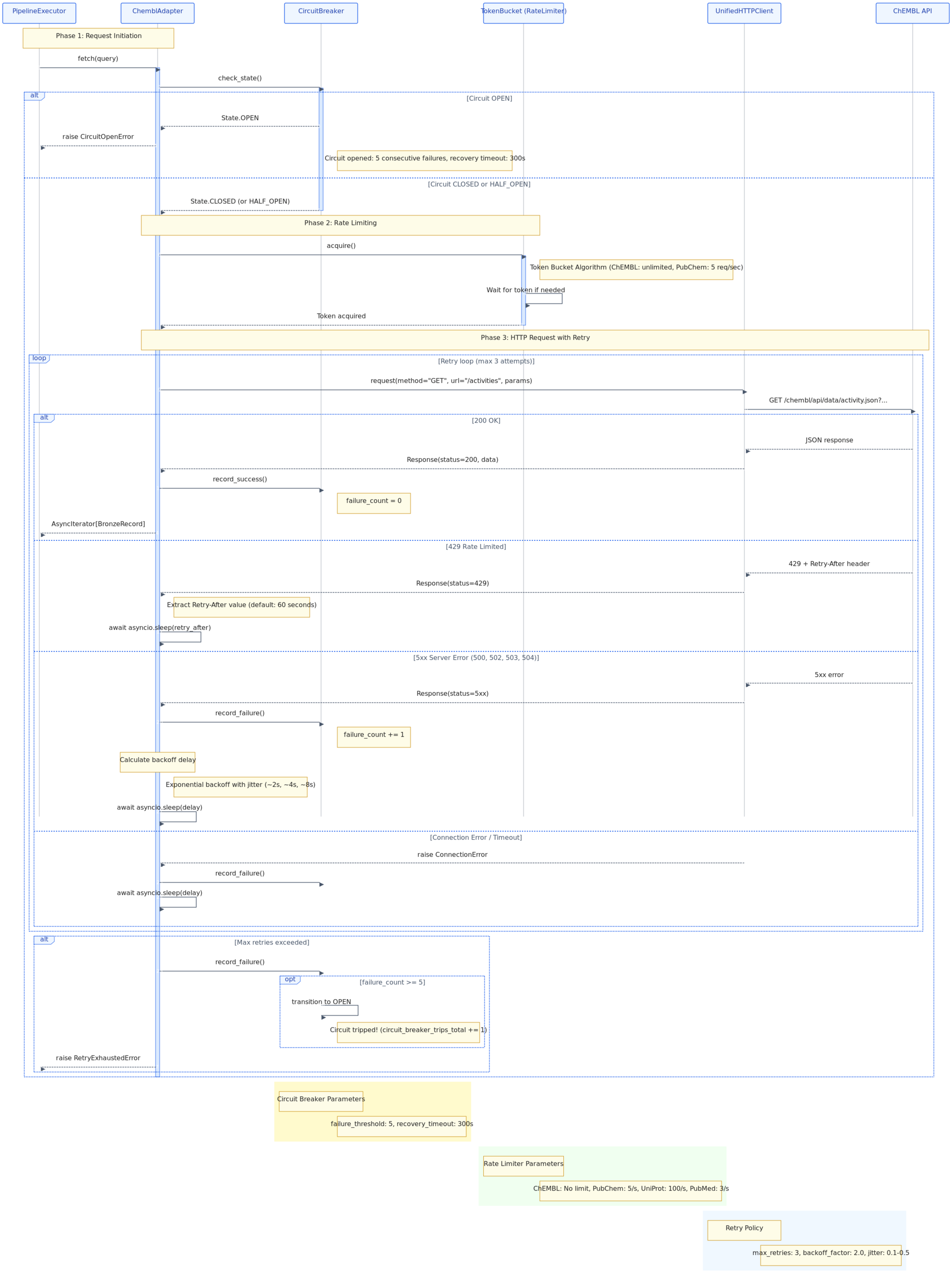

Container for configuration\nand injected services.\n\nDoes NOT orchestrate execution\n(that's PipelineRunner's job)
«abstract»
BasePipeline
_config: PipelineConfig
_runtime: RuntimeConfig
_services: PipelineServices
+create(config, runtime, services) : BasePipeline
+config() : PipelineConfig
+runtime() : RuntimeConfig
+services() : PipelineServices
+pipeline_name() : str
+provider() : str
+entity() : str
+run_id() : RunID
+run_type() : RunType
uses
Template Method Pattern\n━━━━━━━━━━━━━━━━━━━━\n• transform() - public API\n• _transform_impl() - override point\n\nProvides:\n• Content hash computation\n• JSON serialization\n• Field extraction utilities
«abstract»
BaseTransformer
_config: PipelineConfig
_runtime: RuntimeConfig
_logger: LoggerPort
+transform(record: BronzeRecord) : SilverRecord
+transform_batch(records: list) : list[SilverRecord]
_transform_impl(record) : SilverRecord
#compute_content_hash(provider, data) : ContentHash
#serialize_json(value) : str
#entity_to_silver_record(entity) : SilverRecord
_get_required_field(record, path) : Any
_extract_nested(record, path) : Any
_create_entity(**kwargs) : BaseEntity
ChEMBL-specific normalization:\n• ID formatting (CHEMBL123)\n• Unit conversion (nM → μM)\n• Date parsing
«abstract»
BaseChemblTransformer
+provider: str = "chembl"
#_normalize_chembl_id(id) : str
#_normalize_units(value, units) : float
#_parse_chembl_date(date_str) : date
«abstract»
BasePubchemTransformer
+provider: str = "pubchem"
#_normalize_cid(cid) : str
«abstract»
BaseUniprotTransformer
+provider: str = "uniprot"
#_normalize_accession(acc) : str
«abstract»
BasePubmedTransformer
+provider: str = "pubmed"
#_normalize_pmid(pmid) : str
ActivityTransformer
+entity: str = "activity"
_transform_impl(record) : SilverRecord
-_normalize_activity_value(record) : float
-_extract_target_info(record) : dict
MoleculeTransformer
+entity: str = "molecule"
_transform_impl(record) : SilverRecord
-_normalize_smiles(smi) : str
-_extract_properties(record) : dict
AssayTransformer
+entity: str = "assay"
_transform_impl(record) : SilverRecord
-_normalize_assay_type(type) : str
TargetTransformer
+entity: str = "target"
_transform_impl(record) : SilverRecord
-_extract_uniprot_accessions(record) : list
-_extract_gene_names(record) : list
DocumentTransformer
+entity: str = "document"
_transform_impl(record) : SilverRecord
-_parse_author_list(authors) : list
CompoundTransformer
+entity: str = "compound"
_transform_impl(record) : SilverRecord
-_extract_structure(record) : dict
ProteinTransformer
+entity: str = "protein"
_transform_impl(record) : SilverRecord
-_extract_features(record) : dict
-_extract_go_terms(record) : list
PublicationTransformer
+entity: str = "publication"
_transform_impl(record) : SilverRecord
-_parse_author_list(authors) : list
-_extract_mesh_terms(record) : list

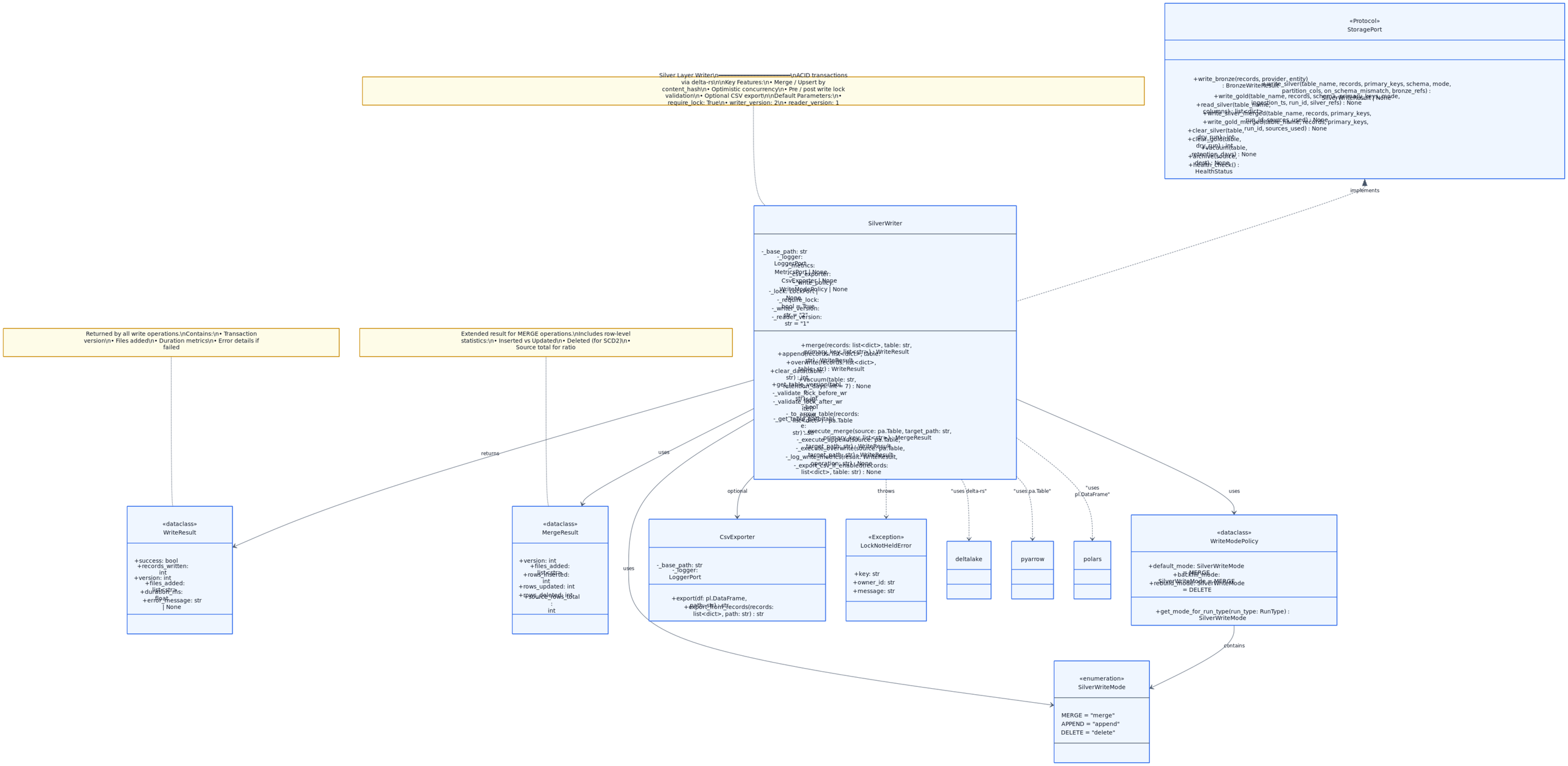

PipelineExecutor
BronzeWriter
LockPort
zstd Compressor
Local FS
LineageLog
MetricsPort
Phase 1: Prepare Batch
write_batch(records, provider="chembl", entity="activity")
Generate batch_id (UUID)
Validate records (JSON parseable)
opt
[require_lock=True]
validate_owner(key, owner_id)
True (valid)
Phase 2: Serialize & Compress
Serialize records to JSONL
One JSON object per line\nUTF-8 encoding
compress(jsonl_bytes)
zstd compression\nLevel: 3\nThreads: -1 (auto)
compressed_bytes
Phase 3: Build Path & Write
Build path
Path: bronze/v1/{provider}/{entity}/{date}/batch_{batch_id}.jsonl.zst
write_bytes(path, compressed_bytes)
OK (file written)
opt
[save_json=True]
write_bytes(path.json, uncompressed)
OK
Phase 4: Record Lineage
record(entry)
LineageLogEntry (batch_id, pipeline, bronze_paths, version)
OK
Phase 5: Emit Metrics
observe_histogram("bronze_write_duration_ms", duration)
increment_counter("bronze_records_written", count)
observe_histogram("bronze_batch_size_bytes", size)
BronzeWriteResult
BronzeWriteResult (path, batch_id, record_count, compressed_size, ratio)
Error Handling
On failure: log error, raise BronzeWriteError, pipeline handles checkpoint
Atomicity Guarantee
Write to temp file first, then atomic rename (no partial writes)

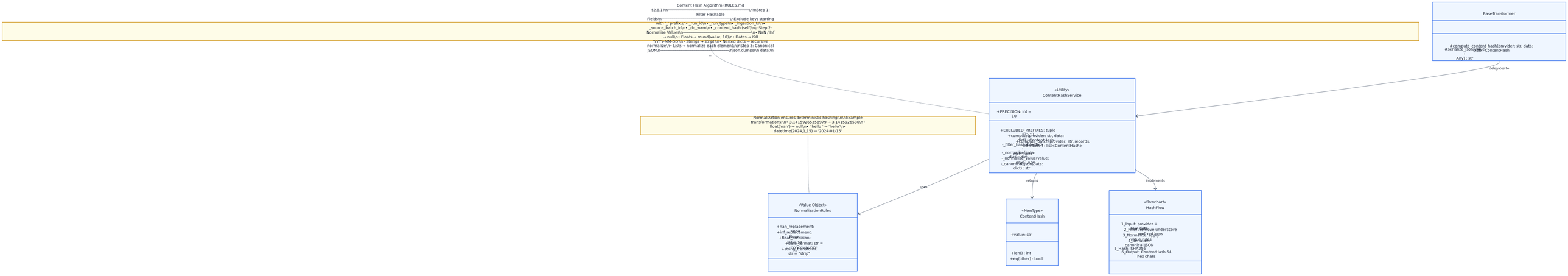

RecordProcessor
SilverWriter
LockPort
PyArrow
deltalake (Rust)
Local FS
MetricsPort
Phase 1: Pre-Write Lock Validation
merge(records, table="silver/chembl/activity", primary_key=["content_hash"])
validate_owner(key="lock:chembl_activity", owner_id)
True (valid)
Phase 2: Convert to Arrow
to_arrow_table(records)
Schema inference
Type mapping:
• str → utf8
• int → int64
• float → float64
• datetime → timestamp
pa.Table
Phase 3: Open Delta Table
open_table(path)
read _delta_log/
Transaction log entries
DeltaTable(version=N)
Phase 4: Execute Merge Operation
alt
[mode == "merge" (SilverWriteMode.MERGE)]
merge(source=table, target=path, predicate="content_hash")
MERGE INTO target
USING source
ON source.content_hash = target.content_hash
WHEN MATCHED THEN UPDATE SET *
WHEN NOT MATCHED THEN INSERT *
Compute diff
Identify:
• New records (INSERT)
• Changed records (UPDATE)
• Unchanged (skip)
Write new Parquet files
file_paths
Write _delta_log/{version+1}.json
OK
MergeResult(inserted, updated, deleted)
[mode == "append" (SilverWriteMode.APPEND)]
write_deltalake(table, mode="append")
Write Parquet files
file_paths
Write _delta_log/{version+1}.json
WriteResult(rows_inserted)
[mode == "overwrite" (SilverWriteMode.DELETE)]
write_deltalake(table, mode="overwrite")
Write new Parquet files
Write _delta_log with tombstones
WriteResult(rows_inserted)
Phase 5: Post-Write Lock Validation
validate_owner(key="lock:chembl_activity", owner_id)
True (still valid)
Safety Guard Pattern:
If lock lost during write,
another process may have
written conflicting data.
This check ensures consistency.
Phase 6: Emit Metrics
observe_histogram("silver_write_duration_ms", duration)
increment_counter("silver_records_written", count)
set_gauge("silver_table_version", version)
WriteResult
WriteResult contains:
• success: bool
• records_written: int
• version: int
• files_added: list[str]
• merge_stats: MergeStats
ACID Properties
Delta Lake guarantees:
• Atomicity: All-or-nothing writes
• Consistency: Schema enforcement
• Isolation: Optimistic concurrency
• Durability: Transaction log
Error Handling
On lock validation failure:
1. Abort transaction
2. Raise LockNotHeldError
3. DO NOT commit partial writes

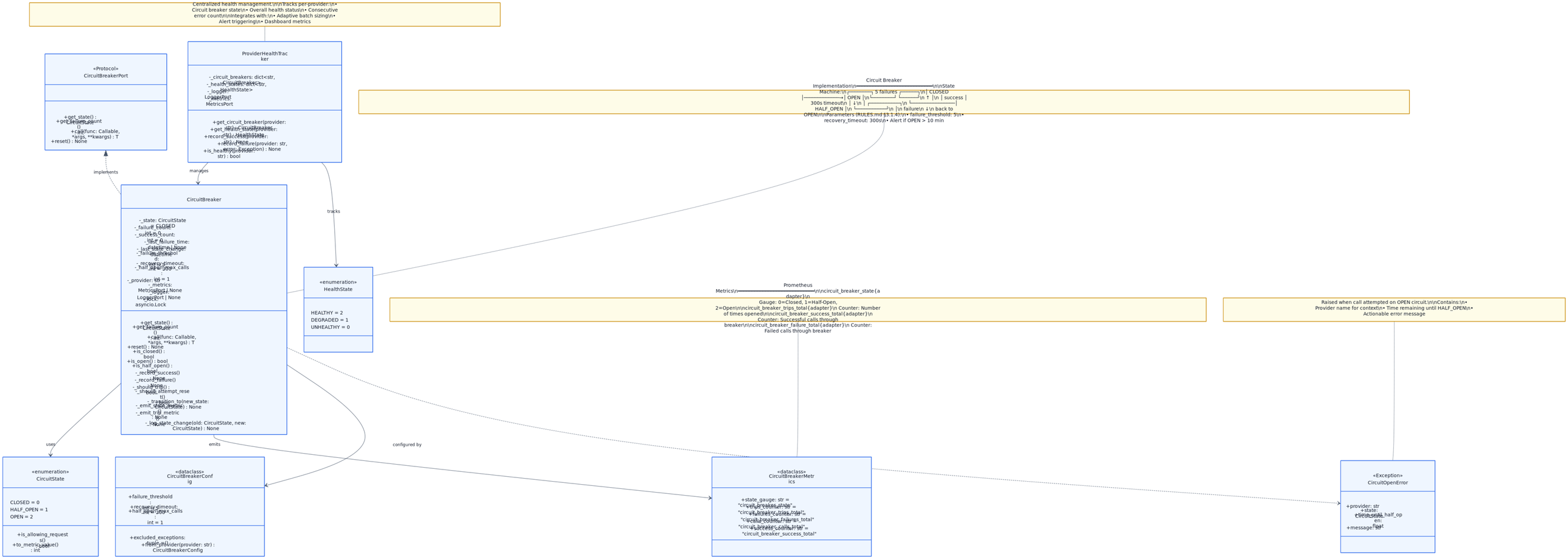

SCHEMA_MISMATCH
MISSING_REQUIRED_FIEL D
INVALID_VALUE
INVALID_SMILES
TYPE_COERCION_FAILE D
CONSTRAINT_VIOLATIO N
DQ validation failure
ingestion_ts
pipeline (str)
error_code (str)
error_message (str)
payload (str, truncated 64KB)
bronze_batch_id (UUID)
bronze_file_uri (str)
Triage Workflow
1. make quarantine-inspect
2. Review error patterns
3. Decide IGNORE or FIX
4. Document decision
Weekly review recommended
Triage decison\n(data issue, not fixable)
Fix applied,\nreplay successful
Invalid source data
Deprecated entity
Known API issue
Duplicate record
1. Identify root cause
2. Fix transformer / schema
3. Replay from Bronze
4. Verify in Silver
Quarantine Record Structure
• Retention: 30 days
• Format: Parquet
• Path: quarantine / {pipeline} / {date} / {error_code} /
Linkage to Bronze required for replay capability
After 30 days\n(local retention policy)
After 30 days\n(local retention policy)
Replay Process
1. Load original Bronze record
2. Apply updated transformer
3. Validate with new schema
4. Write to Silver if pass
5. Mark as REPROCESSED
quarantine_records_total
quarantine_by_error_code
quarantine_age_days_histogram
reprocess_success_rate
QuarantineStats

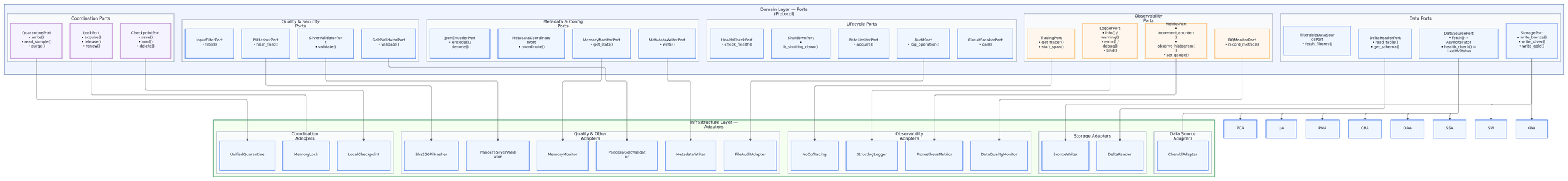
Activity Silver Schema
_run_type: str
_run_id: UUID
assay_id: str (FK)
standard_type: str
_content_hash: str
activity_id: str (PK)
molecule_id: str (FK)
_ingestion_ts: datetime
target_id: str | None (FK)
standard_units: str | None
pchembl_value: float | None
standard_value: float | None
DQ Failures
Related Entities (Silver)
External API
ChEMBL API / activities endpoint
Extract Phase
Fetch activity_id batch (ChemblAdapter)
Fetch related entities assay_id, molecule_id, target_id
Write Bronze jSONL + zstd
Record Lineage batch_id, paths
Transform Phase
Normalize units nM → µM standardization
Add metadata _run_id, _run_type, _ingestion_ts
Compute content_hash sha256(provider + canonical_json)
Create Activity entity
Validate Phase
ActivityTableSchem a (Pandera)
Check standard_value range 0 < value < 1e12
Validate SMILES if present (optional field)
Error Rate Check
Pass
Fail
Q
Load Phase
Silver Layer silver / chembl / activity / year= / month= /
Gold Layer gold / activity_by_target / target_id= /
assay_id FK
molecule_id FK
target_id FK
Assay
Molecule
Target


PipelineExecutor
ChemblAdapter
CircuitBreaker
TokenBucket (RateLimiter)
UnifiedHTTPClient
ChEMBL API
Phase 1: Request Initiation
fetch(query)
check_state()
alt
[Circuit OPEN]
State.OPEN
raise CircuitOpenError
Circuit opened: 5 consecutive failures, recovery timeout: 300s
[Circuit CLOSED or HALF_OPEN]
State.CLOSED (or HALF_OPEN)
Phase 2: Rate Limiting
acquire()
Token Bucket Algorithm (ChEMBL: unlimited, PubChem: 5 req/sec)
Wait for token if needed
Token acquired
Phase 3: HTTP Request with Retry
loop
[Retry loop (max 3 attempts)]
request(method="GET", url="/activities", params)
GET /chembl/api/data/activity.json?...
alt
[200 OK]
JSON response
Response(status=200, data)
record_success()
failure_count = 0
AsyncIterator[BronzeRecord]
[429 Rate Limited]
429 + Retry-After header
Response(status=429)
Extract Retry-After value (default: 60 seconds)
await asyncio.sleep(retry_after)
[5xx Server Error (500, 502, 503, 504)]
5xx error
Response(status=5xx)
record_failure()
failure_count += 1
Calculate backoff delay
Exponential backoff with jitter (~2s, ~4s, ~8s)
await asyncio.sleep(delay)
[Connection Error / Timeout]
raise ConnectionError
record_failure()
await asyncio.sleep(delay)
alt
[Max retries exceeded]
record_failure()
opt
[failure_count >= 5]
transition to OPEN
Circuit tripped! (circuit_breaker_trips_total += 1)
raise RetryExhaustedError
Circuit Breaker Parameters
failure_threshold: 5, recovery_timeout: 300s
Rate Limiter Parameters
ChEMBL: No limit, PubChem: 5/s, UniProt: 100/s, PubMed: 3/s
Retry Policy
max_retries: 3, backoff_factor: 2.0, jitter: 0.1-0.5


«Protocol»
StoragePort
+write_bronze(records, provider, entity) : BronzeWriteResult
+write_silver(table_name, records, primary_keys, schema_mode, partition_cols, on_schema_mismatch, bronze_refs) : SilverWriteResult | None
+write_gold(table_name, records, schema_mode, ingestion_ts, run_id, silver_refs) : None
+read_silver(table_name, columns) : list<dict>
+write_silver_merged(table_name, records, primary_keys, dry_run) : MergeResult
+write_gold_merged(table_name, records, primary_keys, dry_run) : MergeResult
+clear_silver(table, run_id, sources_used) : None
+clear_gold(table, run_id) : None
+vacuum(table, retention_hours) : None
+health_check() : HealthStatus
implements
Silver Layer Writer\n━━━━━━━━━━━━━━━━━━\nACID transactions via delta-rs\n\nKey Features:\n• Merge / Upsert by content_hash\n• Optimistic concurrency\n• Pre / post write lock validation\n• Optional CSV export\n\nDefault Parameters:\n• require_lock: True\n• writer_version: 2\n• reader_version: 1
SilverWriter
-_base_path: str
-_logger: LoggerPort
-_metrics: MetricsPort | None
-_csv_exporter: CsvExporter | None
-_write_policy: WriteModePolicy | None
-_lock_policy: LockPolicy | None
-_require_lock: bool = True
-_writer_version: str = "2"
-_reader_version: str = "1"
+merge(records: list<dict>, table: str, primary_key: list<str>) : WriteResult
+append(records: list<dict>, table: str) : WriteResult
+overwrite(records: list<dict>, table: str) : WriteResult
+clear_data(table: str) : int
+vacuum(table: str, retention_hours: int = 7) : None
+get_table_version(table: str) : int
-_validate_lock_before_wr(str) : bool
-_validate_lock_after_wr(int) : bool
-_to_arrow_table(records: list<dict>) : pa.Table
-_get_table_path(tab): str
-_execute_merge(source: pa.Table, target_path: str, primary_key: list<str>) : MergeResult
-_execute_overwrite(source: pa.Table, target_path: str) : WriteResult
-_log_write(metrics: WriteResult, operation: str) : None
-_export_csv_if_enabled(records: list<dict>, table: str) : None
Returned by all write operations.\nContains:\n• Transaction version\n• Files added\n• Duration metrics\n• Error details if failed
«dataclass»
WriteResult
+success: bool
+records_written: int
+version: int
+files_added: list<str>
+duration_ms: float
+error_message: str | None
Extended result for MERGE operations.\nIncludes row-level statistics:\n• Inserted vs Updated\n• Deleted (for SCD2)\n• Source total for ratio
«dataclass»
MergeResult
+version: int
+files_added: list<str>
+rows_inserted: int
+rows_updated: int
+rows_deleted: int
+source_rows_total: int
CsvExporter
-_base_path: str
-_logger: LoggerPort
+export(df: pl.DataFrame, path: str)
+export_records(records: list<dict>, path: str) : str
«Exception»
LockNotHeldError
+key: str
+owner_id: str
+message: str
deltalake
pyarrow
polars
«dataclass»
WriteModePolicy
+default_mode: SilverWriteMode = MERGE
+backfill_mode: SilverWriteMode = MERGE
+rebuild_mode: SilverWriteMode = DELETE
+get_mode_for_run_type(run_type: RunType) : SilverWriteMode
«enumeration»
SilverWriteMode
MERGE = "merge"
APPEND = "append"
DELETE = "delete"
returns
uses
uses
optional
throws
"uses delta-rs"
"uses pa.Table"
"uses pl.DataFrame"
uses
contains


Content Hash Algorithm (RULES.md §2.8.1)\n\n──────────────────────────────────\n\nStep 1: Filter Hashable Fields\n\n──────────────────────────────────\n\nExclude keys starting with `_` prefix:\n• `_run_id`\n• `_run_type`\n• `_ingestion_ts`\n• `_source_batch_id`\n• `_dq_warn`\n• `_content_hash` (self)\n\nStep 2: Normalize Values\n\n──────────────────────────────────\n\n• NaN / Inf → null\n• Floats → round(value, 10)\n• Dates → ISO 'YYYY-MM-DD'\n• Strings → strip\n• Nested dicts → recursive normalize\n• Lists → normalize each element\n\nStep 3: Canonical JSON\n\n──────────────────────────────────\n\njson.dumps(\n data,\n ...
Normalization ensures deterministic hashing.\n\nExample transformations:\n• 3.14159265358979 → 3.1415926536\n• float('nan') → null\n• ' hello ' → 'hello'\n• datetime(2024,1,15) → '2024-01-15'
«Utility»
ContentHashService
+PRECISION: int = 10
+EXCLUDED_PREFIXES: tuple
+compute(provider: str, data: dict) : ContentHash
-_filter_hashable(dict) : dict
-_normalize(value) : Any
-_normalize_value(value: float) : Any
-_canonical_json(data: dict) : str
+compute_batch(provider: str, records: list) : list<ContentHash>
uses
returns
implements
«Value Object»
NormalizationRules
+nan_replacement: None
+inf_replacement: None
+float_precision: 10
+date_format: str = '%Y-%m-%d'
+string_strip: bool = True
str = "strip"
«NewType»
ContentHash
+value: str
+len() : int
+eq(other) : bool
«flowchart»
HashFlow
1_Input: provider + data
2_Filter: Remove underscore prefixed keys
3_Normalize: Apply rules
4_Serialize: Sorted canonical JSON
5_Hash: SHA256
6_Output: ContentHash 64 hex chars
BaseTransformer
#compute_content_hash(provider: str, data: dict) : ContentHash
#serialize_json(data: Any) : str
delegates to


Centralized health management.\n\nTracks per-provider:\n• Circuit breaker state\n• Overall health status\n• Consecutive error count\n\nIntegrates with:\n• Adaptive batch sizing\n• Alert triggering\n• Dashboard metrics
«Protocol»
CircuitBreakerPort
+get_state() : CircuitState
+get_failure_count() : int
+call(func: Callable, *args, **kwargs) : T
+reset() : None
ProviderHealthTracker
- _circuit_breakers: dict<str, CircuitBreaker>
- _health_trackers: dict<str, HealthState>
- _logger: LoggerPort
- _metrics: MetricsPort
+get_circuit_breaker(provider: str) : CircuitBreaker
+get_health_status(provider: str) : HealthState
+record_success(provider: str) : None
+record_failure(provider: str, error: Exception) : None
+is_healthy(provider: str) : bool
implements
manages
tracks
CircuitBreaker
- _state: CircuitState = CLOSED
- _failure_count: int = 0
- _success_count: int = 0
- _last_failure_time: float | None = None
- _failure_threshold: int
- _recovery_timeout: float
- _half_open_max_calls: int
- _provider: str
- _metrics: MetricsPort | None
- _logger: LoggerPort | None
- _lock: asyncio.Lock
+get_state() : CircuitState
+get_failure_count() : int
+call(func: Callable, *args, **kwargs) : T
+reset() : None
+is_closed() : bool
+is_open() : bool
+is_half_open() : bool
- _record_success() : None
- _record_failure() : None
- _should_attempt_reset() : bool
- _transition_to(new_state: CircuitState) : None
- _emit_trip_metric() : None
- _log_state_change(old: CircuitState, new: CircuitState) : None
«enumeration»
HealthState
HEALTHY = 2
DEGRADED = 1
UNHEALTHY = 0
Circuit Breaker Implementation\n\nState Machine:\n\n CLOSED \n 5 failures \n OPEN \n 300s timeout\n HALF_OPEN \n success \n failure\n back to OPEN\n\nParameters (RULES.md §3.1.4):\n• failure_threshold: 5\n• recovery_timeout: 300s\n• Alert if OPEN > 10 min
Prometheus Metrics\n\ncircuit_breaker_state{adapter}\n Gauge: 0=Closed, 1=Half-Open, 2=Open\n\ncircuit_breaker_trips_total{adapter}\n Counter: Number of times opened\n\ncircuit_breaker_success_total{adapter}\n Counter: Successful calls through breaker\n\ncircuit_breaker_failure_total{adapter}\n Counter: Failed calls through breaker
Raised when call attempted on OPEN circuit.\n\nContains:\n• Provider name for context\n• Time remaining until HALF_OPEN\n• Actionable error message
«enumeration»
CircuitState
CLOSED = 0
HALF_OPEN = 1
OPEN = 2
+is_allowing_requests() : bool
+to_metric_value() : int
«dataclass»
CircuitBreakerConfig
+failure_threshold : int = 5
+recovery_timeout : float = 300.0
+half_open_max_calls : int = 1
+excluded_exceptions : tuple = ()
+default_provider(provider: str) : CircuitBreakerConfig
«dataclass»
CircuitBreakerMetrics
+state_gauge: str = "circuit_breaker_state"
+trips_counter: str = "circuit_breaker_trips_total"
+success_counter: str = "circuit_breaker_success_total"
+failure_counter: str = "circuit_breaker_failure_total"
+call_counter: str = "circuit_breaker_calls_total"
«Exception»
CircuitOpenError
+provider: str
+state: CircuitState = half_open
+time_until: float
+message: str
uses
emits
configured by
raises


Domain Layer — Ports (Protocol)
Coordination Ports
QuarantinePort
• write()
• read_sample()
• purge()
LockPort
• acquire()
• release()
• renew()
CheckpointPort
• save()
• load()
• delete()
Quality & Security Ports
InputFilterPort
• filter()
PiiHasherPort
• hash_field()
SilverValidatorPort
• validate()
GoldValidatorPort
• validate()
Metadata & Config Ports
JsonEncoderPort
• encode() / decode()
MetadataCoordinatorPort
• coordinate()
MemoryMonitorPort
• get_stats()
MetadataWriterPort
• write()
Lifecycle Ports
HealthCheckPort
• check_health()
ShutdownPort
• is_shutting_down()
RateLimiterPort
• acquire()
AuditPort
• log_operation()
CircuitBreakerPort
• call()
Observability Ports
TracingPort
• get_tracer()
• start_span()
LoggerPort
• info() / warning()
• error() / debug()
• bind()
MetricsPort
• increment_counter()
• observe_histogram()
• set_gauge()
DQMonitorPort
• record_metrics()
Data Ports
FilterableDataSourcePort
• fetch_filtered()
DeltaReaderPort
• read_table()
• get_schema()
DataSourcePort
• fetch() → AsyncIterator
• health_check() → HealthStatus
StoragePort
• write_bronze()
• write_silver()
• write_gold()
Infrastructure Layer — Adapters
Coordination Adapters
UnifiedQuarantine
MemoryLock
LocalCheckpoint
Quality & Other Adapters
Sha256PiiHasher
PanderaSilverValidator
MemoryMonitor
PanderaGoldValidator
MetadataWriter
FileAuditAdapter
Observability Adapters
NoOpTracing
StructlogLogger
PrometheusMetrics
DataQualityMonitor
Storage Adapters
BronzeWriter
DeltaReader
Data Source Adapters
ChemblAdapter
PCA
UA
PMA
CRA
OAA
SSA
SW
GW

Legend

✅ Allowed Import

❌ Forbidden Import

Enforcement Mechanism

ai-selfreview-rules .md ARCH-001 detection scripts

import-linter Blocks PR on violation

tests / architecture / pytest architecture tests

BioETL Five-Layer Architecture

Interfaces Layer CLI (Click), HealthServer src / bioetl / interfaces /

Composition Layer GenericPipelineFactory, RunnerFactory ServicesBuilder, PipelineRegistry src / bioetl / composition /

Infrastructure Layer ChemblAdapter, PubChemAdapter SilverWriter, MemoryLock UnifiedHTTPClient, PrometheusMetrics src / bioetl / infrastructure /

Application Layer PipelineRunner, BatchExecutor PreflightService, PostrunService 23 Transformers src / bioetl / application /

Domain Layer 24 Ports (Protocol) Entities, Aggregates, Schemas Config, Types, Exceptions src / bioetl / domain /

✅ imports ❌ FORBIDDEN

Entry Point

CLI run command

bootstrap / runtime / assembly.py

Composition Factories

Logger & Observability

BootstrapLogger
• configure structlog

DQServicesFactory
• create()
→ DQ analyzers + monitor

Client & Data Source

HttpClientFactory
• create(provider)
→ UnifiedHTTPClient

DataSourceFactory
• create(provider, config)
→ DataSourcePort impl

Storage & Services

StorageFactory
• create(config)
→ BronzeWriter + SilverWriter + GoldWriter

StorageAdapter
• create_bronze()
• create_silver()
• create_gold()

ServicesBuilder
• with_storage()
• with_lock()
• with_logger()
• with_metrics()
• build() → PipelineServices

Pipeline Construction

RunnerFactory
• create_runner()
→ PipelineRunner

GenericPipelineFactory
• _pipeline_class
• _transformer_class
• _gold_schema

injects

uses

uses

injects

uses

uses

injects

injects

injects

injects

injects

injects

injects

registers

SW

GW

ML

LC

UQ

PMM

NOT

PLREG

Created Instances

Infra Instances

[A] UnifiedHTTPClient + TokenBucket + CircuitBreaker

[A] Provider Adapter (ChemblAdapter, etc.)

BronzeWriter

StructlogLogger

Application Instances

PipelineServices (frozen bundle)

PipelineRunner (fully assembled)

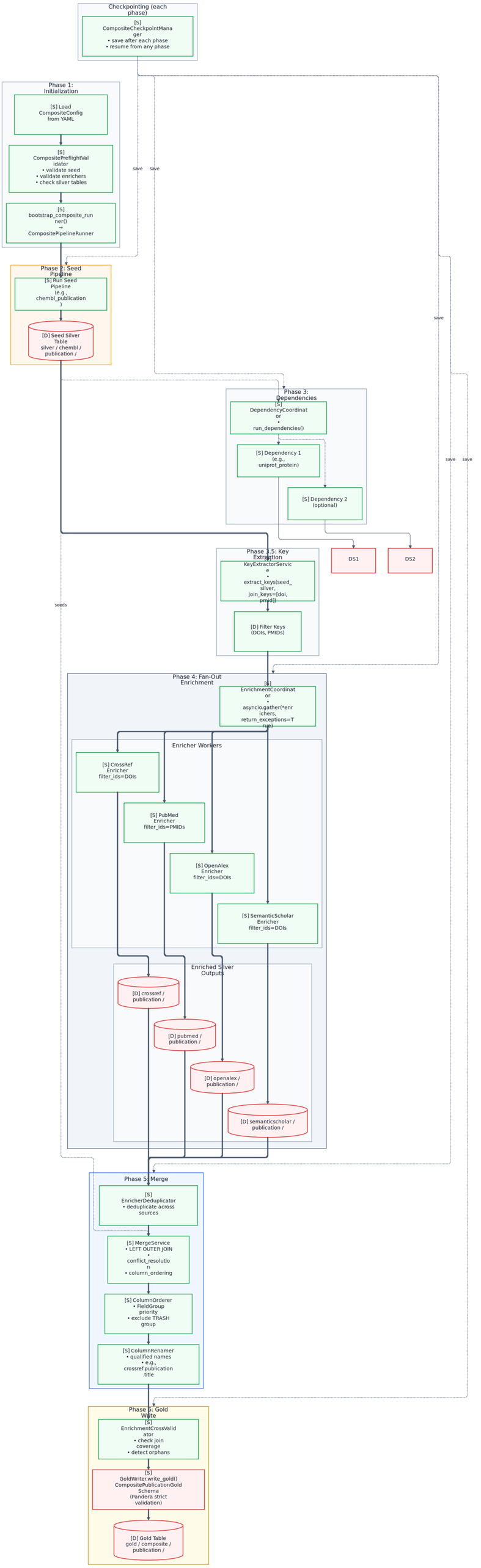
Registration

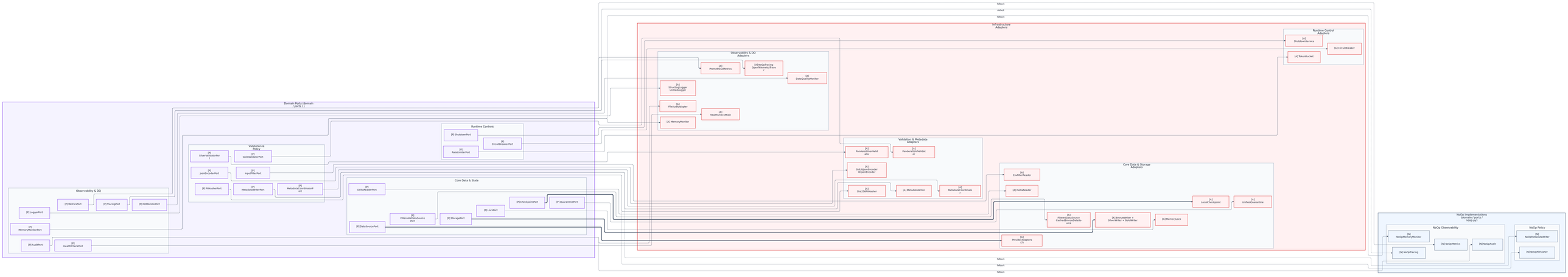
ProviderRegistry
• 8 providers registered

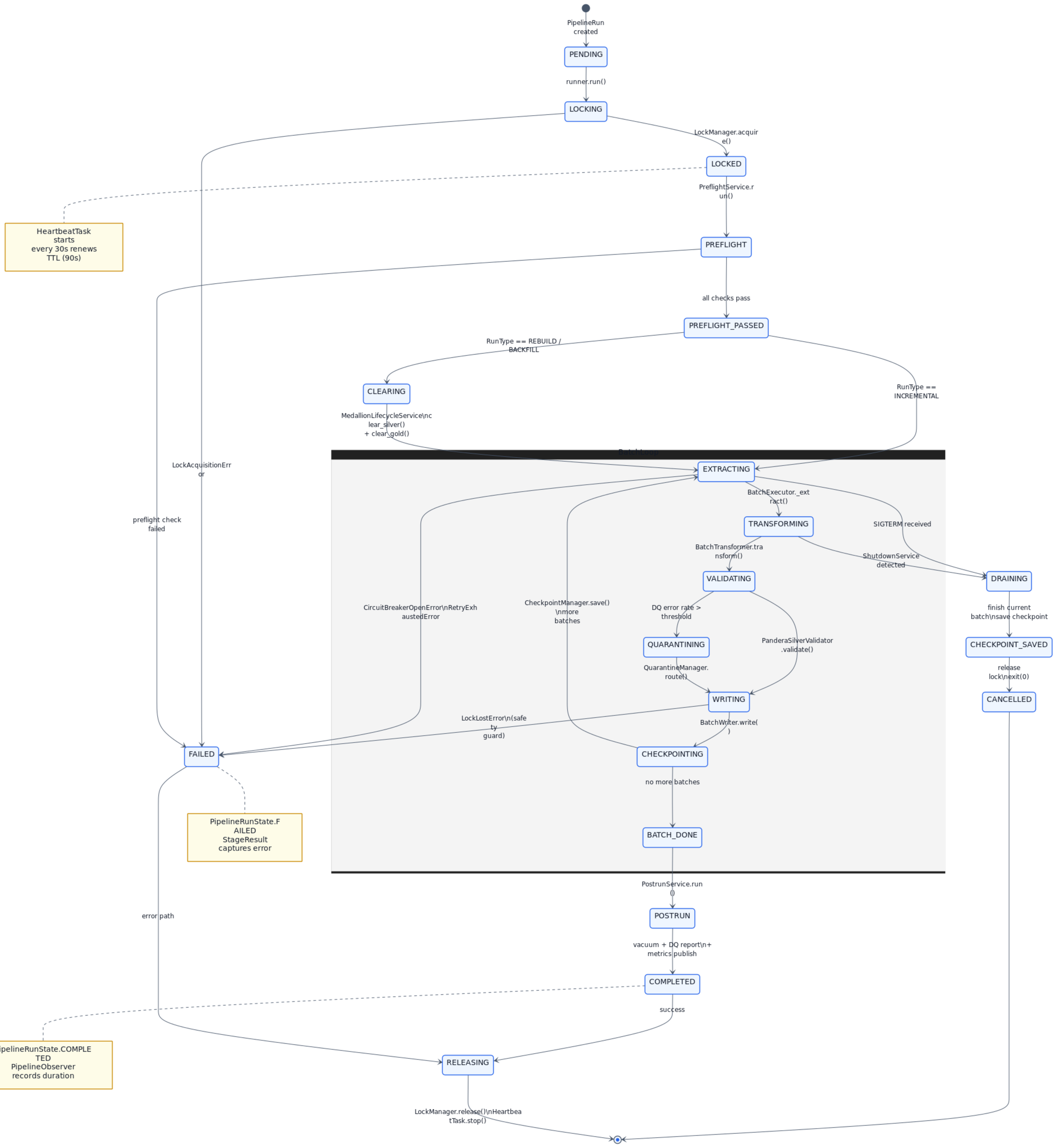
ObservabilityBundle
• logger + metrics + tracing

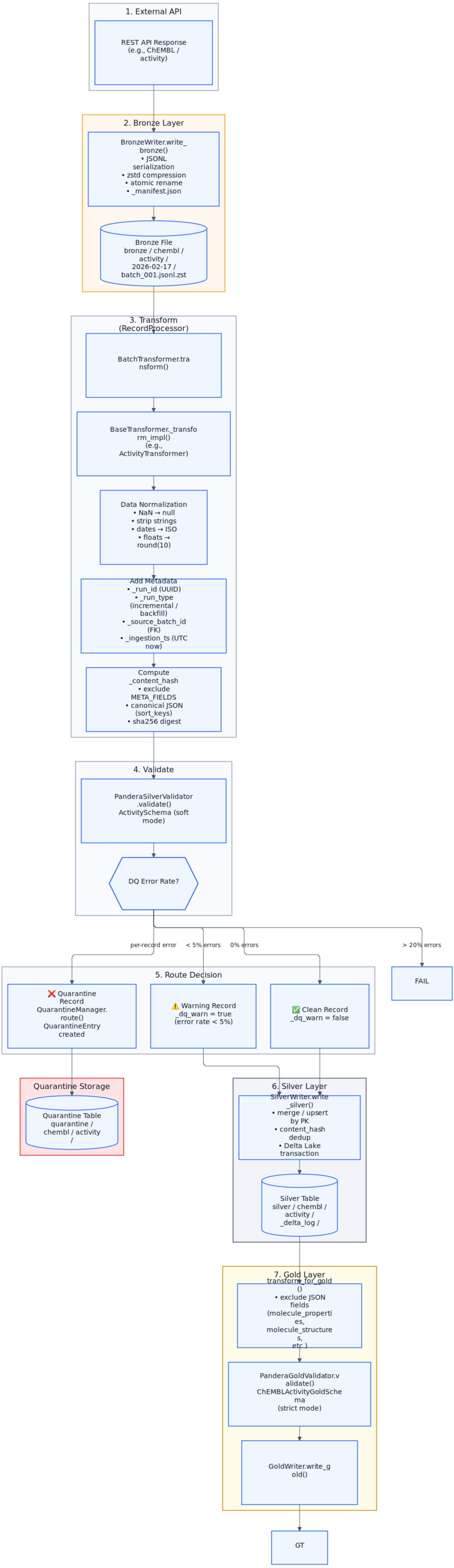
maps

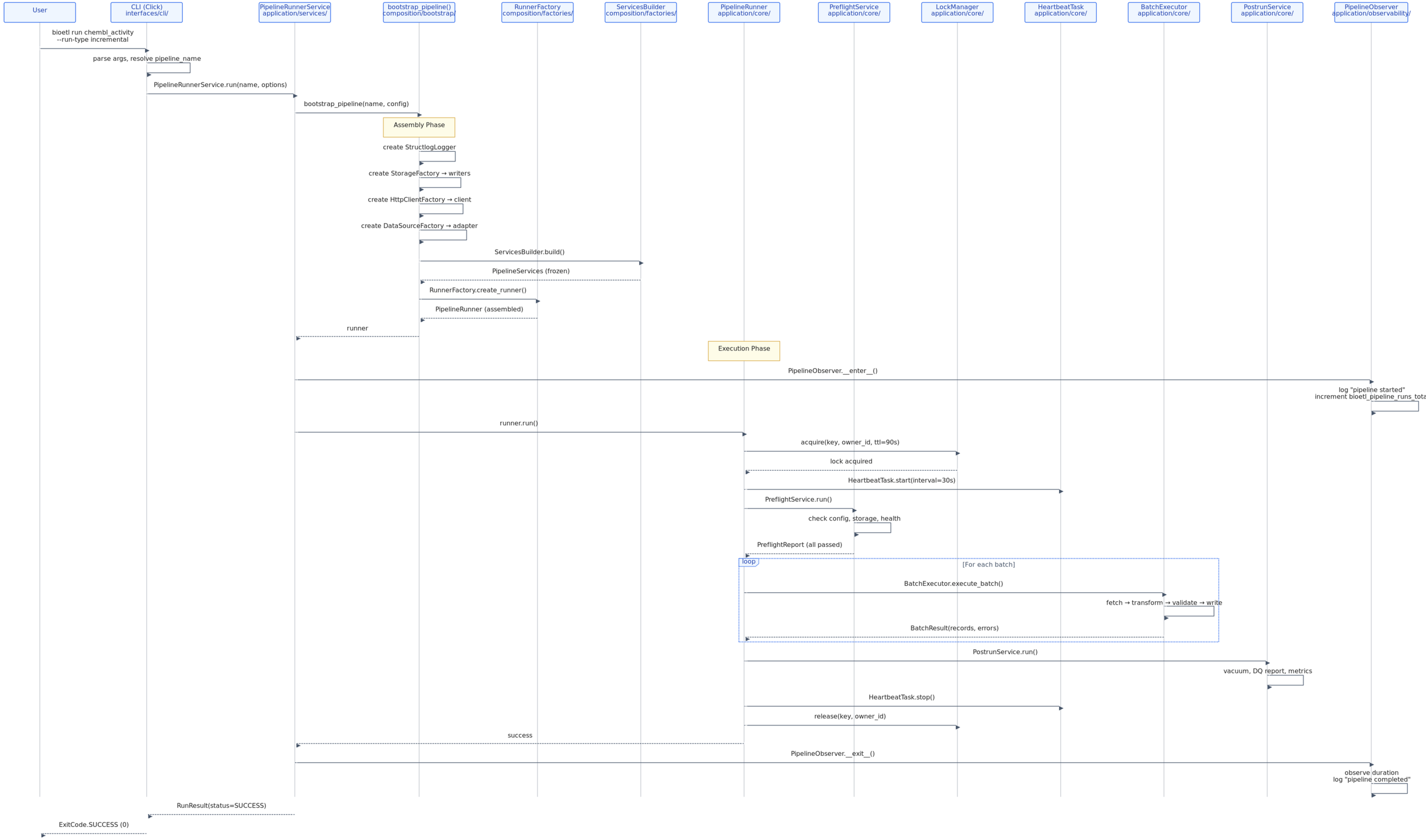
DSR

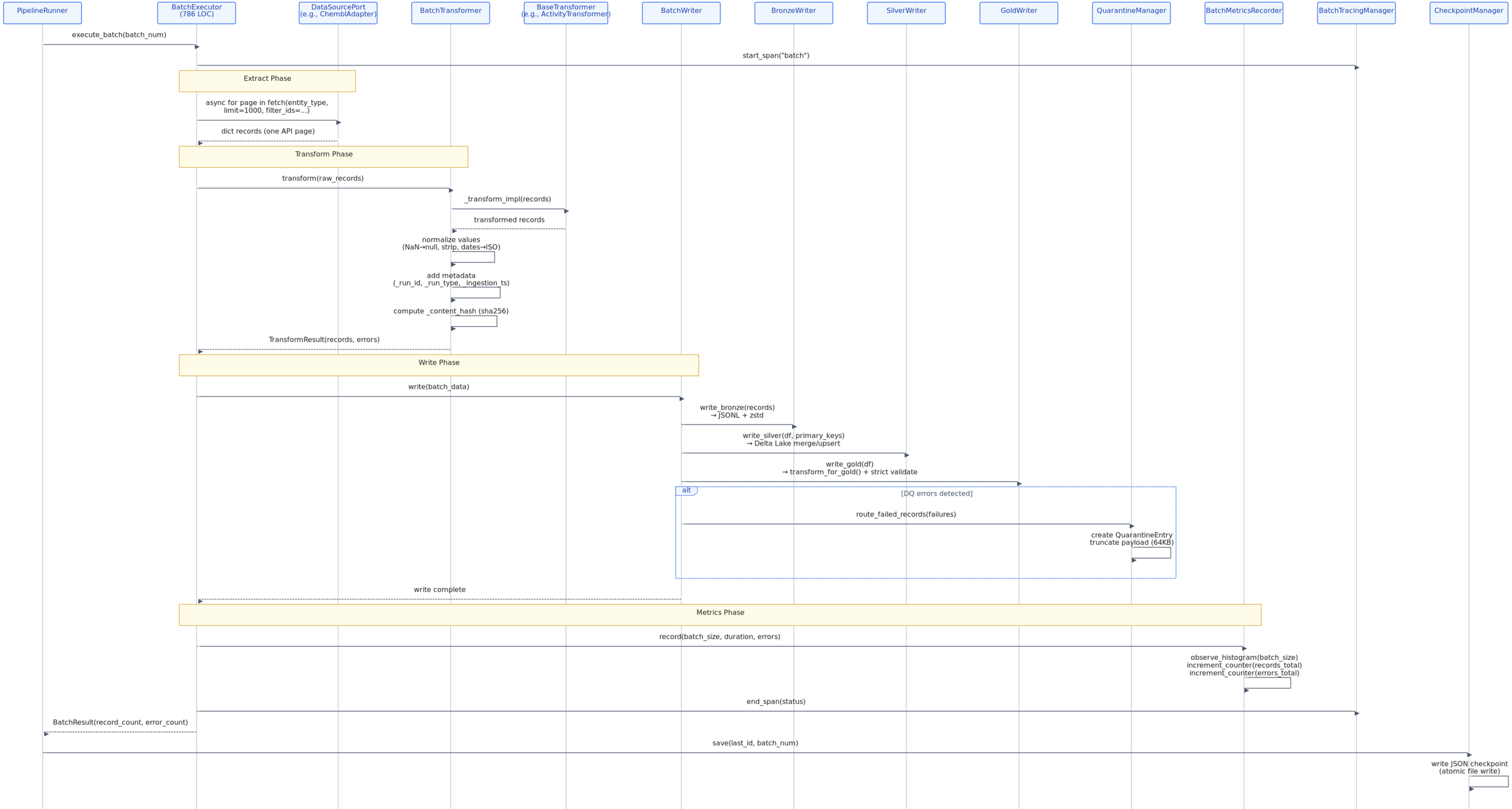

Checkpointing (each phase)
[S] CompositeCheckpointManager
• save after each phase
• resume from any phase
Phase 1: Initialization
[S] Load CompositeConfig from YAML
[S] CompositePreflightValidator
• validate seed
• validate enrichers
• check silver tables
[S] bootstrap_composite_runner()
→ CompositePipelineRunner
save
save
Phase 2: Seed Pipeline
[S] Run Seed Pipeline (e.g., chembl_publication)
[D] Seed Silver Table silver / chembl / publication /
save
Phase 3: Dependencies
[S] DependencyCoordinator
• run_dependencies()
[S] Dependency 1 (e.g., uniprot_protein)
[S] Dependency 2 (optional)
save
save
DS1
DS2
Phase 3.5: Key Extraction
[S] KeyExtractorService
• extract_keys(seed_silver, join_keys=[doi, pmid])
[D] Filter Keys (DOIs, PMIDs)
seeds
Phase 4: Fan-Out Enrichment
[S] EnrichmentCoordinator
• asyncio.gather(*enrichers, return_exceptions=True)
Enricher Workers
[S] CrossRef Enricher filter_ids=DOIs
[S] PubMed Enricher filter_ids=PMIDs
[S] OpenAlex Enricher filter_ids=DOIs
[S] SemanticScholar Enricher filter_ids=DOIs
Enriched Silver Outputs
[D] crossref / publication /
[D] pubmed / publication /
[D] openalex / publication /
[D] semanticscholar / publication /
Phase 5: Merge
[S] EnricherDeduplicator
• deduplicate across sources
[S] MergeService
• LEFT OUTER JOIN
• conflict_resolution
• column_ordering
[S] ColumnOrderer
• FieldGroup priority
• exclude TRASH group
[S] ColumnRenamer
• qualified names
• e.g., crossref.publication.title
Phase 6: Gold Write
[S] EnrichmentCrossValidator
• check join coverage
• detect orphans
[S] GoldWriter.write_gold() CompositePublicationGold Schema (Pandera strict validation)
[D] Gold Table gold / composite / publication /

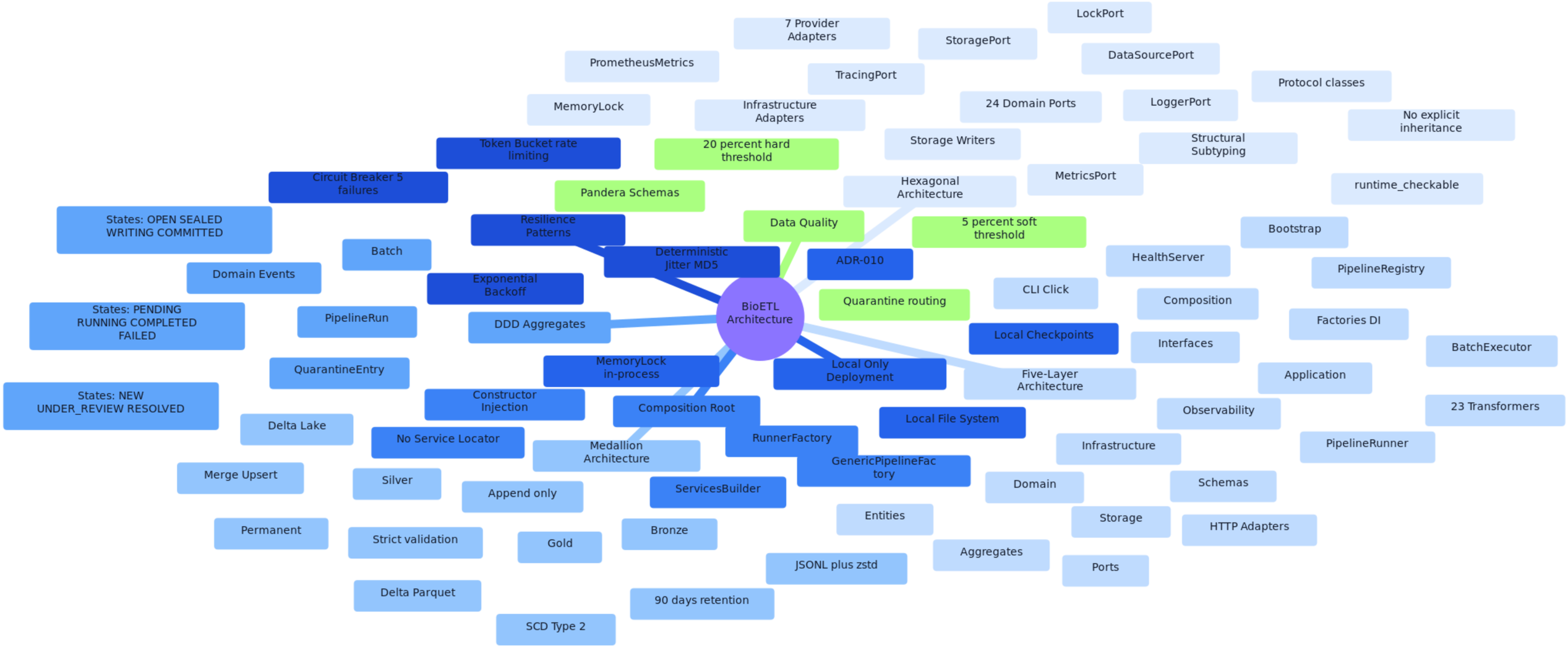

fallback
default
fallback
Domain Ports (domain / ports /)
Observability & DQ
[P] LoggerPort
[P] MetricsPort
[P] TracingPort
[P] DQMonitorPort
[P] MemoryMonitorPort
[P] AuditPort
[P] HealthCheckPort
Validation & Policy
[P] SilverValidationPort
[P] GoldValidationPort
[P] JsonEncoderPort
[P] InputMaxPort
[P] PiiHasherPort
[P] MetadataWriterPort
[P] MetadataCoordinatorPort
Runtime Controls
[P] ShutdownPort
[P] CircuitBreakerPort
[P] RateLimiterPort
Core Data & State
[P] DeltaReaderPort
[P] FilteredDataSource Port
[P] DataSourcePort
[P] StoragePort
[P] LockPort
[P] CheckpointPort
[P] QuarantinePort
Infrastructure Adapters
Observability & DQ Adapters
[A] PrometheusMetrics
[A] NoOpTracing OpenTelemetryTracer
[A] DataQualityMonitor
[A] StructlogLogger UnifiedLogger
[A] FileAuditAdapter
[A] HealthCheckMain
[A] MemoryMonitor
Runtime Control Adapters
[A] ShutdownService
[A] CircuitBreaker
[A] TokenBucket
Validation & Metadata Adapters
[A] PanderaSilverValidator
[A] PanderaGoldValidator
[A] SHA256Hasher
[A] MetadataWriter
[A] MetadataCoordinator
Core Data & Storage Adapters
[A] CsvFileReader
[A] DeltaReader
[A] FilteredDataSource CachedBronzeDataSource
[A] BronzeWriter + SilverWriter + GoldWriter
[A] MemoryLock
[A] LocalCheckpoint
[A] UnifiedQuarantine
[A] ProviderAdapters (5)
NoOp Implementations (development / noop-pyi)
NoOp Observability
[N] NoOpMemoryMonitor
[N] NoOpTracing
[N] NoOpMetrics
[N] NoOpAudit
NoOp Policy
[N] NoOpMetadataWriter
[N] NoOpPiiHasher
fallback
fallback
fallback


PipelineRun created
PENDING
runner.run()
LOCKING
LockManager.acquire()
LOCKED
PreflightService.run()
HeartbeatTask starts every 30s renews TTL (90s)
PREFLIGHT
all checks pass
PREFLIGHT_PASSED
RunType == REBUILD / BACKFILL
RunType == INCREMENTAL
CLEARING
MedallionLifecycleService\nclear_silver() + clear_gold()
LockAcquisitionError
preflight check failed
EXTRACTING
BatchExecutor._extract()
TRANSFORMING
SIGTERM received
BatchTransformer.transform()
ShutdownService detected
VALIDATING
DRAINING
CircuitBreakerOpenError\nRetryExhaustedError
CheckpointManager.save()\nmore batches
DQ error rate > threshold
finish current batch\nsave checkpoint
QUARANTINING
PanderaSilverValidator.validate()
CHECKPOINT_SAVED
QuarantineManager.route()
release lock\nexit(0)
WRITING
CANCELLED
LockLostError\n(safety guard)
BatchWriter.write()
FAILED
CHECKPOINTING
no more batches
PipelineRunState.FAILED StageResult captures error
BATCH_DONE
PostrunService.run()
error path
POSTRUN
vacuum + DQ report\n+ metrics publish
COMPLETED
success
PipelineRunState.COMPLETED PipelineObserver records duration
RELEASING
LockManager.release()\nHeartbeatTask.stop()


1. External API
REST API Response (e.g., ChEMBL / activity)
2. Bronze Layer
BronzeWriter.write_bronze()
• JSONL serialization
• zstd compression
• atomic rename
• _manifest.json
Bronze File
bronze / chembl / activity / 2026-02-17 / batch_001.jsonl.zst
3. Transform (RecordProcessor)
BatchTransformer.transform()
BaseTransformer._transform_impl()
(e.g., ActivityTransformer)
Data Normalization
• NaN → null
• strip strings
• dates → ISO
• floats → round(10)
Add Metadata
• _run_id (UUID)
• _run_type (incremental / backfill)
• _source_batch_id (FK)
• _ingestion_ts (UTC now)
Compute _content_hash
• exclude META_FIELDS
• canonical JSON (sort_keys)
• sha256 digest
4. Validate
PanderaSilverValidator.validate()
ActivitySchema (soft mode)
DQ Error Rate?
per-record error
< 5% errors
0% errors
> 20% errors
FAIL
5. Route Decision
❌ Quarantine Record
QuarantineManager.route()
QuarantineEntry created
⚠️ Warning Record
_dq_warn = true
(error rate < 5%)
✅ Clean Record
_dq_warn = false
Quarantine Storage
Quarantine Table
quarantine / chembl / activity /
6. Silver Layer
SilverWriter.write_silver()
• merge / upsert by PK
• content_hash dedup
• Delta Lake transaction
Silver Table
silver / chembl / activity / _delta_log /
7. Gold Layer
transform_for_gold()
• exclude JSON fields (molecule_properties, molecule_structures, etc.)
PanderaGoldValidator.validate()
ChEMBLActivityGoldSchema
(strict mode)
GoldWriter.write_gold()
GT


User
CLI (Click)
interfaces/cli/
PipelineRunnerService
application/services/
bootstrap_pipeline()
composition/bootstrap/
RunnerFactory
composition/factories/
ServicesBuilder
composition/factories/
PipelineRunner
application/core/
PreflightService
application/core/
LockManager
application/core/
HeartbeatTask
application/core/
BatchExecutor
application/core/
PostrunService
application/core/
PipelineObserver
application/observability/
bioetl run chembl_activity
--run-type incremental
parse args, resolve pipeline_name
PipelineRunnerService.run(name, options)
bootstrap_pipeline(name, config)
Assembly Phase
create StructlogLogger
create StorageFactory → writers
create HttpClientFactory → client
create DataSourceFactory → adapter
ServicesBuilder.build()
PipelineServices (frozen)
RunnerFactory.create_runner()
PipelineRunner (assembled)
runner
Execution Phase
PipelineObserver.__enter__()
log "pipeline started"
increment bioetl_pipeline_runs_total
runner.run()
acquire(key, owner_id, ttl=90s)
lock acquired
HeartbeatTask.start(interval=30s)
PreflightService.run()
check config, storage, health
PreflightReport (all passed)
loop
[For each batch]
BatchExecutor.execute_batch()
fetch → transform → validate → write
BatchResult(records, errors)
PostrunService.run()
vacuum, DQ report, metrics
HeartbeatTask.stop()
release(key, owner_id)
success
PipelineObserver.__exit__()
observe duration
log "pipeline completed"
RunResult(status=SUCCESS)
ExitCode.SUCCESS (0)


PipelineRunner
BatchExecutor (786 LOC)
DataSourcePort (e.g., ChemblAdapter)
BatchTransformer
BaseTransformer (e.g., ActivityTransformer)
BatchWriter
BronzeWriter
SilverWriter
GoldWriter
QuarantineManager
BatchMetricsRecorder
BatchTracingManager
CheckpointManager
execute_batch(batch_num)
start_span("batch")
Extract Phase
async for page in fetch(entity_type, limit=1000, filter_ids=...)
dict records (one API page)
Transform Phase
transform(raw_records)
_transform_impl(records)
transformed records
normalize values (NaN→null, strip, dates→ISO)
add metadata (_run_id, _run_type, _ingestion_ts)
compute _content_hash (sha256)
TransformResult(records, errors)
Write Phase
write(batch_data)
write_bronze(records) → JSONL + zstd
write_silver(df, primary_keys) → Delta Lake merge/upsert
write_gold(df) → transform_for_gold() + strict validate
alt
[DQ errors detected]
route_failed_records(failures)
create QuarantineEntry truncate payload (64KB)
write complete
Metrics Phase
record(batch_size, duration, errors)
observe_histogram(batch_size) increment_counter(records_total) increment_counter(errors_total)
end_span(status)
BatchResult(record_count, error_count)
save(last_id, batch_num)
write JSON checkpoint (atomic file write)


BioETL Architecture
Resilience Patterns
Token Bucket rate limiting
Circuit Breaker 5 failures
Exponential Backoff
Deterministic Jitter MD5
20 percent hard threshold
Pandera Schemas
Data Quality
Batch
Domain Events
States: OPEN SEALED WRITING COMMITTED
States: PENDING RUNNING COMPLETED FAILED
States: NEW UNDER_REVIEW RESOLVED
PipelineRun
DDD Aggregates
QuarantineEntry
MemoryLock in-process
Constructor Injection
Composition Root
No Service Locator
Medallion Architecture
Delta Lake
Merge Upsert
Silver
Append only
Permanent
Strict validation
Gold
Bronze
Delta Parquet
90 days retention
SCD Type 2
ServicesBuilder
RunnerFactory
GenericPipelineFactory
Local Only Deployment
Local File System
Local Checkpoints
Quarantine routing
ADR-010
Hexagonal Architecture
5 percent soft threshold
Five-Layer Architecture
7 Provider Adapters
PrometheusMetrics
TracingPort
MemoryLock
Infrastructure Adapters
Storage Writers
24 Domain Ports
StoragePort
LockPort
DataSourcePort
LoggerPort
Protocol classes
No explicit inheritance
Structural Subtyping
MetricsPort
runtime_checkable
Bootstrap
HealthServer
PipelineRegistry
CLI Click
Composition
Factories DI
Interfaces
BatchExecutor
Application
Observability
23 Transformers
Infrastructure
PipelineRunner
Domain
Schemas
Storage
HTTP Adapters
Entities
Aggregates
Ports
JSONL plus zstd

Interfaces Layer
(Click)

$ bioetl run chembl_activity\n--run-type incremental\n--batch-size 1000

Parse CLI Arguments
• pipeline_name
• run_type (RunType enum)
• batch_size,
resume flag

Build RunOptions
• pipeline_name
• run_type
• batch_size
• resume: bool

Application
Service Layer

PipelineRunnerService.run()
• lookup pipeline in registry
• validate run options

PipelineRegistry.get(name)
→ PipelineFactoryPort

Composition Layer

bootstrap_pipeline()
→ assembly.py

Full Assembly Sequence
1. Logger
2. Config
3. Storage
4. HTTP Client
5. Adapter
6. Services
7. Runner

Application Core

PipelineObserver.enter()
→ log started, inc counter

PipelineRunner.run()

LockManager.acquire()
→ HeartbeatTask.start()

PreflightService.run()
→ PreflightReport

pass

BatchExecutor loop
→ extract / transform / validate / write

PostrunService.run()
→ vacuum, DQ report

LockManager.release()
→ HeartbeatTask.stop()

PipelineObserver.exit()
→ observe duration

LockLostError

success

error

fail

LockAcquisitionError

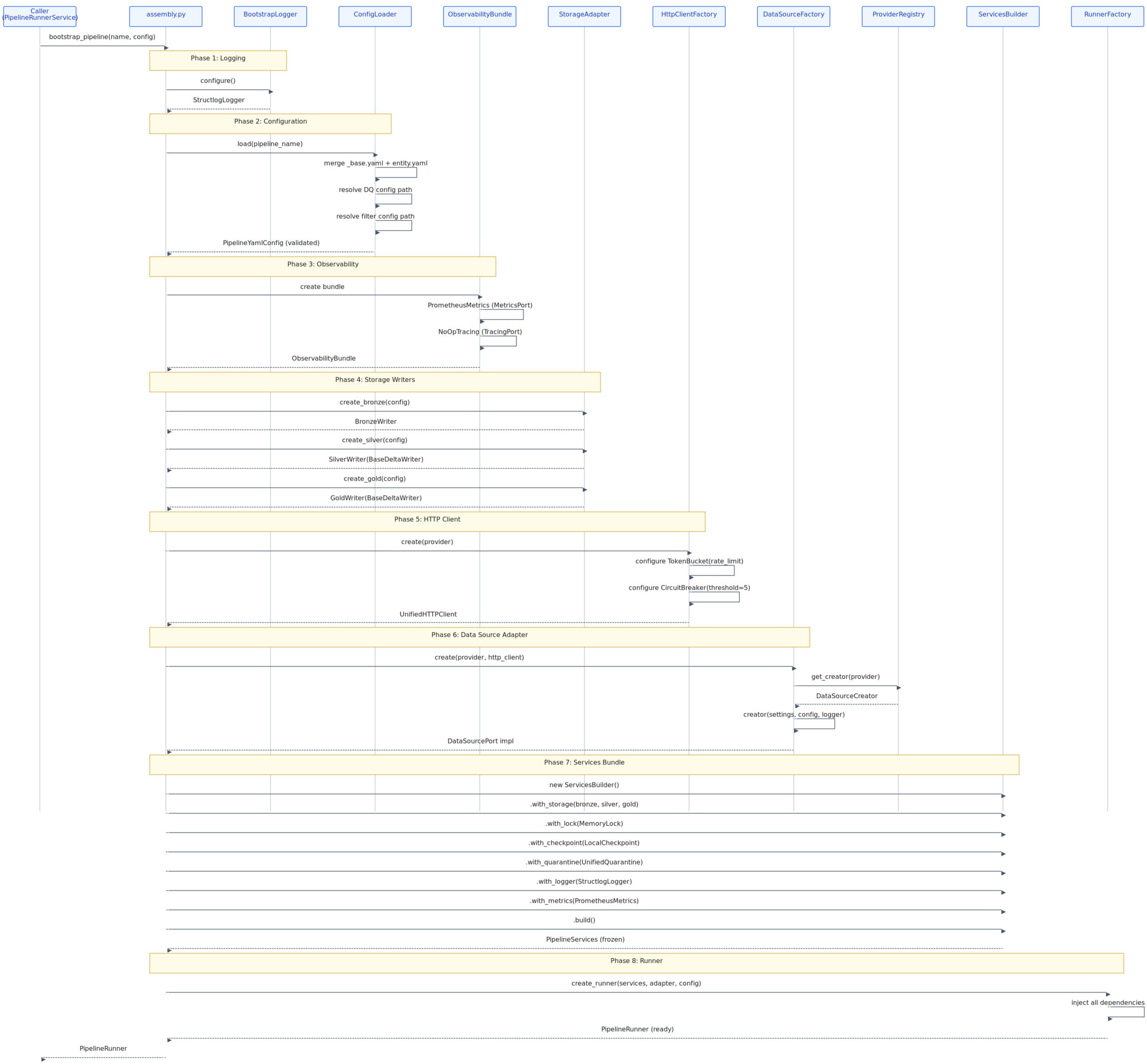
Result & Exit

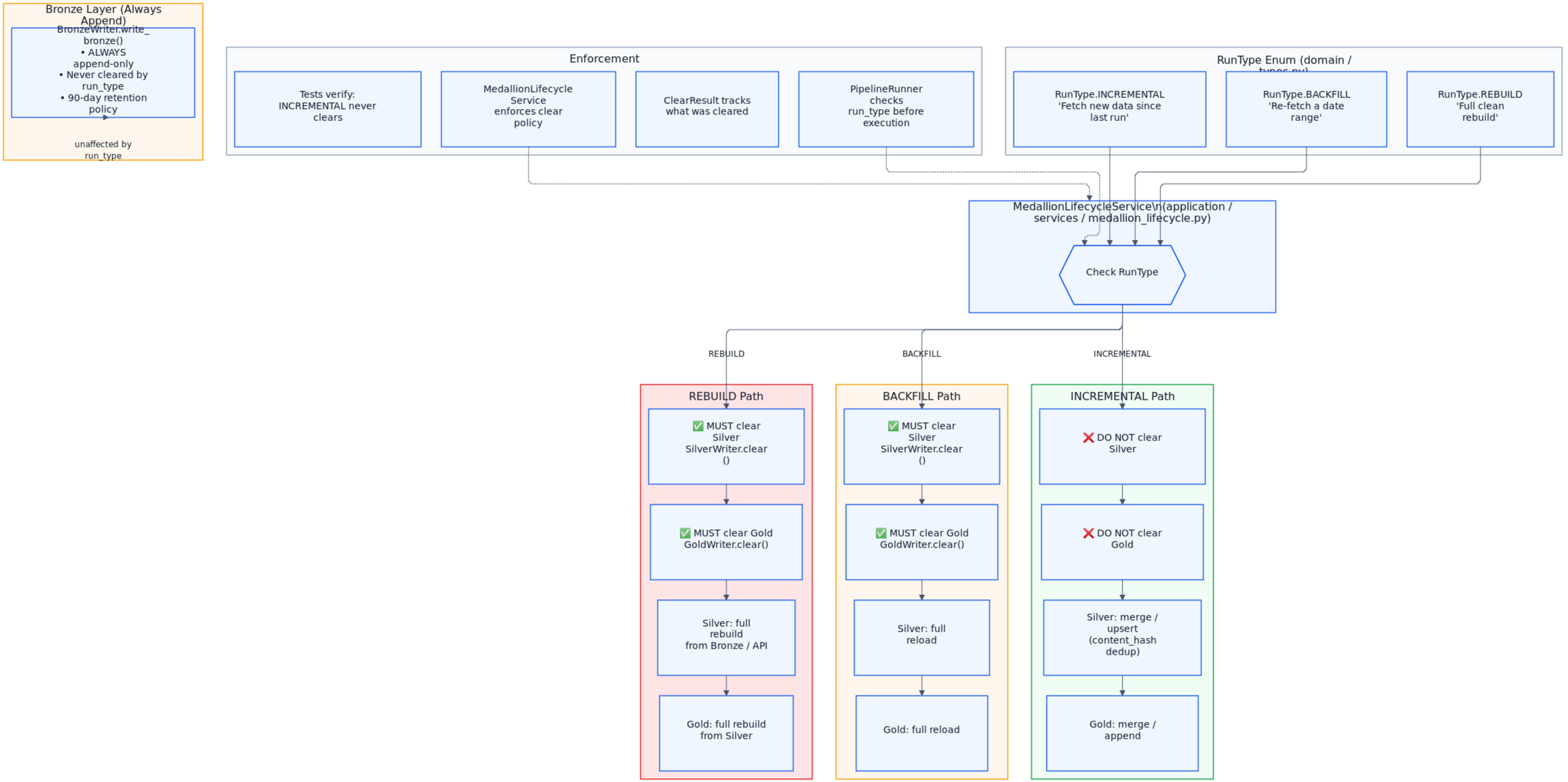
RunResult(status=SUCCESS)
PipelineRunResult.SUCCESS

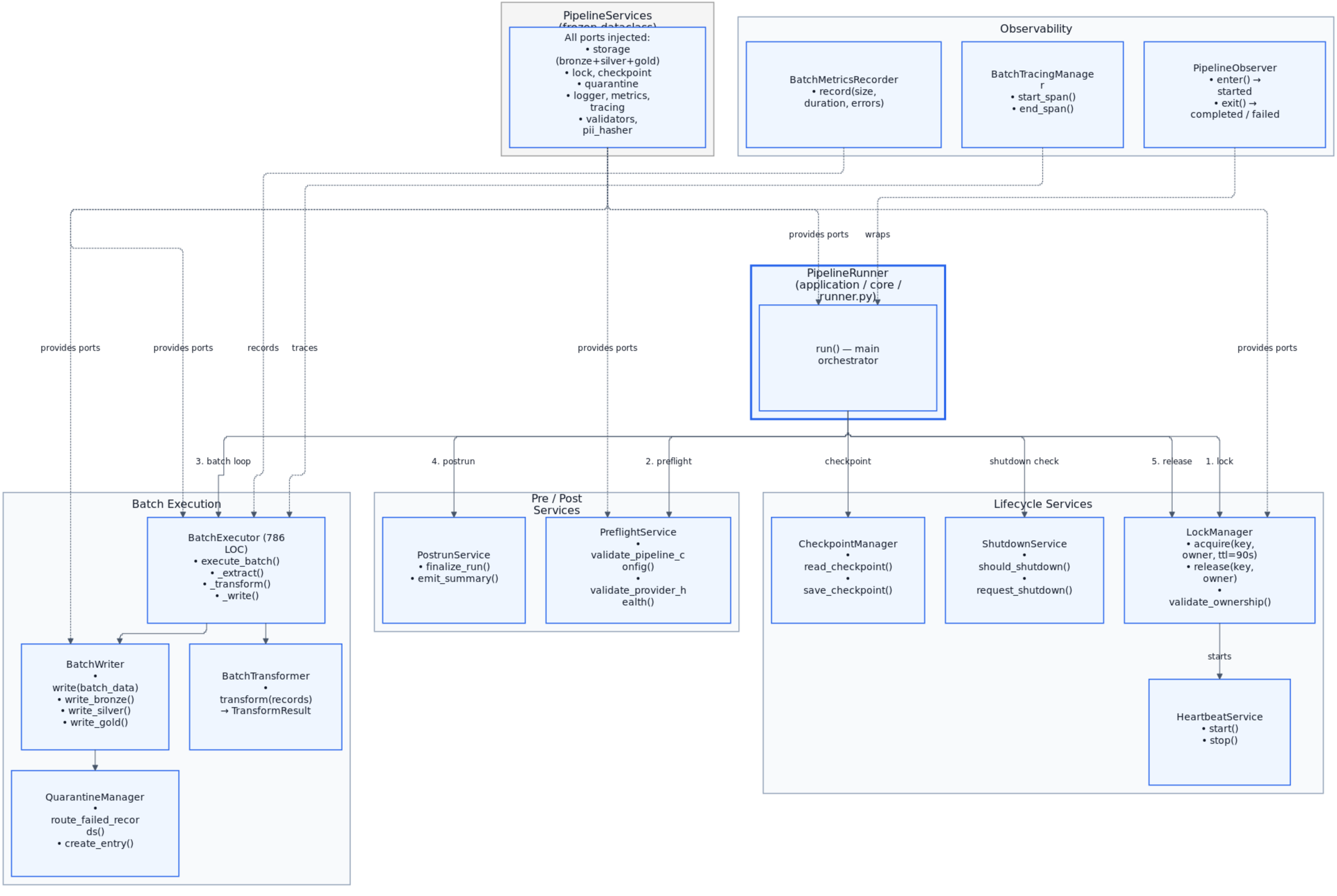
RunResult(status=FAILED)
PipelineRunResult.FAILED

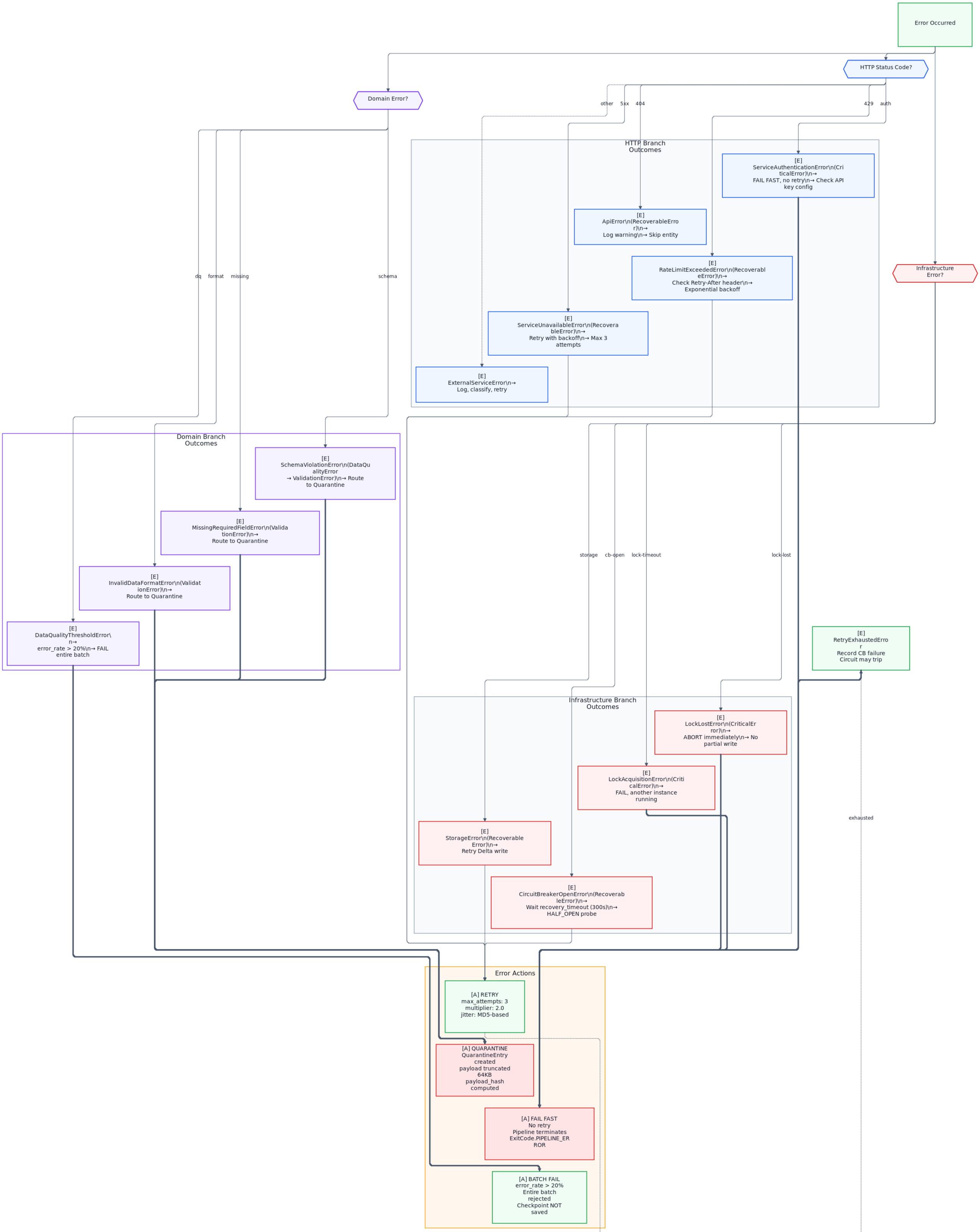
ExitCode.SUCCESS
(0)

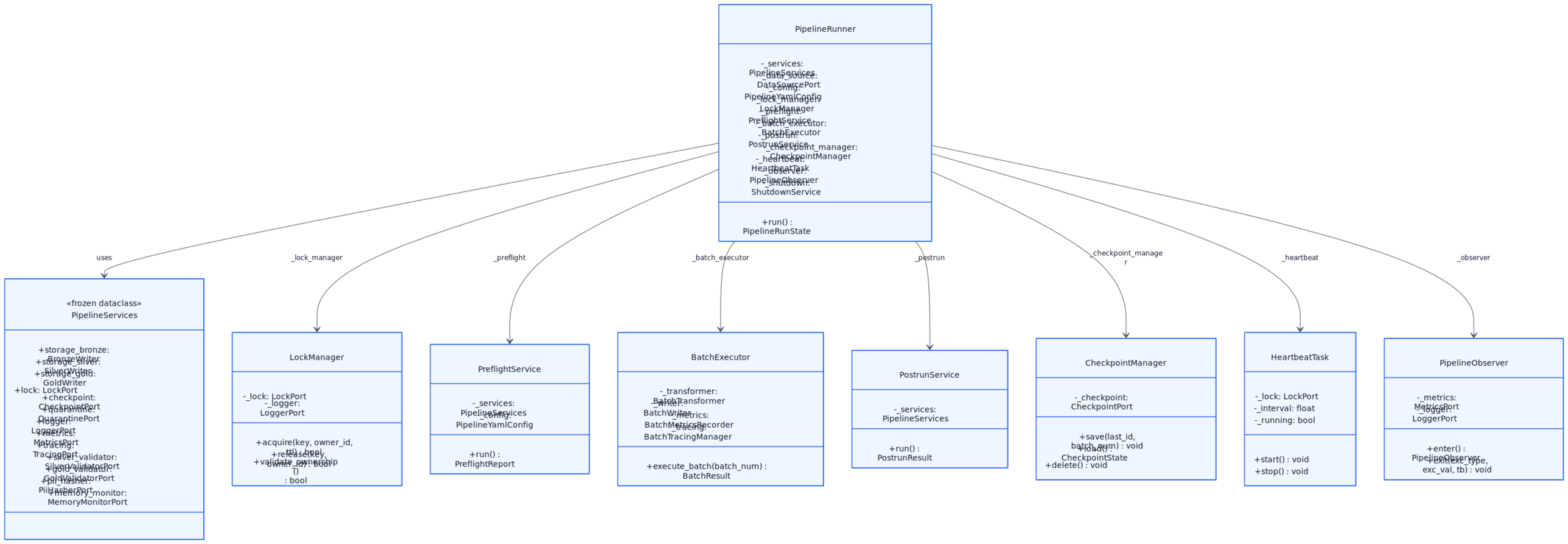
ExitCode.PIPELINE_ERROR
(1)
ExitCode.CONFIG_ERROR
(2)


Caller (PipelineRunnerService)
assembly.py
BootstrapLogger
ConfigLoader
ObservabilityBundle
StorageAdapter
HttpClientFactory
DataSourceFactory
ProviderRegistry
ServicesBuilder
RunnerFactory
bootstrap_pipeline(name, config)
Phase 1: Logging
configure()
StructlogLogger
Phase 2: Configuration
load(pipeline_name)
merge _base.yaml + entity.yaml
resolve DQ config path
resolve filter config path
PipelineYamlConfig (validated)
Phase 3: Observability
create bundle
PrometheusMetrics (MetricsPort)
NoOpTracing (TracingPort)
ObservabilityBundle
Phase 4: Storage Writers
create_bronze(config)
BronzeWriter
create_silver(config)
SilverWriter(BaseDeltaWriter)
create_gold(config)
GoldWriter(BaseDeltaWriter)
Phase 5: HTTP Client
create(provider)
configure TokenBucket(rate_limit)
configure CircuitBreaker(threshold=5)
UnifiedHTTPClient
Phase 6: Data Source Adapter
create(provider, http_client)
get_creator(provider)
DataSourceCreator
creator(settings, config, logger)
DataSourcePort impl
Phase 7: Services Bundle
new ServicesBuilder()
.with_storage(bronze, silver, gold)
.with_lock(MemoryLock)
.with_checkpoint(LocalCheckpoint)
.with_quarantine(UnifiedQuarantine)
.with_logger(StructlogLogger)
.with_metrics(PrometheusMetrics)
.build()
PipelineServices (frozen)
Phase 8: Runner
create_runner(services, adapter, config)
inject all dependencies
PipelineRunner (ready)
PipelineRunner


Bronze Layer (Always Append)
BronzeWriter.write_bronze()
• ALWAYS append-only
• Never cleared by run_type
• 90-day retention policy
unaffected by run_type
Enforcement
Tests verify: INCREMENTAL never clears
MedallionLifecycle Service enforces clear policy
ClearResult tracks what was cleared
PipelineRunner checks run_type before execution
RunType Enum (domain / types.py)
RunType.INCREMENTAL 'Fetch new data since last run'
RunType.BACKFILL 'Re-fetch a date range'
RunType.REBUILD 'Full clean rebuild'
MedallionLifecycleService\n(application / services / medallion_lifecycle.py)
Check RunType
REBUILD
BACKFILL
INCREMENTAL
REBUILD Path
✅ MUST clear Silver SilverWriter.clear()
✅ MUST clear Gold GoldWriter.clear()
Silver: full rebuild from Bronze / API
Gold: full rebuild from Silver
BACKFILL Path
✅ MUST clear Silver SilverWriter.clear()
✅ MUST clear Gold GoldWriter.clear()
Silver: full reload
Gold: full reload
INCREMENTAL Path
❌ DO NOT clear Silver
❌ DO NOT clear Gold
Silver: merge / upsert (content_hash dedup)
Gold: merge / append


PipelineServices
(frozen dataclass)
All ports injected:
• storage
(bronze+silver+gold)
• lock, checkpoint
• quarantine
• logger, metrics,
tracing
• validators,
pii_hasher
Observability
BatchMetricsRecorder
• record(size,
duration, errors)
BatchTracingManage
r
• start_span()
• end_span()
PipelineObserver
• enter() →
started
• exit() →
completed / failed
provides ports
wraps
PipelineRunner
(application / core /
runner.py)
run() — main
orchestrator
provides ports
provides ports
records
traces
provides ports
provides ports
3. batch loop
4. postrun
2. preflight
checkpoint
shutdown check
5. release
1. lock
Batch Execution
BatchExecutor (786
LOC)
• execute_batch()
• _extract()
• _transform()
• _write()
BatchWriter
•
write(batch_data)
• write_bronze()
• write_silver()
• write_gold()
BatchTransformer
•
transform(records)
→ TransformResult
QuarantineManager
•
route_failed_recor
ds()
• create_entry()
Pre / Post
Services
PostrunService
• finalize_run()
• emit_summary()
PreflightService
•
validate_pipeline_c
onfig()
•
validate_provider_h
ealth()
Lifecycle Services
CheckpointManager
•
read_checkpoint()
•
save_checkpoint()
ShutdownService
•
should_shutdown()
•
request_shutdown()
LockManager
• acquire(key,
owner, ttl=90s)
• release(key,
owner)
•
validate_ownership()
starts
HeartbeatService
• start()
• stop()


Error Occurred
Domain Error?
HTTP Status Code?
Infrastructure Error?
dq
format
missing
schema
other
5xx
404
429
auth
storage
cb-open
lock-timeout
lock-lost
exhausted
Domain Branch Outcomes
[E] SchemaViolationError\n(DataQualityError → ValidationError)\n→ Route to Quarantine
[E] MissingRequiredFieldError\n(ValidationError)\n→ Route to Quarantine
[E] InvalidDataFormatError\n(ValidationError)\n→ Route to Quarantine
[E] DataQualityThresholdError\n→ error_rate > 20%\n→ FAIL entire batch
HTTP Branch Outcomes
[E] ServiceAuthenticationError\n(CriticalError)\n→ FAIL FAST, no retry\n→ Check API key config
[E] ApiError\n(RecoverableError)\n→ Log warning\n→ Skip entity
[E] RateLimitExceededError\n(RecoverableError)\n→ Check Retry-After header\n→ Exponential backoff
[E] ServiceUnavailableError\n(RecoverableError)\n→ Retry with backoff\n→ Max 3 attempts
[E] ExternalServiceError\n→ Log, classify, retry
Infrastructure Branch Outcomes
[E] LockLostError\n(CriticalError)\n→ ABORT immediately\n→ No partial write
[E] LockAcquisitionError\n(CriticalError)\n→ FAIL, another instance running
[E] StorageError\n(RecoverableError)\n→ Retry Delta write
[E] CircuitBreakerOpenError\n(RecoverableError)\n→ Wait recovery_timeout (300s)\n→ HALF_OPEN probe
[E] RetryExhaustedError\n→ Record CB failure\n→ Circuit may trip
Error Actions
[A] RETRY max_attempts: 3 multiplier: 2.0 jitter: MD5-based
[A] QUARANTINE QuarantineEntry created payload truncated 64KB payload_hash computed
[A] FAIL FAST No retry Pipeline terminates ExitCode.PIPELINE_ERROR
[A] BATCH FAIL error_rate > 20% Entire batch rejected Checkpoint NOT saved


PipelineRunner
-_services: PipelineServices
-_data_source: DataSourcePort
-_config: PipelineYamlConfig
-_lock_manager: LockManager
-_preflight: PreflightService
-_batch_executor: BatchExecutor
-_postrun: PostrunService
-_checkpoint_manager: CheckpointManager
-_heartbeat: HeartbeatTask
-_observer: PipelineObserver
-_shutdown: ShutdownService
+run() : PipelineRunState
uses
_lock_manager
_preflight
_batch_executor
_postrun
_checkpoint_manager
_heartbeat
_observer
«frozen dataclass»
PipelineServices
+storage_bronze: BronzeWriter
+storage_silver: SilverWriter
+storage_gold: GoldWriter
+lock: LockPort
+checkpoint: CheckpointPort
+quarantine: QuarantinePort
+logger: LoggerPort
+metrics: MetricsPort
+tracing: TracingPort
+silver_validator: SilverValidatorPort
+gold_validator: GoldValidatorPort
+pii_hasher: PiiHasherPort
+memory_monitor: MemoryMonitorPort
LockManager
-_lock: LockPort
-_logger: LoggerPort
+acquire(key, owner_id, ttl) : bool
+release(key, owner_id) : bool
+validate_ownership() : bool
PreflightService
-_services: PipelineServices
-_config: PipelineYamlConfig
+run() : PreflightReport
BatchExecutor
-_transformer: BatchTransformer
-_writer: BatchWriter
-_metrics: BatchMetricsRecorder
-_tracing: BatchTracingManager
+execute_batch(batch_num) : BatchResult
PostrunService
-_services: PipelineServices
+run() : PostrunResult
CheckpointManager
-_checkpoint: CheckpointPort
+save(last_id, batch_num) : void
+load() : CheckpointState
+delete() : void
HeartbeatTask
-_lock: LockPort
-_interval: float
-_running: bool
+start() : void
+stop() : void
PipelineObserver
-_metrics: MetricsPort
-_logger: LoggerPort
+enter() : PipelineObserver
+exit(exc_type, exc_val, tb) : void

Seed Pipeline Result

Seed Silver Table (e.g., chembl / publication)

seed data

Key Extraction

KeyExtractorService.extract_keys()
• read seed Silver via DeltaReader
• select join_key columns (doi, pmid)
• deduplicate → unique key list

PMID Keys (~30,000 unique)

DOI Keys (~50,000 unique)

Fan-Out (EnrichmentCoordinator)

EnrichmentCoordinator.run_enrichers()
• asyncio.gather(*tasks, return_exceptions=True)
• timeout per enricher

timeout / fail\n(required=false)

parallel

parallel

parallel

timeout / fail\n(required=false)

parallel

SemanticScholar Enricher
filter_ids = DOIs
required = false
timeout = 1800s

CrossRef Enricher
filter_ids = DOIs
required = true
timeout = 3600s

PubMed Enricher
filter_ids = PMIDs
required = true
timeout = 3600s

OpenAlex Enricher
filter_ids = DOIs
required = false
timeout = 1800s

Enricher Silver Tables

semanticscholar / publication /

crossref / publication /

pubmed / publication /

openalex / publication /

Fan-In (MergeService)

EnricherDeduplicator
• remove duplicates across enrichers

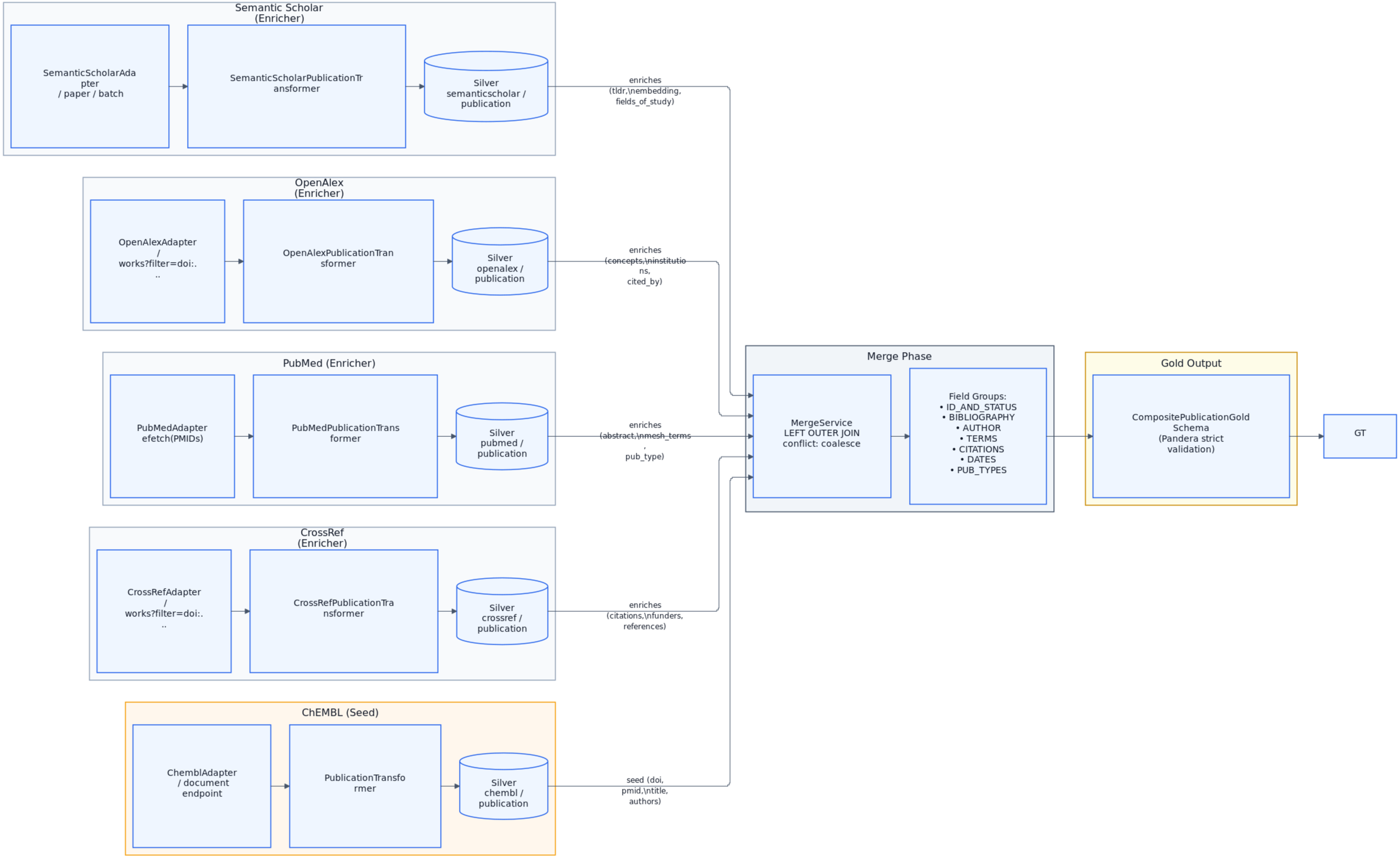
MergeService.merge()
• LEFT OUTER JOIN seed ⟕ enrichers
• ConflictResolution (coalesce: first non-null)

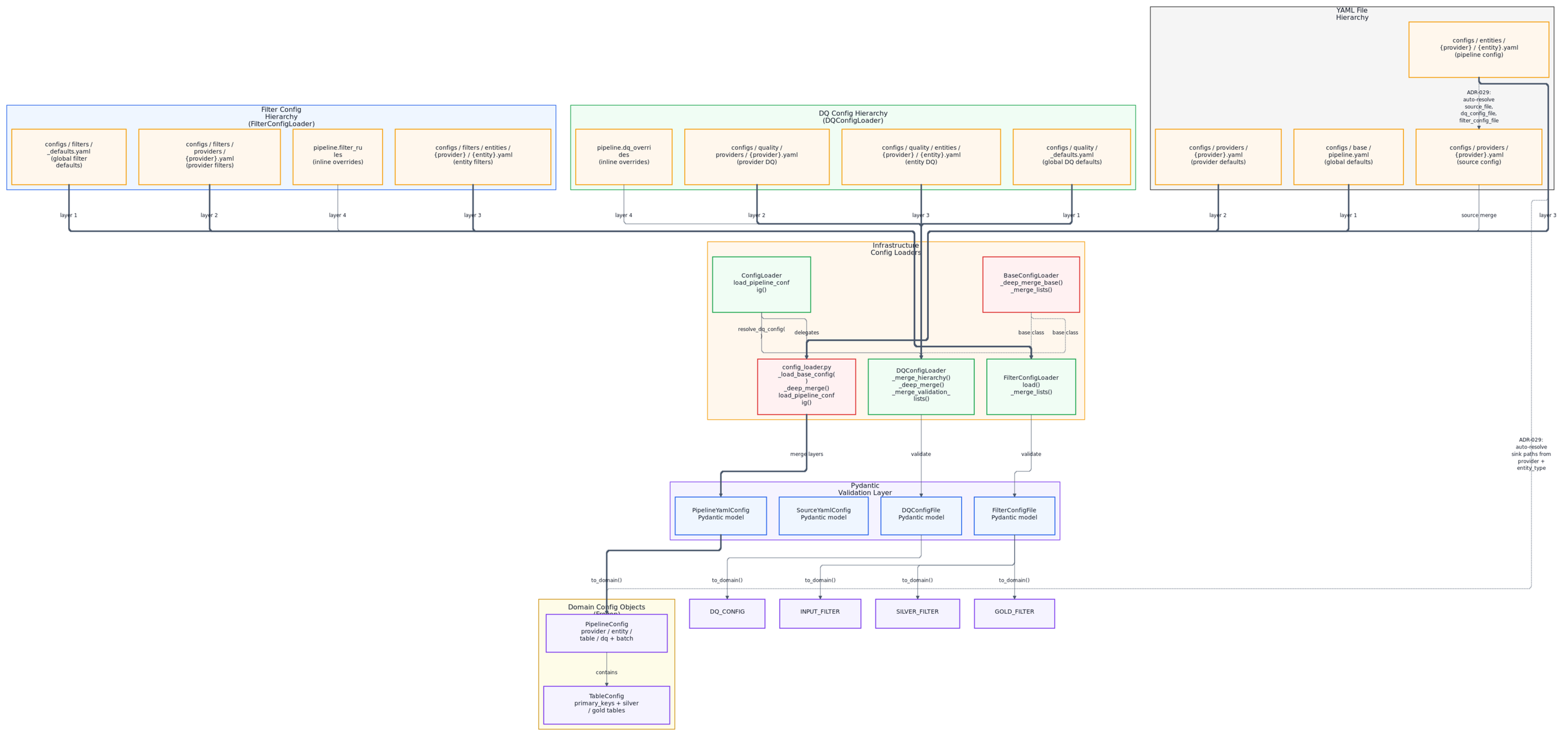
ColumnOrderer
• FieldGroup priority
• exclude TRASH group

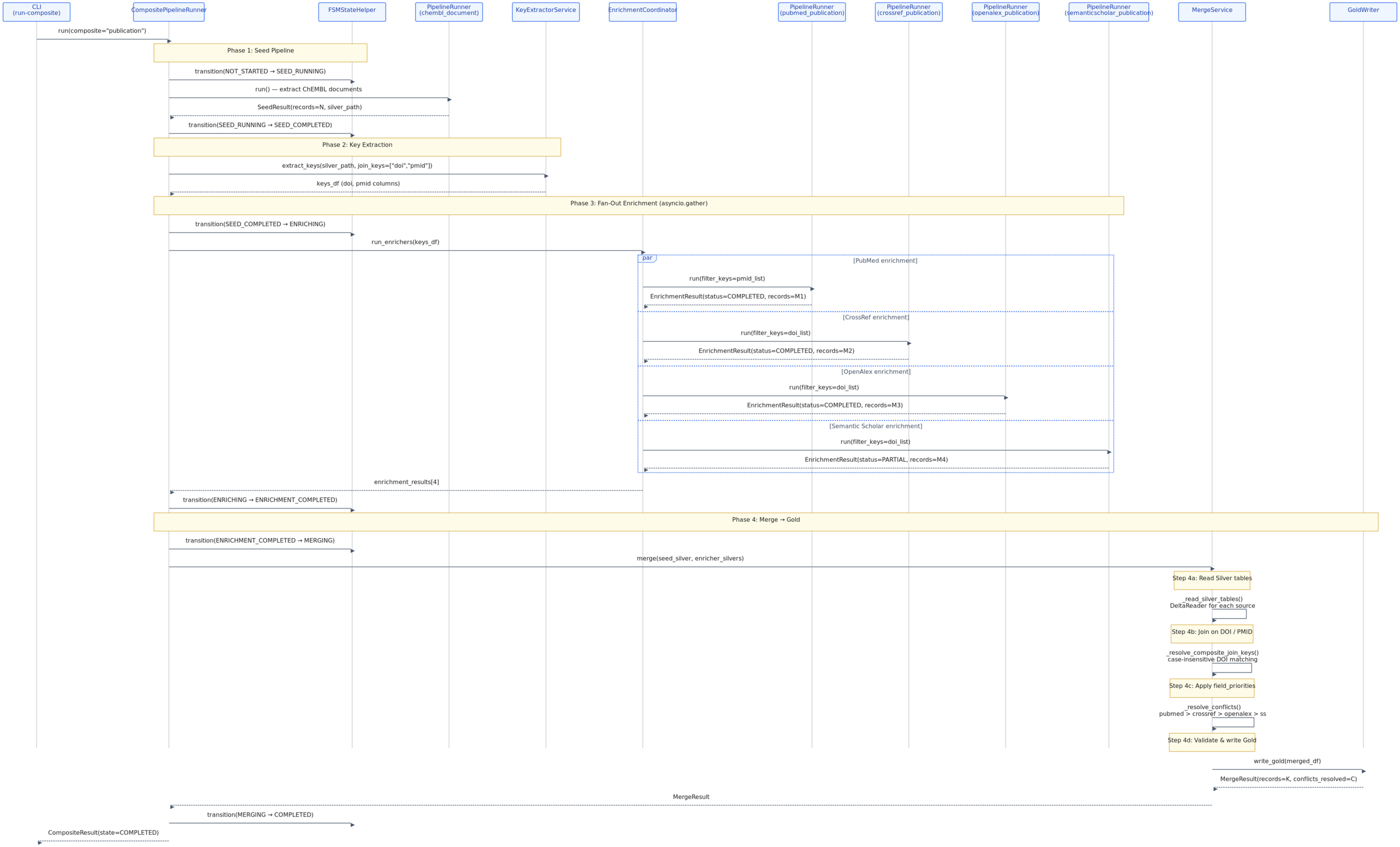
Gold Output

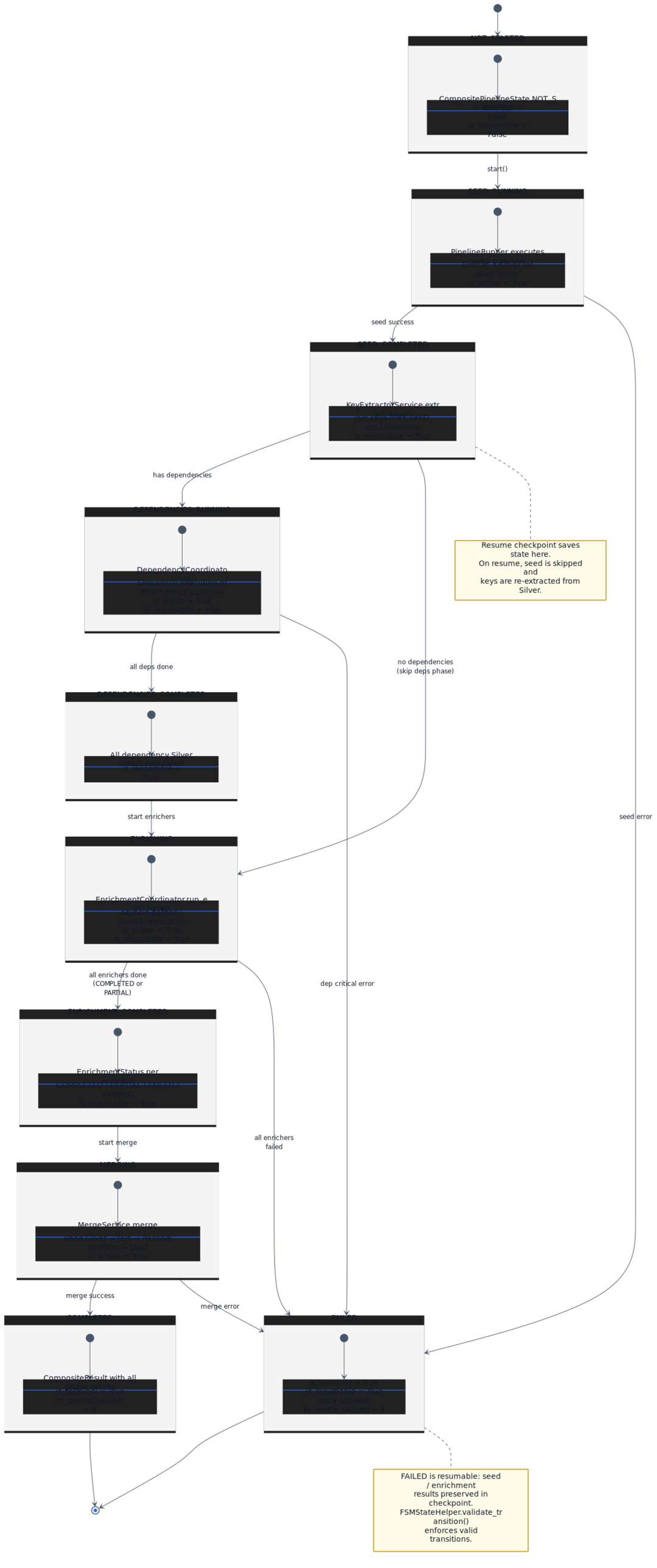
GoldWriter.write_gold()
CompositePublicationGold Schema

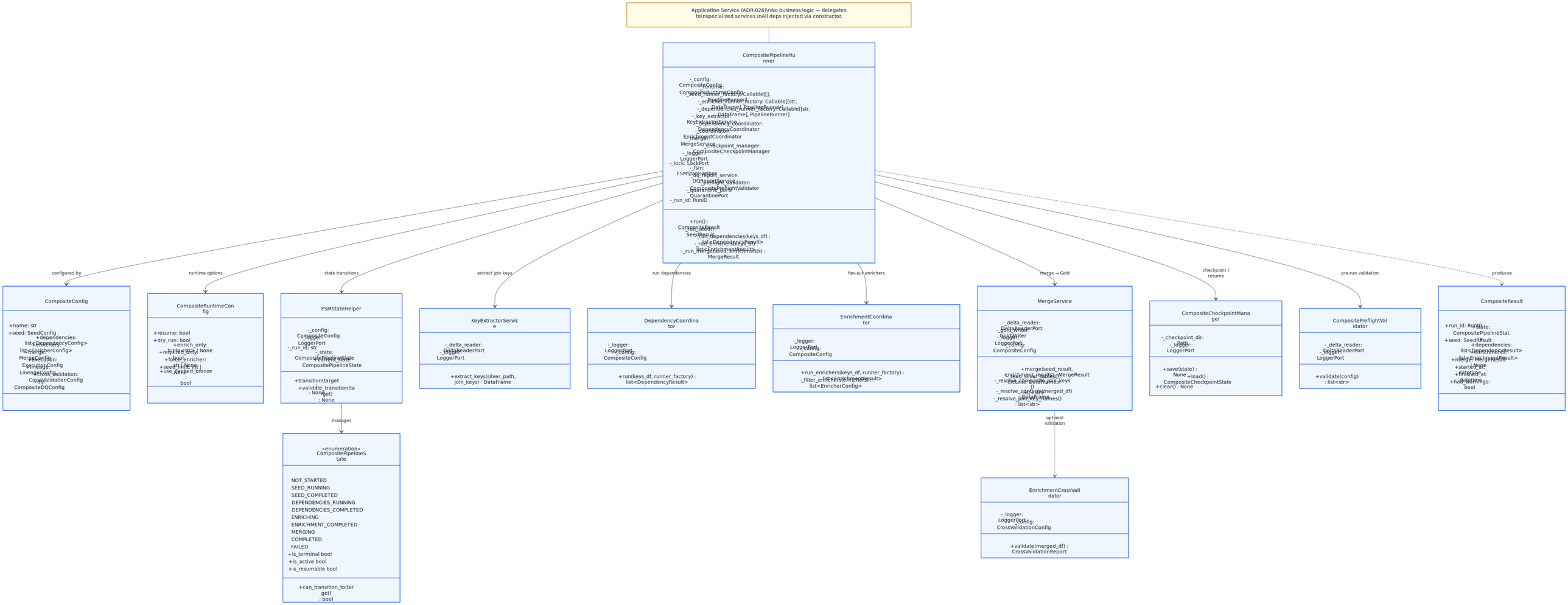
gold / composite / publication /

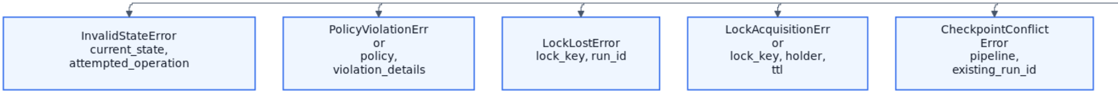

Semantic Scholar (Enricher)
SemanticScholarAdapter / paper / batch
SemanticScholarPublicationTransformer
Silver semanticscholar / publication
enriches (tldr,\nembedding, fields_of_study)
OpenAlex (Enricher)
OpenAlexAdapter / works?filter=doi:...
OpenAlexPublicationTransformer
Silver openalex / publication
enriches (concepts,\ninstitutions, cited_by)
PubMed (Enricher)
PubMedAdapter efetch(PMIDs)
PubMedPublicationTransformer
Silver pubmed / publication
enriches (abstract,\nmesh_terms, pub_type)
CrossRef (Enricher)
CrossRefAdapter / works?filter=doi:...
CrossRefPublicationTransformer
Silver crossref / publication
enriches (citations,\nfunders, references)
ChEMBL (Seed)
ChemblAdapter / document endpoint
PublicationTransformer
Silver chembl / publication
seed (doi, pmid,\ntitle, authors)
Merge Phase
MergeService LEFT OUTER JOIN conflict: coalesce
Field Groups:
• ID_AND_STATUS
• BIBLIOGRAPHY
• AUTHOR
• TERMS
• CITATIONS
• DATES
• PUB_TYPES
Gold Output
CompositePublicationGold Schema (Pandera strict validation)
GT

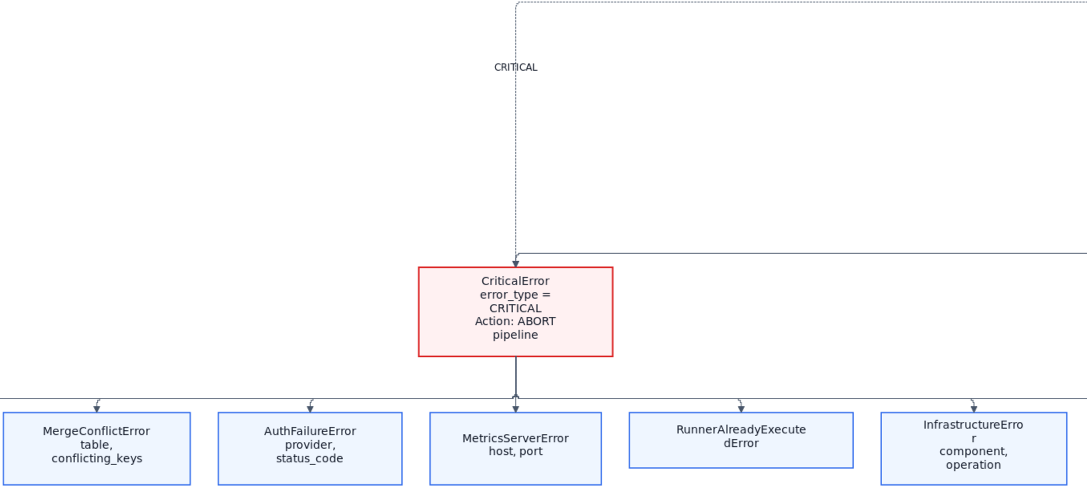

YAML File Hierarchy
configs / entities / {provider} / {entity}.yaml (pipeline config)
ADR-029: auto-resolve source_file, dq_config_file, filter_config_file
Filter Config Hierarchy (FilterConfigLoader)
configs / filters / _defaults.yaml (global filter defaults)
configs / filters / providers / {provider}.yaml (provider filters)
pipeline.filter_rules (inline overrides)
configs / filters / entities / {provider} / {entity}.yaml (entity filters)
DQ Config Hierarchy (DQConfigLoader)
pipeline.dq_overrides (inline overrides)
configs / quality / providers / {provider}.yaml (provider DQ)
configs / quality / entities / {provider} / {entity}.yaml (entity DQ)
configs / quality / _defaults.yaml (global DQ defaults)
configs / providers / {provider}.yaml (provider defaults)
configs / base / pipeline.yaml (global defaults)
configs / providers / {provider}.yaml (source config)
layer 1
layer 2
layer 4
layer 3
layer 4
layer 2
layer 3
layer 1
layer 2
layer 1
source merge
layer 3
Infrastructure Config Loaders
ConfigLoader load_pipeline_config()
BaseConfigLoader _deep_merge_base() _merge_lists()
resolve_dq_config()
delegates
base class
base class
config_loader.py _load_base_config() _deep_merge() load_pipeline_config()
DQConfigLoader _merge_hierarchy() _deep_merge() _merge_validation_lists()
FilterConfigLoader load() _merge_lists()
merge layers
validate
validate
ADR-029: auto-resolve sink paths from provider + entity_type
Pydantic Validation Layer
PipelineYamlConfig Pydantic model
SourceYamlConfig Pydantic model
DQConfigFile Pydantic model
FilterConfigFile Pydantic model
to_domain()
to_domain()
to_domain()
to_domain()
to_domain()
Domain Config Objects (Frozen)
PipelineConfig provider / entity / table / dq + batch
contains
TableConfig primary_keys + silver / gold tables
DQ_CONFIG
INPUT_FILTER
SILVER_FILTER
GOLD_FILTER

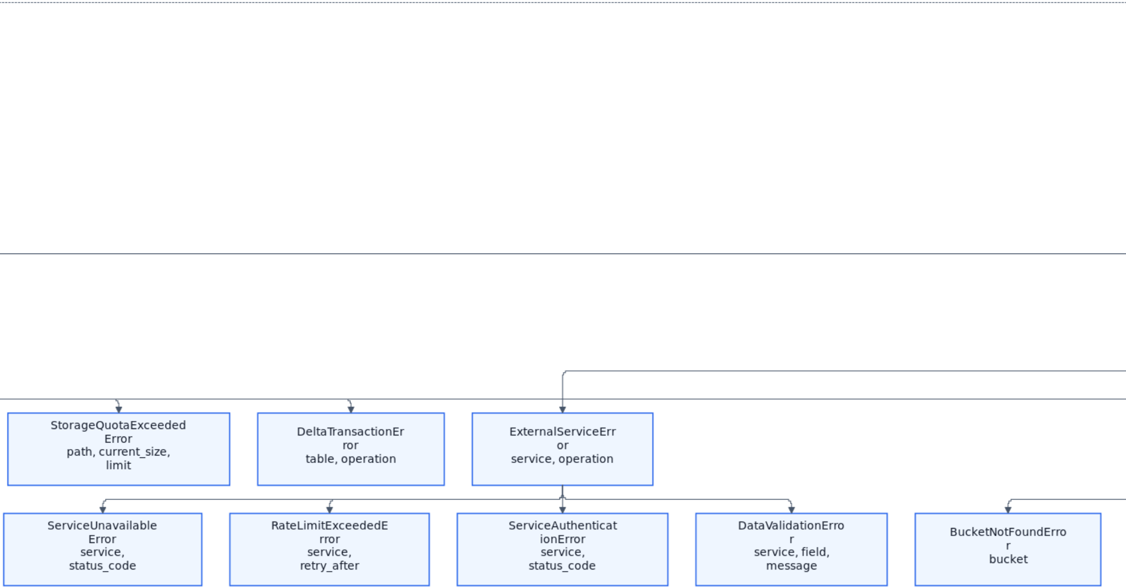

CLI (run-composite)
CompositePipelineRunner
FSMStateHelper
PipelineRunner (chembl_document)
KeyExtractorService
EnrichmentCoordinator
PipelineRunner (pubmed_publication)
PipelineRunner (crossref_publication)
PipelineRunner (openalex_publication)
PipelineRunner (semanticscholar_publication)
MergeService
GoldWriter
run(composite="publication")
Phase 1: Seed Pipeline
transition(NOT_STARTED → SEED_RUNNING)
run() — extract ChEMBL documents
SeedResult(records=N, silver_path)
transition(SEED_RUNNING → SEED_COMPLETED)
Phase 2: Key Extraction
extract_keys(silver_path, join_keys=["doi","pmid"])
keys_df (doi, pmid columns)
Phase 3: Fan-Out Enrichment (asyncio.gather)
transition(SEED_COMPLETED → ENRICHING)
run_enrichers(keys_df)
par
[PubMed enrichment]
run(filter_keys=pmid_list)
EnrichmentResult(status=COMPLETED, records=M1)
[CrossRef enrichment]
run(filter_keys=doi_list)
EnrichmentResult(status=COMPLETED, records=M2)
[OpenAlex enrichment]
run(filter_keys=doi_list)
EnrichmentResult(status=COMPLETED, records=M3)
[Semantic Scholar enrichment]
run(filter_keys=doi_list)
EnrichmentResult(status=PARTIAL, records=M4)
enrichment_results[4]
transition(ENRICHING → ENRICHMENT_COMPLETED)
Phase 4: Merge → Gold
transition(ENRICHMENT_COMPLETED → MERGING)
merge(seed_silver, enricher_silvers)
Step 4a: Read Silver tables
_read_silver_tables()
DeltaReader for each source
Step 4b: Join on DOI / PMID
_resolve_composite_join_keys()
case-insensitive DOI matching
Step 4c: Apply field_priorities
_resolve_conflicts()
pubmed > crossref > openalex > ss
Step 4d: Validate & write Gold
write_gold(merged_df)
MergeResult(records=K, conflicts_resolved=C)
MergeResult
transition(MERGING → COMPLETED)
CompositeResult(state=COMPLETED)


start()
seed success
seed error
has dependencies
no dependencies
(skip deps phase)
all deps done
start enrichers
all enrichers done
(COMPLETED or
PARTIAL)
dep critical error
all enrichers
failed
start merge
merge success
merge error
Resume checkpoint saves
state here.
On resume, seed is skipped
and
keys are re-extracted from
Silver.
FAILED is resumable: seed
/ enrichment
results preserved in
checkpoint.
FSMStateHelper.validate_tr
ansition()
enforces valid
transitions.


Application Service (ADR-026)\nNo business logic — delegates to\nspecialized services.\nAll deps injected via constructor.
CompositePipelineRunner
CompositeConfig
+name: str
+seed: SeedConfig
+dependencies: list<DependencyConfig>
+enrichers: list<EnricherConfig>
+merge: MergeConfig
+validation: ValidationConfig
+lineage: LineageConfig
+dq: CompositeDQConfig
configured by
CompositeRuntimeConfig
+resume: bool
+dry_run: bool
+enrich_only: bool
+force_enricher: str | None
+seed_timeout: int | None
+use_cached_bronze: bool
runtime options
FSMStateHelper
-_config: CompositeConfig
-_logger: LoggerPort
-_run_id: str
-_state: CompositePipelineState
+transition(target: CompositePipelineState) : None
+validate_transition(target) : None
state transitions
«enumeration»
CompositePipelineState
NOT_STARTED
SEED_RUNNING
SEED_COMPLETED
DEPENDENCIES_RUNNING
DEPENDENCIES_COMPLETED
ENRICHING
ENRICHMENT_COMPLETED
MERGING
COMPLETED
FAILED
+is_terminal bool
+is_active bool
+is_resumable bool
+can_transition_to(target) : bool
manages
KeyExtractorService
-_delta_reader: DataReaderPort
-_logger: LoggerPort
+extract_keys(silver_path, join_keys) : DataFrame
extract join keys
DependencyCoordinator
-_logger: LoggerPort
-_config: CompositeConfig
+run(keys_df, runner_factory) : list<DependencyResult>
run dependencies
EnrichmentCoordinator
-_logger: LoggerPort
-_config: CompositeConfig
+run_enrichers(keys_df, runner_factory) : list<EnrichmentResult>
-_filter_enrichers(force) : list<EnricherConfig>
fan-out enrichers
MergeService
-_delta_reader: DataReaderPort
-_gold_writer: GoldWriter
-_logger: LoggerPort
-_config: CompositeConfig
+merge(seed_result, enrichment_results) : MergeResult
-_resolve_conflicts(merged_df) : DataFrame
-_resolve_join_keys() : list<str>
merge → Gold
optional validation
EnrichmentCrossValidator
-_logger: LoggerPort
-_config: CrossValidationConfig
+validate(merged_df) : CrossValidationReport
CompositeCheckpointManager
-_checkpoint_dir: str
-_logger: LoggerPort
+save(state) : None
+load() : CompositeCheckpointState
+clear() : None
checkpoint / resume
CompositePreflightValidator
-_delta_reader: DataReaderPort
-_logger: LoggerPort
+validate(config) : list<str>
pre-run validation
CompositeResult
+run_id: RunID
+state: CompositePipelineState
+seed: SeedResult
+dependencies: list<DependencyResult>
+enrichments: list<EnrichmentResult>
+merge: MergeResult
+started_at: datetime
+finished_at: datetime
+has_warnings: bool
produces

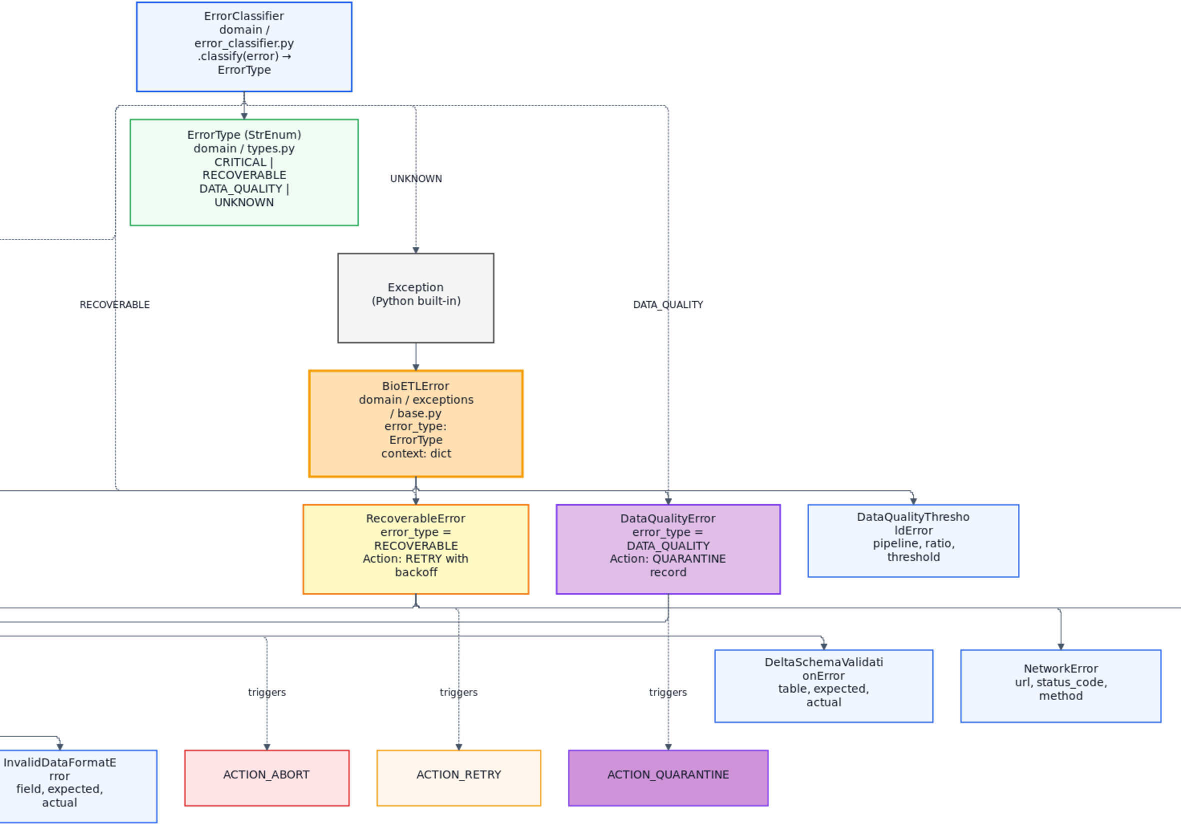

InvalidStateError
current_state,
attempted_operation
PolicyViolationError
policy,
violation_details
LockLostError
lock_key, run_id
LockAcquisitionError
lock_key, holder,
ttl
CheckpointConflictError
pipeline,
existing_run_id

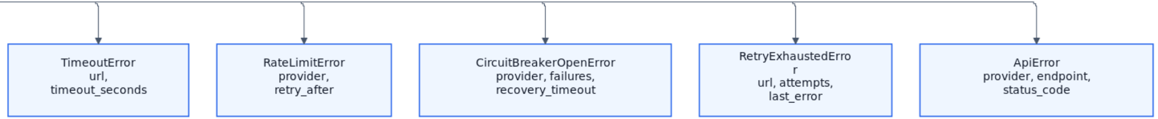

CRITICAL
CriticalError
error_type =
CRITICAL
Action: ABORT
pipeline
MergeConflictError
table,
conflicting_keys
AuthFailureError
provider,
status_code
MetricsServerError
host, port
RunnerAlreadyExecute
dError
InfrastructureErro
r
component,
operation


StorageQuotaExceededError
path, current_size, limit
DeltaTransactionError
table, operation
ExternalServiceError
service, operation
ServiceUnavailableError
service, status_code
RateLimitExceededError
service, retry_after
ServiceAuthenticationError
service, status_code
DataValidationError
service, field, message
BucketNotFoundError
bucket

StorageError
path, operation
TableNotFoundError
table
UploadError
path, size
SchemaEvolutionError
table, old_schema,
new_schema
BronzeValidationEr
ror
path, reason
CachedBronzeEmptyE
rror
path

ValidationError
field, message,
record_id
DeltaWriteConflict
Error
table, version
DeltaOptimizeError
table
SchemaViolationError
schema_name,
violations: list
MissingRequiredFie
ldError
field, entity_type


ErrorClassifier
domain / error_classifier.py
.classify(error) → ErrorType
ErrorType (StrEnum)
domain / types.py
CRITICAL | RECOVERABLE
DATA_QUALITY | UNKNOWN
UNKNOWN
RECOVERABLE
DATA_QUALITY
Exception
(Python built-in)
BioETLError
domain / exceptions / base.py
error_type: ErrorType
context: dict
RecoverableError
error_type = RECOVERABLE
Action: RETRY with backoff
DataQualityError
error_type = DATA_QUALITY
Action: QUARANTINE record
DataQualityThresholdError
pipeline, ratio, threshold
DeltaSchemaValidationError
table, expected, actual
NetworkError
url, status_code, method
triggers
triggers
triggers
InvalidDataFormatError
field, expected, actual
ACTION_ABORT
ACTION_RETRY
ACTION_QUARANTINE


TimeoutError url, timeout_seconds
RateLimitError provider, retry_after
CircuitBreakerOpenError provider, failures, recovery_timeout
RetryExhaustedError url, attempts, last_error
ApiError provider, endpoint, status_code